

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО  
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ

Казанский государственный энергетический университет

Кафедра Инженерная кибернетика

**ПРОЕКТИРОВАНИЕ МАТЕМАТИЧЕСКОГО  
ОБЕСПЕЧЕНИЯ ПРЕДПРИЯТИЙ**

Учебное пособие  
для студентов и магистров

Казань, 2022

Проектирование математического обеспечения предприятий. Учебное пособие для студентов / Составитель: Смирнов Ю.Н., Казань: Изд-во КГЭУ, 2022. - 145 с.

Рецензенты: д.ф.-м.н., профессор Габбасов Н.С.  
к.ф.-м.н., доцент Углов А.Н.

Учебное пособие разработано на кафедре «Математическое моделирование и информационные технологии в экономике» и предназначено для использования в учебном процессе студентами очной формы обучения по дисциплине «Проектирование математического обеспечения предприятий».

Учебное пособие содержит описание основных понятий и принципов методологии проектирования математического обеспечения предприятий. В нем рассмотрены принципы и этапы проектирования информационно-математической модели деятельности предприятия, а также способы разработки имитационной информационно-математической модели в среде визуального проектирования SIMULINK.

Кроме того, рассматривается моделирование временной динамики развития бизнес-процессов и методология управления бизнес-процессами.

Печатается по решению методической комиссии экономического факультета Камской государственной инженерно-экономической академии

© Казанский государственный энергетический университет,  
2022 г.

# СОДЕРЖАНИЕ

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| 1. МЕТОДОЛОГИЯ ПРОЕКТИРОВАНИЯ МАТЕМАТИЧЕСКОГО ОБЕСПЕЧЕНИЯ .....               | 5  |
| 1.1. Состав математического обеспечения.....                                  | 5  |
| 1.2. Методология проектирования математического обеспечения предприятия ..... | 6  |
| 1.3. Общая ИММ деятельности предприятия .....                                 | 10 |
| 1.4. Имитационная ИММ деятельности предприятия .....                          | 11 |
| 1.5. Структура информационно-математической модели .....                      | 14 |
| 1.6. Временная динамика развития бизнес-процесса и ее моделирование .....     | 16 |
| 1.7. Методология управления бизнес-процессом с использованием ИММ.....        | 17 |
| 1.8. Математические модели и методы решения аналитических задач               | 19 |
| 2. ПРИМЕНЕНИЕ СРЕДЫ MATLAB ДЛЯ РЕАЛИЗАЦИИ МАТЕМАТИЧЕСКИХ МОДЕЛЕЙ ЗАДАЧ.....   | 21 |
| 2.1. Математические функции MATLAB.....                                       | 21 |
| 2.2. Основные приемы работы с векторами и матрицами .....                     | 23 |
| 2.2.1. Ввод векторов и матриц .....                                           | 23 |
| 2.2.2. Доступ к элементам векторов и матриц.....                              | 24 |
| 2.2.3. Основные матричные операции .....                                      | 25 |
| 2.2.4. Создание матриц специального вида .....                                | 27 |
| 2.3. Решение задач линейного программирования.....                            | 31 |
| 2.4. Решение задач целочисленного линейного программирования.....             | 35 |
| 2.5. Реализация математических моделей при помощи m-функций .....             | 37 |
| 2.5.1. Синтаксис и вызов m-функций.....                                       | 37 |
| 2.5.2. Типы данных .....                                                      | 39 |

|        |                                                                                                   |     |
|--------|---------------------------------------------------------------------------------------------------|-----|
| 2.5.3. | Операторы .....                                                                                   | 40  |
| 2.5.4. | Программирование m-функций .....                                                                  | 41  |
| 2.5.5. | Проверка входных параметров и выходных значений m-функций .....                                   | 45  |
| 2.5.6. | Принципы решения задач линейного программирования в MATLAB .....                                  | 46  |
| 2.6.   | Интегрирование MATLAB и MS Excel. Экспорт и импорт .....                                          | 52  |
| 2.7.   | Интегрирование MATLAB и баз данных MS SQL Server. Экспорт и импорт .....                          | 54  |
| 2.8.   | Визуализация данных .....                                                                         | 56  |
| 2.8.1. | Двумерные графики и диаграммы .....                                                               | 57  |
| 2.8.2. | Трехмерные графики и диаграммы .....                                                              | 65  |
| 2.8.3. | Надписи, оформление и пояснения к графикам .....                                                  | 81  |
| 3.     | ИСПОЛЬЗОВАНИЕ ПАКЕТА SIMULINK ДЛЯ СОЗДАНИЯ ИМИТАЦИОННОЙ ИНФОРМАЦИОННО-МАТЕМАТИЧЕСКОЙ МОДЕЛИ ..... | 93  |
| 3.1.   | Основные приемы работы в среде Simulink .....                                                     | 93  |
| 3.2.   | Библиотека Simulink .....                                                                         | 94  |
| 3.3.   | Использование S-функций для реализации математических моделей .....                               | 106 |
| 4.     | ПРИМЕР РАЗРАБОТКИ ИМИТАЦИОННОЙ ИММ .....                                                          | 108 |
| 4.1.   | Постановка задачи .....                                                                           | 108 |
| 4.2.   | Проектирование и разработка базы данных SQL Server ... ..                                         | 108 |
| 4.3.   | Программирование m-функций решения задач, входящих в состав ИИММ .....                            | 117 |
| 4.4.   | Разработка временной динамики работы ИИММ в среде SIMULINK .....                                  | 141 |
| 5.     | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                                                            | 145 |

# **1. МЕТОДОЛОГИЯ ПРОЕКТИРОВАНИЯ МАТЕМАТИЧЕСКОГО ОБЕСПЕЧЕНИЯ**

## **1.1. Состав математического обеспечения**

Математическое обеспечение предприятия – это математические модели, методы решения, алгоритмы, методики расчетов, используемые при решении различных задач предприятия (бизнес – задач). При этом в состав математического обеспечения входят:

1. Модели и методы линейного программирования
  - 1.1. модель использования ресурсов (планирования производства);
  - 1.2. модель планирования производства с ограничением на технико-экономические параметры;
  - 1.3. модель размещения производства;
  - 1.4. модель использования взаимозаменяемых ресурсов;
  - 1.5. модель решения транспортной задачи;
  - 1.6. модель решения задачи коммивояжера.
2. Модели и методы динамического программирования
  - 2.1. модель распределения инвестиционных ресурсов;
  - 2.2. модель стратегического планирования;
  - 2.3. модель замены оборудования.
3. Модели и методы решения нелинейных оптимизационных задач
  - 3.1. метод градиентного спуска;
  - 3.2. метод наискорейшего спуска;
  - 3.3. метод общего поиска;
  - 3.4. метод покоординатного спуска.
4. Модели и методы сетевого планирования и управления
5. Модели и методы систем массового обслуживания
  - 5.1. модели СМО с отказами;
  - 5.2. модели СМО с неограниченной очередью;
  - 5.3. модели СМО с ограниченной длиной очереди;
  - 5.4. модели СМО с ограниченным временем ожидания.
6. Модели и методы управления запасами
7. Модели и методы статистического анализа и прогнозирования (корреляционно-регрессионный анализ, трендовый анализ, эконометрическое моделирование)

8. Методы и модели маркетинга
9. Модели и методы финансового менеджмента
10. Модели и методы производственного менеджмента
11. Модели и методы организационного менеджмента

## 1.2. Методология проектирования математического обеспечения предприятия

Методология ПМО предприятия основана на методологии процессно-задачного реинжиниринга бизнес-процессов предприятия. Эта методология основана на представлении предприятия как совокупности взаимосвязанных бизнес-процессов, а бизнес-процесса – как совокупности последовательно – параллельно соединенных бизнес – задач. Все бизнес – задачи могут быть разбиты на 4 класса: аналитические, организационные, технологические и учетные. Эти классы задач связаны между собой и образуют полный цикл решения бизнес – задач предприятия. Схематично взаимосвязь четырех классов задач представлена на рисунке:

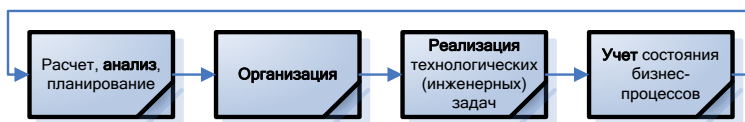


Рисунок 1.1. – Организационно-логическая сущность бизнес-процессов

Подавляющее большинство аналитических задач решаются с применением математических моделей, методов, алгоритмов и методик расчета. Решение учетных задач формирует исходные данные для решения аналитических задач.

Основные определяющие этапы ПМО предприятия и создания информационно-математических моделей деятельности предприятия:

### 1. Назначение, цели и состав бизнес-процессов

В соответствии с целями и задачами бизнес-процессов предприятия определяются цели и задачи математического обеспечения предприятия и его отдельных моделей и методов,

на их основе формируются целевые функции математических моделей и методов.

## *2. Основные задачи бизнес-процессов предприятия*

Выделение бизнес – задач производится методом потоков на основе принципов полноты, детерминированности, связности, автономности, ресурсоемкости, продуктивности и динамичности. Принципы ресурсоемкости и продуктивности ложатся в основу метода потоков выделения бизнес – задач. Перечисленным восьми принципам должны удовлетворять все бизнес-процессы и бизнес – задачи.

## *3. Классификация задач бизнес-процессов предприятия*

Четыре класса бизнес - задач определяют разные классы инструментариев их решения. Для решения аналитических задач используются математические модели и методы, алгоритмы и методики расчета; для организационных задач – средства информационных систем; для технологических – маршрутно-технологические карты, технологическое оборудование; для учетных - средства информационных систем в соответствии с учетной политикой предприятия. Кроме того, классификация задач бизнес-процессов позволяет определить состав задач общей информационно-математической модели деятельности предприятия.

## *4. Определение исполнителей бизнес-задач и диаграмма ответственности*

На этом этапе корректируется и оптимизируется организационная структура управления предприятием. Одним из шагов этой оптимизации является нормализация организационной структуры или приведение во взаимно-однозначное соответствие организационной структуры и бизнес-задач предприятия. Другим шагом такой оптимизации является оптимизация иерархической структуры управления с использованием математических моделей. Кроме того, постановка бизнес-задач, определение показателей эффективности их решения, анализ результатов решения можно

эффективно осуществить только с участием владельцев и исполнителей этих задач.

#### *5. Организационно-логическая сущность бизнес-процессов предприятия*

Может быть представлена в виде диаграммы, где основными элементами являются бизнес-задачи, и реализована по приведенной выше схеме (рис. 1.1). Организационно-логическая схема (ОЛС) позволяет построить логическую (временную) взаимосвязь бизнес-задач и, соответственно, используемых для их решения математических методов, алгоритмов, методик расчета. ОЛС бизнес-процессов совместно с временным нормированием бизнес-задач и приведенной схемой позволяет определить циклы решения задач бизнес-процессов.

#### *6. Таблица связей задач бизнес-процессов предприятия (таблица потоков)*

Такая таблица может быть обобщенной, отражает все возможные потоки, и частной, отражающей только потоки определенного вида. Она позволяет определить входные ресурсы для решения бизнес-задач и их выходные продукты, установить все информационные связи между используемыми математическими методами, моделями, алгоритмами и методиками расчетов. Таблица потоков представляет собой продуктовую модель деятельности предприятия. Совершенствование любого сегмента деятельности предприятия или любого бизнес-процесса затрагивает эту продуктовую модель. Организационный менеджмент предприятия должен быть построен и реализован в соответствии с оптимизированной продуктовой моделью.

#### *7. Схема связей задач бизнес-процессов предприятия (диаграмма потоков)*

Строится на основе таблицы потоков и в соответствии с организационно-логическими связями между задачами. Диаграмма потоков позволяет построить графический образ информационно-математической модели (ИММ)



функционирования как отдельных бизнес-процессов, так и всего предприятия. Таким образом, ИММ строится на основе ОЛС и диаграммы потоков.

#### *8. Документирование бизнес-задач*

На этом этапе производится подробное описание бизнес-задач, осуществляется их постановка, определяются границы допустимых условий и параметров, влияющих на результаты решения, формируется конечная цель решения бизнес-задач. Все это позволяет в последующем построить ИММ модель решения этих бизнес-задач.

#### *9. Показатели эффективности бизнес-процессов решения бизнес-задач*

Различаются показатели эффективности решения самих бизнес-задач, а также агрегированные показатели эффективности бизнес-процессов. Эти показатели зависят от принятых управленческих решений и входных потоков бизнес-задач, а также от схемы реализации (ОЛС) бизнес-задач и бизнес-процессов. Показатели эффективности служат основой составления целевой функции в мат. моделях, дают оценку качеству принятия управленческих решений. Основываясь на указанные зависимости показателя эффективности бизнес-процессов и бизнес-задач, можно провести имитационное исследование и определить наиболее приемлемые параметры решения бизнес-задач и функционирования бизнес-процессов.

#### *10. Разработка стандарта управления предприятием*

Такой стандарт включает в себя полное документирование всех бизнес-задач и бизнес-процессов, технологию управления, правила и законы принятия решений. Этот стандарт влияет, с одной стороны, на будущую ИММ деятельности предприятия, с другой стороны, построенная ИММ может использоваться для определения правила законов принятия решений, для изменения положений стандарта оперативного, среднесрочного и стратегического управления. Кроме того, существует полная взаимозависимость стандарта управления предприятием с системой менеджмента качества. Система менеджмента

качества является одним из основных элементов стандарта управления предприятием.

### **1.3. Общая ИММ деятельности предприятия**

Существуют четыре класса бизнес-задач предприятия, каждый из которых можно разделить на подклассы.

Аналитические задачи имеют подклассы задач планирования, анализа и прогнозирования, расчета. Математические модели и методы этих подклассов аналитических задач могут существенно отличаться. Первый, как правило, использует оптимизационные модели, второй – статистические модели, третий – различные методики и алгоритмы. Кроме того, эти подклассы аналитических задач можно выделить в соответствии со структурой бизнес-процесса, так как в разных бизнес-процессах решаются разные бизнес-задачи с использованием различных методов и моделей.

Класс организационных задач можно разбить на подклассы организационных задач бизнес-процессов в соответствии со структурой бизнес-процесса. Решение организационных задач формирует директивные документы, регламентирующие все параметры реализации технологических задач. Организационные задачи решаются в соответствии с аналитическими плановыми задачами, и приводит [их решение] к согласованию всех ресурсов по времени, объему, месту потребления.

Классы технологических задач можно представить в виде подклассов этих задач в соответствии со структурой бизнес-процессов. Эти задачи решаются в соответствии с организационно-распорядительной и технической документацией.

Класс учетных задач включает подклассы учетных задач бизнес-процессов. Эти учетные задачи включают в себя задачи финансового и управленческого учета.

Предположим, весь плановый период разбит на временные отрезки. Эти временные отрезки можно называть периодом оперативного планирования, а суммы этих отрезков – периодом среднесрочного планирования. Решение всех аналитических в начальный период планирования может формировать технико-

экономические показатели деятельности предприятия на краткосрочный период – период оперативного планирования.

Предположим все организационные и технологические за указанный период решены в полном объеме, тогда плановые экономические показатели в конце периода оперативного планирования являются результатом возможного решения всех учетных задач, поэтому их можно принимать в качестве учетных данных в конце оперативного периода планирования. Таким образом, можно решить все аналитические задачи на следующем временном отрезке и тем самым сформировать траекторию развития предприятия за среднесрочный период.

Общая ИММ деятельности предприятия – это система всех аналитических и учетных бизнес-задач предприятия, использующая математические модели, методы, алгоритмы, методики их решения, и построенная как последовательность временных циклов траектории развития предприятия.

Общая ИММ интегрирует в себе множество мат. моделей с разными критериями и условиями. Наборы этих критериев и условий определяют адекватность этой модели к реальным процессам предприятия. В некоторых случаях многокритериальные общие ИММ можно свести к однокритериальной задаче с интегрированными всеми условиями решения всех бизнес-задач. Решение общей ИММ возможно только в структуре единой ИС предприятия, если эта ИС включает в себя методы, модели, алгоритмы решения бизнес-задач.

#### **1.4. Имитационная ИММ деятельности предприятия**

ИМММ деятельности предприятия – это часть ОИММ, построенная на базе основных бизнес-процессов. Она предназначена для исследования отдельных сегментов деятельности предприятия и построения оптимальной траектории их развития.

ИМММ может быть реализована в виде единого программного комплекса, работающего на реальной информационной базе предприятия.

Состав задач ИИММ включает в себя аналогично ОИММ аналитические и учетные задачи выделенных бизнес-процессов.

Создание ИИММ деятельности предприятия основано на следующих этапах:

1. Определение базовых бизнес-процессов. Таковыми бизнес-процессами могут являться «Финансы», «МТО», «Производство», «Маркетинг», «Сбыт». Указанные бизнес-процессы обеспечивают формирование основных технико-экономических показателей предприятия. В создаваемой модели ресурсное обеспечение деятельности предприятия отражается в виде ограничения в математических моделях аналитических задач выделенных бизнес-процессов.

2. Определение состава целевых задач ИИММ. Эти задачи должны быть определяющими для бизнес-процесса, для его выходного продукта. Возможно выделение неполного состава аналитических и учетных задач бизнес-процесса. Например, задача размещения производства имеет существенно меньшее значение, чем задача оптимального планирования объемов производства.

3. Построение организационно-логической схемы решения задач ИММ. Эта схема строится на основе организационно-логической сущности бизнес-процессов и имеет существенно меньшее количество задач. В отдельных случаях в этой схеме может быть предусмотрено определенное количество внутренних циклов решения за плановый период.

4. Построение таблицы и схемы потоков. Такая таблица и схема отражают информационные потоки между задачами ИММ. Схема связей по потокам совместно со схемой логических связей задач ИММ формирует структуру и алгоритм ИИММ.

5. Определение структуры входных данных для задач ИММ. Алгоритм решения задач в значительной степени предопределяет структуру входных данных. Реальные исходные данные формируются на основе решения учетных задач бизнес-процессов.

6. Построение оптимизационных математических моделей решения аналитических задач. Оптимизация решения каждой задачи формирует не только прямой, но и синергетический эффект (системный). Как эффект взаимосвязей бизнес-задач. В

результате использования различных оптимизационных моделей ИММ становится многокритериальной. ИММ представляется в виде последовательности решаемых задач. В отдельных случаях возможно построение интегрированной математической модели решения всех выделенных задач с единственной целевой функцией и множеством ограничений. Создание интегральной математической модели – проблема сложная и трудоемкая, но её решение может дать существенный экономический эффект.

7. Определение структуры выходных данных. Результаты решения каждой задачи могут представлять автономный интерес для управления отдельным бизнес-процессом. Однако всегда существуют такие результирующие показатели, которые отражают качество реализации всех бизнес-процессов. Такие показатели могут быть объемными (количественными) и финансовыми.

8. Определение временных циклов решения задач ИММ. Все входные и выходные данные в задачах ИММ имеют временную точку, поэтому циклы решения задач отражают временную динамику развития бизнес-процесса.

9. Выбор программной среды реализации и создание программного обеспечения ИММ. Как правило, для поддержки данных используется какая-либо СУБД (возможно из состава ИС предприятия). Для решения математических задач используются развитые математические пакеты программ. Для интеграции всех данных используется какая-либо среда программирования, создания приложений. Таким образом, программное обеспечение ИИММ представляет собой гетерогенную среду инструментарий.

10. Тестирование, проверка адекватности ИИММ, определение границы допустимых применений для исследования и принятия решений. Проверка адекватности модели производится на основе тривиальных и известных решений. Граница допустимых применений определяется на основе многих расчетов и анализа результатов. Общая методология проверки ИИММ основана на сравнительном анализе результатов прошлых периодов.

## 1.5. Структура информационно-математической модели

Все оптимизационные модели деятельности предприятия формируются в основном на базе аналитических задач бизнес-процессов. Учетные задачи используются как исходные данные для решения оптимизационных задач.

Таким образом, в состав ИММ входит 2 класса задач: аналитические и учетные.

Решение оптимизационной модели строится по следующей схеме:

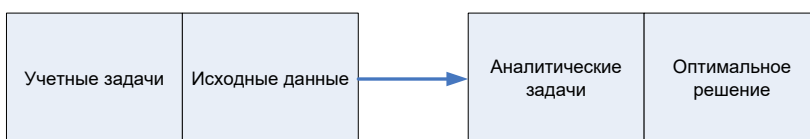


Рисунок 1.2. – Схема решения оптимизационной модели

Взаимосвязь по потокам этих двух классов задач строится на основе связей бизнес-задач по потокам с исключением организационных и технологических задач. При этом предполагается точное решение организационных и технологических задач в соответствии с построенным планом или сценарием развития бизнес-процесса.

По приведенной схеме результаты решения аналитических задач рассматриваются как результаты решения учетных задач в предположении точной реализации всех планов. За плановый период последовательность решения аналитических задач определяется потоковыми связями этих задач. Например, можно построить следующую ИИММ, которая включает следующие аналитические задачи:

1. Бюджетирование расходов.
2. Анализ спроса.
3. Анализ складских запасов.
4. Анализ заказов.
5. Анализ цен на продукцию.
6. Анализ цен на ресурсы.
7. Планирование оптимальных объемов производства.

## 8. Планирование ресурсного обеспечения производства (оптимизация складских запасов).

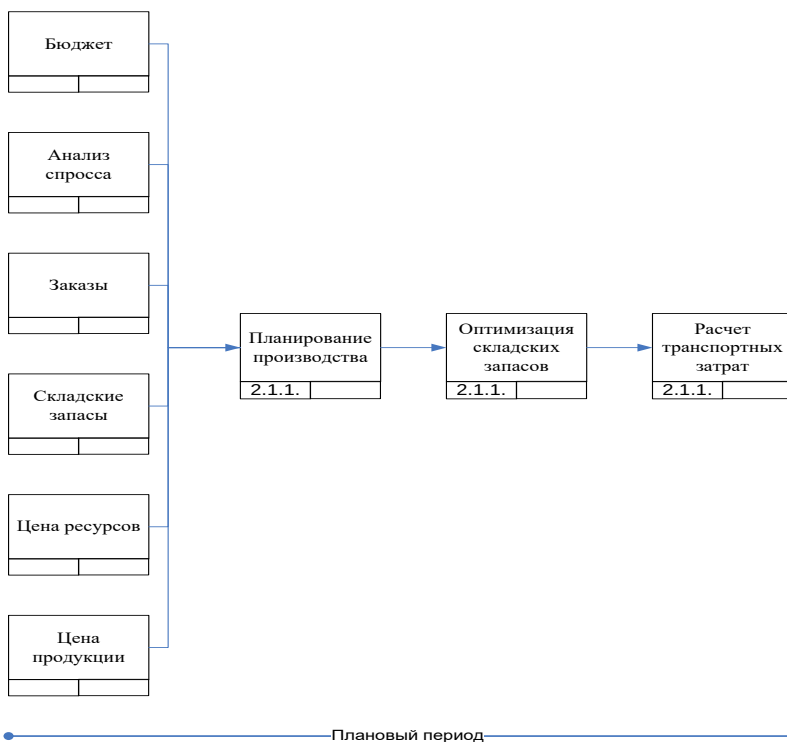


Рисунок 1.3. – Имитационная информационно-математическая модель

В результате решения оптимизационной модели определяется динамика изменения основных показателей за плановый период. В некоторых случаях возможно существование циклов решения задач, входящих в ИММ. Такие итерации используются для оптимизации некоторого агрегированного показателя эффективности решения нескольких оптимизационных задач. Это связано с использованием нескольких целевых функций в ИММ. Например, в качестве такого агрегированного показателя можно использовать суммарные плановые затраты на реализацию всего бизнес-процесса.

Множество сценариев развития этого бизнес-процесса формирует множество этих показателей. Окончательный выбор сценария осуществляется по определенным критериям, включая агрегированный показатель.

Использование итерационных циклов дает возможность упрощать используемые оптимизационные модели. Создание единой интегрированной оптимизационной модели для всего бизнес-процесса является задачей трудоемкой и сложной, хотя и дает наилучшие управленческие решения.

Предполагается, что использование «приближенной» многокритериальной ИММ для бизнес-процесса дает достаточно точные результаты и экономически оправдано.

### **1.6. Временная динамика развития бизнес-процесса и ее моделирование**

Краткосрочное управление позволяет выйти на требуемые показатели в указанный срок. Наиболее актуальной задачей управления является управление динамикой развития процессов в среднесрочном периоде. Управление этой динамикой может основываться на следующей методологии:

1. Определяется краткосрочный период планирования. Как правило, в качестве краткосрочного периода используется месяц. Это связано с основными периодами затрат материальных, трудовых, амортизационных. Теоретически минимальный размер этого периода может быть установлен как день.

2. Строится ИИММ основных бизнес-процессов за краткосрочный период. Эта модель позволяет на основе учетных данных найти оптимальные показатели на конец краткосрочного периода. Эти результаты принимаются в качестве учетных показателей (данных) на конец указанного периода.

3. Выбирается продолжительность среднесрочного планирования. Этот период кратен периоду краткосрочного планирования. Как правило, период среднесрочного планирования составляет год. Продолжительность среднесрочного планирования связывают с календарным годом, отчетными периодами и т.д.



4. Строится сценарий развития бизнес-процесса в виде цепи информационно-математических моделей, решаемых в краткосрочном периоде. Этот сценарий может быть реализован по следующей схеме:

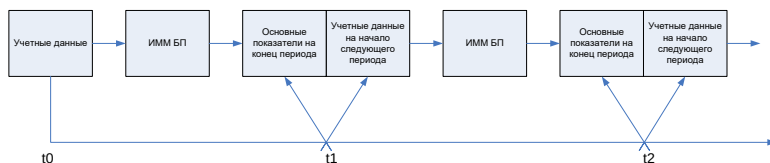


Рисунок 1.4. – Сценарий развития бизнес-процесса

5. На каждом периоде используется единая методология решения ИММ и единая методология формирования основных показателей.

6. Строится ступенчатая функция основных показателей, характеризующая динамику развития процессов, вычисляются агрегированные показатели среднесрочного периода (суммарная прибыль, суммарный объем производства, суммарные затраты, средняя себестоимость продукции и т.д.). Эти показатели являются критериями выбора оптимального сценария развития.

### 1.7. Методология управления бизнес-процессом с использованием ИММ

Планирование на среднесрочный период позволяет построить оптимальную траекторию развития процессов. Однако в процессе реализации могут быть различные отклонения плановых и фактических показателей, поэтому в начале каждого краткосрочного периода возникает необходимость корректировки планов на будущий период с учетом целевых показателей и управления. Общая методология управления бизнес-процессами с использованием ИММ может быть реализована на основе следующего алгоритма:

1. Организовываются и реализовываются бизнес-процессы в соответствии с оптимальным планом на краткосрочном периоде, формируются учетные, фактические данные состояния бизнес-процессов.

2. Производится план-фактный сравнительный анализ, и формируются исходные данные плана на будущий краткосрочный период.

3. С учетом управляющего воздействия на основе ИММ находится оптимальный план на текущий период (находятся целевые показатели на конец периода).

4. Вновь организуются и реализовываются бизнес-процессы, и формируются новые учетные фактические данные, и цикл повторяется по мере реализации бизнес-процессов в разрезе времени.

Эта методология представлена в виде следующей схемы:

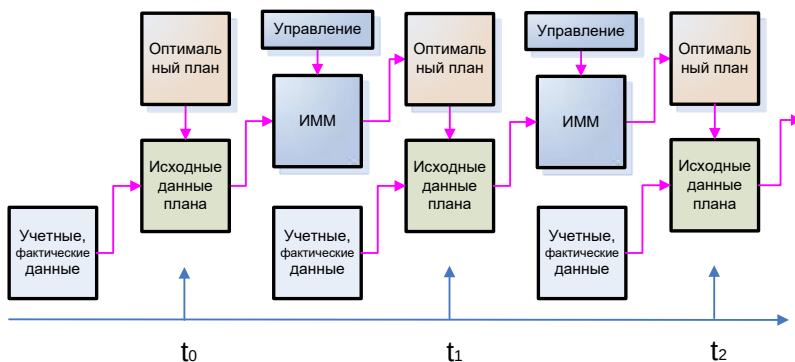


Рисунок 1.5. – Схема управления бизнес процессом

Приведенная диаграмма управления бизнес-процессами имеет 5 уровней:

1. управление, используемое для воздействия на целевые показатели деятельности предприятия на основе среднесрочных и стратегических управленческих решений;

2. оптимальный план – оптимальные показатели бизнес-процессов, выход на которые является конечной целью управления;

3. ИММ – как инструмент формирования оптимального плана;

4. Исходные данные плана, необходимые для решения ИММ;

5. Учетные, фактические данные, позволяющие адаптировать, бизнес-процессы к реальным условиям, к организации и реализации бизнес-процессов (учетные, фактические данные отражают эффективность организации и реализации бизнес-процессов).

На каждом периоде времени процесс управления требует решения однотипных задач, приведенных в схеме, поэтому система управления может быть стандартизована. В этом стандарте управления должны быть документированы все процедуры решения указанных задач. Документированные процедуры должны входить также в систему менеджмента качества. Таким образом, приведенная методология управления гарантирует качество процессов планирования и управления, формирования качественных входных данных и получение качественных, оптимальных целевых показателей.

**Замечание:** для формирования оптимальных траекторий (планов) развития бизнес-процессов на каждом цикле управления могут быть использованы имитационные исследования, позволяющие найти наилучшие управленческие решения.

## **1.8. Математические модели и методы решения аналитических задач**

Существует 3 класса математических моделей и методов, применяемых в экономике и управлении:

### **1. Общие математические модели и методы**

1.1. Модели и методы линейного программирования. Применение линейных моделей оправдано для значительного количества оптимизационных задач в экономике – планирование объемов производства, размещение производства, транспортная задача.

1.2. Модели и методы динамического программирования. Эти модели относятся, как правило, к нелинейным моделям с дискретными точками управления – распределение ресурсов, план замены оборудования.

1.3. Модели и методы решения нелинейных оптимизационных задач.

1.4. Модели и методы сетевого планирования и управления.

1.5. Модели и методы систем массового обслуживания.

1.6. Модели методы статистического анализа и прогнозирования.

1.7. Модели и методы кластерного анализа (выделение группы объектов по свойствам этих объектов).

Вышеприведенные математические модели и методы применимы ко многим бизнес-процессам, для решения их задач.

2. Проблемно-ориентированные математические модели и методы

В большинстве случаев эти методы и модели можно рассматривать как развитие общих методов и моделей применительно отдельным бизнес-процессам или бизнес-задачам.

2.1. Модели и методы маркетинга.

2.2. Модели и методы финансового менеджмента.

2.3. Модели и методы производственного менеджмента.

2.4. Модели и методы организационного менеджмента.

2.5. Модели и методы логистики

2.5.1. Модели и методы закупочной логистики.

2.5.2. Модели и методы складской логистики.

2.5.3. Модели и методы транспортной логистики.

2.5.4. Модели и методы производственной логистики.

2.5.5. Модели и методы сбытовой логистики.

3. Математические модели и методы оптимизации потоков

3.1. Модели и методы оптимизации материальных потоков. Эти модели можно рассматривать как интегрированные модели логистики.

3.2. Модели и методы оптимизации финансовых потоков (логистика финансов).

3.3. Модели и методы оптимизации информационных потоков (модели и методы информационной логистики)

## 2. ПРИМЕНЕНИЕ СРЕДЫ MATLAB ДЛЯ РЕАЛИЗАЦИИ МАТЕМАТИЧЕСКИХ МОДЕЛЕЙ ЗАДАЧ

Язык программирования, используемый в среде MATLAB, является высокоуровневым языком программирования. В его состав входят основанные на *матрицах* структуры данных, широкий спектр функция для работы с ними, интегрированная среда разработки.

### 2.1. Математические функции MATLAB

В системе MATLAB имеется обширная библиотека математических функций. Каждой функции соответствует определенное имя. Функция ставит в соответствие значениям своих аргументов значение результата.

Аргументы функции всегда указываются в круглых скобках после имени функции и, если их больше одного, разделяются запятыми. В качестве аргументов могут использоваться другие функции и любые выражения языка MATLAB (при условии соответствия типов аргументов).

**Элементарная математическая функция** - это, как правило, функция от одной переменной, и в этом случае устанавливается соответствие между массивами значений аргумента и результата.

Аргумент указывается в круглых скобках после имени функции. Имя переменной, которой присваивается значение функции, располагается слева от знака равенства. Если имя присваиваемой переменной не указано, значение функции присваивается служебной переменной **ans**.

Тип результата вычисления математической функции всегда совпадает с типом ее аргумента

К числу математических функций MATLAB относятся:

#### 1. Базовые функции

- ABS - абсолютное значение
- SIGN - вычисление знака числа
- CEIL – функция округления до ближайшего целого числа  $\geq X$

- FIX – функция округления с усечением дробной части числа
- FLOOR - функция округления до ближайшего целого числа  $\leq X$
- ROUND - функции округления до ближайшего целого числа

## 2. Трансцендентные функции

- SQRT - квадратный корень
- EXP - экспоненциальная функция
- LOG - функция натурального логарифма
- LOG2 - функция логарифма по основанию 2

## 3. Тригонометрические функции

- SIN, SINH - функции синуса
- ASIN, ASINH - функции обратного синуса
- CSC, CSCH - функции косеканса
- ACSC, ACSCH - функции обратного косеканса
- COS, COSH - функции косинуса
- ACOS, ACOSH - функции обратного косинуса
- SEC, SECH - функции секанса
- ASEC, ASECH - функции обратного секанса
- TAN, TANH - функции тангенса
- ATAN, ATAN2, ATANH - функции обратного тангенса
- COT, COTH - функции котангенса
- ACOT, ACOTH - функции обратного котангенса

## 4. Функции анализа и обработки данных

- SUM, CUMSUM - суммирование элементов массива
- PROD, CUMPROD - произведение элементов массива
- SORT - сортировка элементов массива по возрастанию
- MAX - определение максимальных элементов массива
- MIN - определение минимальных элементов массива

- MEDIAN - определение срединных значений (медиан) элементов массива
- MEAN - определение средних значений элементов массива
- STD - определение стандартных отклонений элементов массива
- COV - определение ковариационной матрицы элементов массива
- CORRCOEFF - определение коэффициентов корреляции элементов массива

## 2.2. Основные приемы работы с векторами и матрицами

### 2.2.1. Ввод векторов и матриц

Матрица в MATLAB рассматривается как вектор-столбец, каждый элемент которого является вектором-строкой. Для задания вектора необходимо указать его элементы, заключенные в скобки «[ ]»:

```
» A = [2 4 8]
A =
      2      4      8
```

*Примечание.* После завершения ввода матричной структуры, в рабочей области MATLAB отобразится результат ввода. Чтобы этого не происходило, необходимо в конце строки указать символ «;».

Для ввода элементов вектора по определенному правилу можно использовать следующий прием:

```
» x = -5 : 0.5 : 5;
```

В этом случае вектор X будет состоять из всех значений из отрезка [-5 ; 5] с шагом 0.5

Для ввода вектора-столбца необходимо элементы вектора разделить символом «;»:

```

» A = [2; 4; 8]
A =
     2
     4
     8

```

Таким образом, если учесть, матрица – это вектор, элементами которого являются вектора, то для ввода матрицы необходимо записать следующее:

```

» B = [2 4 8; 1 3 5]
B =
     2     4     8
     1     3     5

```

или

```

» B = [[2 ; 1] [4 ; 3] [8 ; 5]]
B =
     2     4     8
     1     3     5

```

### 2.2.2. Доступ к элементам векторов и матриц

Доступ к элементам матрицы осуществляется при помощи двух индексов - номеров строки и столбца, заключенных в круглые скобки:

```

» B (2, 3)
ans =
     5

```

Для выделения из матрицы столбца или строки следует в качестве одного из индексов использовать номер столбца или строки матрицы, а другой индекс заменить двоеточием

```

» A = B (2, :)
A =

```



```

1      3      5
» A = B (:, 3)
A =
      8
      5

```

Также можно осуществлять выделение блоков матриц при помощи двоеточия:

```

» C = [1 2 0 2; 4 10 12 5; 0 11 10 5; 9 2 3 5]
C =
      1      2      0      2
      4     10     12      5
      0     11     10      5
      9      2      3      5

» U = C (2:3, 2:3)
U =
     10     12
     11     10

```

### 2.2.3. Основные матричные операции

При использовании матричных операций следует помнить, что для сложения или вычитания матрицы должны быть одного размера, а при перемножении число столбцов первой матрицы обязано равняться числу строк второй матрицы. Сложение и вычитание матриц, так же как чисел и векторов, осуществляется при помощи знаков плюс и минус, а умножение — знаком звездочка – «\*»:

```

» A = [2 4 3; 5 8 5]
A =
      2      4      3
      5      8      5

```

```
» B = [3 1 7; 6 3 8]
```

```
B =
```

```
     3     1     7
     6     3     8
```

```
» S = A + B
```

```
S =
```

```
     5     5    10
    11    11    13
```

```
» R = A - B
```

```
R =
```

```
    -1     3    -4
    -1     5    -3
```

```
» C = [3 1; 4 3; 2 5]
```

```
C =
```

```
     3     1
     4     3
     2     5
```

```
» P = A*C
```

```
P =
```

```
    28    29
    57    54
```

Умножение матрицы на число тоже осуществляется при помощи звездочки, причем умножать на число можно как справа, так и слева. Возведение квадратной матрицы в целую степень производится с использованием оператора «^».

```
» U = P ^ 2
```

```
U =
```

```
    2437    2378
    4674    4569
```

Для выполнения поэлементных матричных операций перед

оператором указывается знак точка «.»:

```
» D = A. * B
D =
     6     4    21
    30    24    40
```

```
» K = A. ^ 2
K =
     4    16     9
    25    64    25
```

Транспонирование матрицы производится при помощи апострофа «'»

```
» T = A'
T =
     2     5
     4     8
     3     5
```

#### 2.2.4. Создание матриц специального вида

MATLAB позволяет создавать и обрабатывать матрицы специального вида:

ZEROS - формирование массива нулей:

- » P = zeros(n) - формирование массива нулей размерностью n x n
- » P = zeros(m, n) - формирование массива нулей размерностью m x n
- » P = zeros(size(A)) - формирование массива нулей соразмерного массиву A

```
» P = zeros(3)
P =
     0     0     0
     0     0     0
     0     0     0
```

```
» P = zeros(2,4)
```

```
P =
```

```
    0    0    0    0
    0    0    0    0
```

```
» P = zeros(size(A))
```

```
P =
```

```
    0    0    0
    0    0    0
```

**ONES - формирование массива единиц**

» P = ones(n) - формирование массива единиц  
размерностью n x n

» P = ones(m, n) - формирование массива единиц  
размерностью m x n

» P = ones(size(A)) - формирование массива единиц  
соразмерного массиву A

```
» P = ones(3)
```

```
P =
```

```
    1    1    1
    1    1    1
    1    1    1
```

```
» P = ones(2,4)
```

```
P =
```

```
    1    1    1    1
    1    1    1    1
```

```
» P = ones(size(A))
```

```
P =
```

```
    1    1    1
    1    1    1
```

**EYE - формирование единичной матрицы**

» P = eye(n) - формирование единичной матрицы  
размерностью n x n

- » `P = eye (m, n)` - формирование единичной матрицы размерностью  $m \times n$
- » `P = eye (size(A))` - формирование единичной матрицы соразмерного массиву `A`

» `P = eye (3)`

`P =`

```

      1      0      0
      0      1      0
      0      0      1

```

» `P = eye (2, 4)`

`P =`

```

      1      0      0      0
      0      1      0      0

```

» `P = eye (size(A))`

`P =`

```

      1      0      0
      0      1      0

```

**RAND** - формирование массива элементов, распределенных по равномерному закону.

- » `P = rand(n)` - формирует массив размера  $n \times n$ , элементами которого являются случайные величины, распределенные по равномерному закону в интервале (0, 1)
- » `P = rand (m, n)` - формирует массив размера  $m \times n$ , элементами которого являются случайные величины, распределенные по равномерному закону в интервале (0, 1)
- » `P = rand (size(A))` - формирует массив соразмерный с матрицей `A`, элементами которого являются случайные величины, распределенные по равномерному закону в интервале (0, 1)
- » `P = rand` - формирует одно случайное число, подчиняющееся равномерному закону распределения в

интервале (0, 1), которое изменяется при каждом последующем вызове

RANDN - формирование массива элементов, распределенных по нормальному закону.

- »  $P = \text{randn}(n)$  - формирует массив размера  $n \times n$ , элементами которого являются случайные величины, распределенные по нормальному закону с математическим ожиданием 0 и среднеквадратическим отклонением 1
- »  $P = \text{randn}(m, n)$  - формирует массив размера  $m \times n$ , элементами которого являются случайные величины, распределенные по нормальному закону с математическим ожиданием 0 и среднеквадратическим отклонением 1
- »  $P = \text{randn}(\text{size}(A))$  - формирует массив соразмерный с матрицей  $A$ , элементами которого являются случайные величины, распределенные по нормальному закону с математическим ожиданием 0 и среднеквадратическим отклонением 1
- »  $P = \text{randn}$  - формирует одно случайное число, распределенное по нормальному закону с математическим ожиданием 0 и среднеквадратическим отклонением 1, которое изменяется при каждом последующем вызове

DIAG - формирует диагональную матрицу из вектора, располагая элементы по диагонали:

```
» V = [1 2 3]
V =
      1      2      3
» U = diag(V)
U =
      1      0      0
      0      2      0
      0      0      3
```

### 2.3. Решение задач линейного программирования

Для решения задач линейного программирования в среде MATLAB применяется функция *linprog*. Данная функция позволяет найти минимум целевой функции  $f(x)$  при заданных ограничениях на ее аргумент.

Общий вид задачи линейного программирования в MATLAB выглядит следующим образом:

Найти  $\min_x f^T x$  при ограничениях

$$\begin{cases} A \cdot x \leq B \\ Aeq \cdot x = Beq \\ lb \leq x \leq ub \end{cases}$$

В данной модели:

$f$  – вектор коэффициентов при аргументах целевой функции,

$A$  – матрица коэффициентов при аргументах целевой функции, находящихся в левой части системы ограничений в неравенстве,

$B$  – вектор коэффициентов, находящийся в правой части системы ограничений в неравенстве,

$Aeq$  – матрица коэффициентов при аргументах целевой функции, находящихся в левой части системы ограничений в равенстве,

$Beq$  – вектор коэффициентов, находящийся в правой части системы ограничений в равенстве,

$lb$  – вектор – нижний предел изменения решения задачи линейного программирования,

$ub$  – вектор – верхний предел изменения решения задачи линейного программирования.

Синтаксис функции *linprog*:

- »  $x = \text{linprog}(f, A, B)$  - находит  $\min_x f^T x$  при условии, что  $A \cdot x \leq B$ .
- »  $x = \text{linprog}(f, A, B, Aeq, Beq)$  - решает указанные выше задачу при условии дополнительного выполнения

ограничений в виде равенств  $Aeq \cdot x = Beq$ . Если нет неравенств, то устанавливается  $A = []$ ,  $B = []$ .

- »  $x = \text{linprog}(f, A, B, Aeq, Beq, lb, ub)$  - определяет набор нижних и верхних границ для проектируемых переменных, так что решение всегда находится в диапазоне  $lb \leq x \leq ub$ . Если нет равенств, то устанавливается  $Aeq = []$ ,  $Beq = []$ .
- »  $x = \text{linprog}(f, A, B, Aeq, Beq, lb, ub, x0)$  - устанавливает начальную точку как  $x0$ . Эта опция имеет место только для средне-масштабного алгоритма. Принимаемый по умолчанию крупно-масштабный алгоритм игнорирует любую стартовую точку.
- »  $x = \text{linprog}(f, A, B, Aeq, Beq, lb, ub, x0, options)$  - проводит оптимизацию с определенными в структурной опции параметрами оптимизации.
- »  $[x, fval] = \text{linprog}(\dots)$  - возвращает значение целевой функции  $fval$  как решение от  $x: fval = f \cdot x$ .
- »  $[x, lambda, exitflag] = \text{linprog}(\dots)$  - возвращает значение  $exitflag$ , которое содержит описание выходных условий.
- »  $[x, lambda, exitflag, output] = \text{linprog}(\dots)$  - возвращает структурный выход с информацией об оптимизации
- »  $[x, fval, lambda, exitflag, output] = \text{linprog}(\dots)$  - возвращает структурную  $lambda$ , чьи поля включают в себя множители Лагранжа как решение от  $x$ .

Выходные аргументы функции содержат общее описание возвращаемых *linprog* аргументов. В этом разделе приводятся общие специфические детали для величин *exitflag*, *lambda* и *output*:

Таблица 2.1. – Выходные аргументы функции *linprog*

|                 |                                                                                                                                                                                                                                                                                                                                  |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>exitflag</i> | <p>Описывает выходные условия.</p> <ul style="list-style-type: none"> <li>• <math>&gt; 0</math> - данная функция сходится к решению по <math>x</math></li> <li>• <math>0</math> - максимальное число оценки функции или итерации было превышено</li> <li>• <math>&lt; 0</math> - функция не сходится к некому решению</li> </ul> |
| <i>lambda</i>   | Структура, которая содержит множители Лагранжа                                                                                                                                                                                                                                                                                   |



|               |                                                                                                                                                                                                                                                                                                      |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               | при решении по x (разделенном по типам условий).<br>Поле структуры: <ul style="list-style-type: none"> <li>• lower - нижние границы lb</li> <li>• upper - верхние границы ub</li> <li>• ineqlin - линейные неравенства</li> <li>• eqlin - линейные равенства</li> </ul>                              |
| <i>output</i> | Структура, которая содержит информацию об оптимизации. Поле структуры: <ul style="list-style-type: none"> <li>• iterations - число выполненных итераций</li> <li>• algorithm - используемый алгоритм</li> <li>• cgitations - число PCG итераций (только для крупно-масштабного алгоритма)</li> </ul> |

Пример решения задачи линейного программирования в MATLAB:

Найти такое X, что целевая функция

$$f(x) = -5x_1 - 4x_2 - 6x_3 \rightarrow \min$$

при условии, что

$$\begin{cases} x_1 - x_2 + x_3 \leq 20 \\ 3x_1 + 2x_2 + 4x_3 \leq 42 \\ 3x_1 + 2x_2 \leq 30 \\ x \geq 0 \end{cases}$$

Решение.

1. Ввод значений коэффициентов целевой функции:

```

» f = [-5; -4; -6]
f =
    -5
    -4
    -6

```

## 2. Ввод матрицы коэффициентов в левой части системы ограничений

```
» A = [1 -1 1; 3 2 4; 3 2 0]
```

```
A =
```

```
     1     -1     1
     3      2     4
     3      2     0
```

## 3. Заполнение вектора коэффициентов в правой части системы ограничений

```
» B = [20; 42; 30]
```

```
B =
```

```
    20
    42
    30
```

## 4. Ввод ограничения на неотрицательность решения:

```
» lb = zeros(3,1)
```

```
lb =
```

```
     0
     0
     0
```

## 5. Нахождение оптимального решения

```
» [x,fval]=linprog(f,A,b,[],[],lb,[])
```

```
Optimization terminated.
```

Оптимальное решение задачи:

```
x =
```

```
    0.0000
   15.0000
    3.0000
```

Значение целевой функции при оптимальном решении

```
fval =  
      -78.0000
```

## 2.4. Решение задач целочисленного линейного программирования

Для решения задач целочисленного линейного программирования в среде MATLAB применяется функция *bintprog*. Данная функция позволяет найти минимум целевой функции  $f(x)$  при заданных ограничениях на ее аргумент.

Общий вид задачи линейного программирования в MATLAB выглядит следующим образом:

Найти  $\min_x f^T x$  при ограничениях

$$\begin{cases} A \cdot x \leq B \\ Aeq \cdot x = Beq \end{cases}$$

В данной модели:

$f$  – вектор коэффициентов при аргументах целевой функции,

$A$  – матрица коэффициентов при аргументах целевой функции, находящихся в левой части системы ограничений в неравенстве,

$B$  – вектор коэффициентов, находящийся в правой части системы ограничений в неравенстве,

$Aeq$  – матрица коэффициентов при аргументах целевой функции, находящихся в левой части системы ограничений в равенстве,

$Beq$  – вектор коэффициентов, находящийся в правой части системы ограничений в равенстве,

$x$  - целочисленный вектор решения, компоненты которого принимают значения 0 или 1

Синтаксис функции *bintprog*:

»  $x = \text{bintprog}(f, A, B)$  - находит  $\min_x f^T x$  при условии, что  $A \cdot x \leq B$ .

- »  $x = \text{bintprog}(f, A, B, Aeq, Beq)$  - решает указанную выше задачу при условии дополнительного выполнения ограничений в виде равенств  $Aeq \cdot x = Beq$ . Если нет неравенств, то устанавливается  $A = []$ ,  $B = []$ .
- »  $x = \text{bintprog}(f, A, B, Aeq, Beq, x0, options)$  - проводит оптимизацию с определенными в структурной опции параметрами оптимизации.
- »  $[x, fval] = \text{bintprog}(\dots)$  - возвращает значение целевой функции  $fval$  как решение от  $x: fval = f \cdot x$ .
- »  $[x, lambda, exitflag] = \text{bintprog}(\dots)$  - возвращает значение  $exitflag$ , которое содержит описание выходных условий.

Выходные аргументы функции содержат общее описание возвращаемых *bintprog* аргументов. В этом разделе приводятся общие специфические детали для величин *exitflag*, и *output*:

Таблица 2.2. – Выходные аргументы функции *bintprog*

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>exitflag</i> | <p>Некое целое число, идентифицирующее причину остановки алгоритма. Далее приводится перечень принимаемых значений и соответствующих причин остановки алгоритма.</p> <ul style="list-style-type: none"> <li>• 1 - Функция сошлась к некому решению <math>x</math>.</li> <li>• 0 - Число итераций превысило значение <code>options.MaxIter</code>.</li> <li>• -2 - Данная задача не имеет решения.</li> <li>• -4 - Число перебранных узлов превышает значение <code>options.MaxNodes</code>.</li> <li>• -5 - Время перебора превышает значение <code>options.MaxTime</code>.</li> <li>• -6 - Число итераций решателя LP для некого узла при решении задачи LP-релаксации превысило значение <code>options.MaxRLP</code>.</li> </ul> |
| <i>output</i>   | <p>Структура, которая содержит информацию об оптимизации. Поле структуры:</p> <ul style="list-style-type: none"> <li>• <code>iterations</code> - число выполненных итераций</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

|  |                                                                                                                                                                                                                                                |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <ul style="list-style-type: none"> <li>• algorithm - используемый алгоритм</li> <li>• nodes - Число узлов перебора.</li> <li>• Time - Превышение времени работы алгоритма.</li> <li>• Message - Причина остановки работы алгоритма.</li> </ul> |
|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 2.5. Реализация математических моделей при помощи m-функций

Довольно часто при работе в среде MATLAB приходится многократно использовать одинаковый набор команд для решения тех или иных задач. В этом случае эту последовательность команд лучше оформить в виде сценария.

Сценарий – это файл, в котором в необходимом порядке записаны команды, подлежащие последовательному выполнению. Для создания такого файла в среде MATLAB используется редактор m-файлов. Он вызывается командой File / New / M-File.

Наиболее важной формой сценариев являются так называемые m-функции, которые допускают наличие входных и выходных параметров.

### 2.5.1. Синтаксис и вызов m-функций.

M-функция состоит из заголовка и тела функции. В заголовке функции указываются массив выходных значений, имя функции и входные параметры.

*function* [val1, val2, ...]  
= *FunctionName*(Param1, Param2, ...),

где *function* – зарезервированное слово для объявления m-функции,

[val1, val2, ...] – массив выходных значений,

*FunctionName* – имя функции,

*Param1, Param2, ...* - входные параметры функции.

Тело функции состоит из последовательно выполняемых команд, с помощью которых происходит преобразование входных параметров в выходные значения.

Указанное имя функции (*FunctionName*) должно совпадать с именем файла, в который будет записана функция. Рассогласования имени функции и имени файла не допускается.

В тексте m-функции могут еще содержаться комментарии. Комментарии – это отдельные текста, начинающиеся с символа %.

Важную роль в среде MATLAB имеют комментарии, располагающиеся сразу за заголовком определения функции. Эти комментарии отображаются в командной строке MATLAB при исполнении команды:

```
» help <Имя функции>
```

Пример создания m-функции, выполняющей поэлементное сложение и вычитание двух матриц.

```
function [A,B] = Matrix(X1,X2)
    A = X1 + X2;
    B = X1 - X2;
end
```

Вызов m-функции:

```
» U = [5 6;7 8];
» V = [1 2;3 4];
» [A,B] = Matrix(U,V)
A =
     6     8
    10    12
B =
     4     4
     4     4
```

В данном примере объявляются две матрицы U и V. Эти матрицы в качестве входных фактических параметров передаются в функцию. В теле функции эти матрицы обрабатываются «под видом» формальных параметров X1 и X2.

### 2.5.2. Типы данных

В среде MATLAB определено шесть базовых типов данных, каждый из которых является многомерным массивом. Эти типы данных приведены в таблице:

Таблица 2.3. – Типы данных MATLAB

| <i>Тип данных</i> | <i>Описание</i>                                                                                                                                                                                                                                                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Double            | <b>Числовой массив</b> удвоенной точности (это наиболее распространенный тип переменной в системе MATLAB)                                                                                                                                                                                                                                         |
| Char              | <b>Массив символов</b> (каждый символ - длиной 16 битов), часто именуется строкой.                                                                                                                                                                                                                                                                |
| Sparse            | <b>Разреженная матрица</b> удвоенной точности (только двумерная). Разреженная структура применяется для хранения матриц с небольшим количеством ненулевых элементов, что позволяет использовать лишь небольшую часть памяти, требуемой для хранения полной матрицы. Разреженные матрицы требуют применения специальных методов для решения задач. |
| Cell              | <b>Массив ячеек.</b> Элементы этого массива содержат другие массивы. Массивы ячеек позволяют объединить связанные данные, возможно различных размеров, в единую структуру.                                                                                                                                                                        |
| Struct            | <b>Массив записей.</b> Он включает имена полей. Поля сами могут содержать массивы. Подобно                                                                                                                                                                                                                                                        |

|      |                                                                                                                                                                                                                                           |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      | массивам ячеек, массивы записей объединяют связанные данные и информацию о них.                                                                                                                                                           |
| Uin8 | <b>Массив 8-разрядных целых чисел без знаков.</b> Он позволяет хранить целые числа в диапазоне от 0 до 255 в 1/8 части памяти, требуемой для массива удвоенной точности. Никакие математические операции для этих массивов не определены. |

### 2.5.3. Операторы

Операторы среды MATLAB делятся на три категории:

1. Арифметические операторы, позволяющие конструировать арифметические выражения и выполнять числовые вычисления.

Среди арифметических операций существуют следующие уровни приоритета их выполнения:

- Транспонирование матриц, поэлементное возведение в степень, возведение матрицы в степень.
- Унарное сложение, унарное вычитание
- Умножение массивов, правое и левое деление массивов, умножение матриц.
- Сложение и вычитание
- Оператор формирования массивов

Внутри каждого уровня операторы имеют равный приоритет и выполняются слева направо. Заданный по умолчанию порядок следования операторов можно изменить, применяя круглые скобки.

2. Операторы отношения.

В среде MATLAB используется шесть операторов отношения:

- < Меньше
- <= Меньше или равно
- > Больше
- >= Больше или равно
- == Равно



- ~= Не равно

Данные операторы выполняют поэлементное сравнение двух массивов, размерности которых должны быть одинаковыми, кроме случаев, когда один из них – скаляр. В этом случае сравнивается скаляр с каждым элементом массива.

### 3. Логические операторы.

В среде MATLAB есть три логических оператора:

- & И
- | ИЛИ
- ~ НЕТ

Логическая конструкция с оператором И (&) является истинной, если оба выражения, входящие в конструкцию – истинны. Если в логическое выражение входят скаляры, то конструкция будет истинной, если оба числа отличные от нуля.

Логическая конструкция с оператором ИЛИ (|) является истинной, если одно из выражений, входящих в конструкцию – истинно. Если в логическое выражение входят скаляры, то конструкция будет истинной, если хотя бы одно число отлично от нуля.

Логическое выражение с оператором NOT (~) возвращает отрицание. Результат будет ложным, если элемент выражения – истинен, и истинным, если элемент выражения – ложен.

#### 2.5.4. Программирование *m*-функций

Как и в любом языке программирования, средства программирования MATLAB имеют конструкции, служащие для управления порядком выполнения операций. Такие конструкции называются операторами управления. К ним относятся *операторы ветвления* и *операторы цикла*.

К операторам ветвления в М-языке относятся *условный оператор* и *оператор переключения*.

Условный оператор может быть записан в одной из приведенных форм:

- If <Условие>  
    <Операторы>  
End

- If <Условие>  
     <Операторы>  
   else  
     <Операторы>  
   End
- If <Условие1>  
     <Операторы>  
   elseif <Условие2>  
     <Операторы>  
   End

В данном операторе под <Условием> понимается выражение (чаще всего это выражение включает в себя операции сравнения и логические операции), истинность или ложность которого необходимо установить.

Если условие истинно, то выполняются команды, стоящие после строки с ключевым словом if. Если условие ложно, то эти команды пропускаются и переходят либо к следующему за условным оператору (первая форма), либо проверяют ещё одно условие в строке с ключевым словом elseif (третья форма условного оператора), либо выполняются без дополнительных проверок команды, стоящие после строки с ключевым словом else (вторая из приведённых выше форм).

Особенностью использования массивов в условных выражениях является следующий факт. Условное выражение, в которое входит массив, будет истинным, если истинны (не равны нулю) все элементы этого массива. Если хотя бы один из элементов массива равен нулю, или задан пустой массив, то условное выражение будет ложным.

Пример: Даны два вектора A и B. Получить новый вектор C, посредством поэлементного перемножения исходных векторов.

```

if length(A) == length(B)
    C = A.*B;
else
    error('Размерности векторов не
    совпадают');
end

```

Оператор переключения имеет следующую конструкцию:

- Switch <Выражение>
  - Case <значение 1>
    - <Операторы>
  - Case <значение 2>
    - <Операторы>
  - ...
  - ...
  - otherwise
  - ...

end

Сначала вычисляется скалярное числовое значение <Выражения>, а затем полученный результат сравнивается с набором значений <значение 1>, <значение 2>, <значение 3> и так далее. В случае его совпадения с одним из значений, выполняются нижестоящие операторы. Если нет совпадения ни с одним из перечисленных значений, то выполняются операторы, стоящие после ключевого слова **otherwise**.

Пример: Вывести строковое представление числа, лежащего в интервале [1 ; 5]

```

switch x
    case 1
        s = 'Один';
    case 2
        s = 'Два';
    case 3
        s = 'Три';
    case 4

```

```

        s = 'Четыре';
    case 5
        s = 'Пять';
    otherwise
        s = 'Число не лежит в заданном
            интервале';
end

```

В m-языке среды MATLAB различают два вида операторов цикла:

1. While <Выражение>  
     <Операторы>  
     End

В цикле данного вида повтор участка кода, обозначенного <Операторы>, повторяется до тех пор, пока <Выражение> является истинным.

Пример: Вычислить сумму элементов вектора.

```

s = 0;
i = 1;
while i <= length(x)
    s = s + x(i);
    i = i + 1;
end

```

2. for varName = <Выражение>  
     <Операторы>  
     end

Цикл данного вида выполняется для всех значений переменной цикла varName. Набор возможных значений для этой переменной определяется <Выражением>.

Пример: Вычислить произведение элементов вектора.

```

p = 1;

```

```

for i = 1:1:length(x)
    p = p * x(i);
end

```

### 2.5.5. Проверка входных параметров и выходных значений *m*-функций

Несоответствие типов, размерностей и числа фактических и формальных параметров М-функций может привести к их неправильной работе. Поэтому внутри *m*-функции необходимо выполнять проверку на корректность ее входных параметров.

В примере, рассмотренном выше, предполагалось, что пользователь в качестве фактических параметров задает две матрицы одинакового размера. Однако всегда существует опасность возникновения ошибки при вызове *m*-функции и указании для нее входных параметров. Чтобы избежать этих ошибок, необходимо дополнить имеющийся код *m*-функции специальными операторами:

```

function [A,B] = Matrix(X1,X2)
    if nargin ~= 2
        error('Количество входных
               параметров должно быть равно 2')
    end
    if nargout ~= 2
        error('Количество выходных
               параметров должно быть равно 2')
    end
    if size(X1) ~= size(X2)
        error('Размерности матриц не
               совпадают')
    end
    A = X1 + X2;
    B = X1 - X2;
end

```

В данном примере последовательно выполняется проверка на ошибки следующих типов:

- Недопустимое число входных параметров функции. Для проверки на наличие этой ошибки предусмотрена специальная переменная *nargin*. Её значением является количество аргументов, фактически переданное функции при вызове. В данном примере количество входных параметров должно быть равно 2. Если переменная *nargin* не равна этому числу, то стандартная функция *error* корректно остановит выполнение m-функции и выведет в командное окно MATLAB соответствующее сообщение.

- Несоответствующее число выходных значений. Для проверки на наличие этой ошибки используется специальная переменная *nargout*. Принцип работы с ней такой же, как и с переменной *nargin*.

- Несоответствие в размерностях входных параметров. Для определения размерностей входных матриц используется функция *size(A)*. Если при сравнении этих размерностей выявляется несоответствие, функция *error* выведет предупреждение об этом.

### 2.5.6. Принципы решения задач линейного программирования в MATLAB

Для успешного решения производственно-экономической задачи в среде MATLAB необходимо придерживаться следующих принципов:

1. Выходным массивом производственно-экономической задачи должен быть один вектор. Если оптимальное решение задачи предполагает несколько выходных векторов, то они должны быть объединены в один вектор с последующим его разбиением. Если предполагается, что оптимальное решение задачи представлено в виде матрицы, то матрица тоже должна быть преобразована в вектор.

2. Если целевая функция задачи в качестве своих аргументов имеет несколько векторов коэффициентов, то эти векторы должны быть объединены в один вектор. Если в качестве аргумента целевой функции выступает матрица, то ее необходимо также преобразовать в вектор.

3. Производственно-экономическая задача на максимизацию значения целевой функции должна быть преобразована в задачу на минимизацию значения целевой функции посредством замены знака коэффициентов при неизвестных в функции на противоположные.

4. Все неравенства, входящие в систему ограничения целевой функции должны быть приведены к типу «меньше или равно».

5. Все матрицы коэффициентов при неизвестных, находящихся в левой части неравенств (равенств) системы ограничений, должны быть объединены в одну матрицу. Все векторы коэффициентов, находящихся в правой части неравенств (равенств) системы ограничений, должны быть объединены в один вектор.

Пример: Производственное предприятие выпускает продукцию трех видов. На производство продукции потребляются материальные ресурсы пяти видов. Цена (С) реализации единицы продукции составляет соответственно 68, 90, 70 условных единиц, стоимость единицы приобретаемых материальных ресурсов (S) – 4, 5, 8, 6, 9 условных единиц соответственно. Запасы материальных ресурсов (В) - 12, 25, 13, 12, 12 единиц, минимальные значения суммарных технико-экономических показателей продукции (F) 13, 14 единиц соответственно.

$$A = \begin{pmatrix} 2 & 1 & 4 \\ 4 & 1 & 5 \\ 2 & 3 & 2 \\ 3 & 6 & 3 \\ 9 & 5 & 1 \end{pmatrix} \quad - \quad \text{технологическая матрица затрат}$$

материальных ресурсов на производство продукции,

$$f = \begin{pmatrix} 3 & 2 & 6 \\ 4 & 5 & 1 \end{pmatrix} \quad - \quad \text{матрица коэффициентов в ограничении на технико-экономические показатели продукции.}$$

Финансовые инвестиции на приобретение материальных ресурсов составляют  $Q = 200$  условных единиц.

Требуется найти такой план производства продукции (X) и такой план закупок материальных ресурсов (Y), которые бы обеспечили максимальное значение дохода от реализации продукции при заданных ограничениях.

Решение. Для решения задачи в среде MATLAB необходимо последовательно выполнить 2 этапа:

1. Сформировать исходные данные и привести их к виду, соответствующему модели задачи линейного программирования в среде MATLAB.

2. Вызвать функцию *linprog* необходимыми параметрами.

Алгоритм решения:

1. Исходная задача линейного программирования имеет следующую математическую модель:

$Z(X, Y) = C \cdot X - S \cdot Y \rightarrow \max$  при ограничениях:

$$\left\{ \begin{array}{l} A \cdot X \leq B + Y \\ f \cdot X \geq F \\ \sum_{i=1}^5 S_i \cdot Y_i \leq Q \\ X \geq 0 \\ Y \geq 0 \end{array} \right.$$

Данная задача является задачей на нахождение максимума целевой функции, а в системе ее ограничений существуют неравенства со знаком « $\geq$ ».

Для приведения ее к виду, соответствующему модели задачи линейного программирования в среде MATLAB необходимо:

- изменить тип задачи с задачи «на максимум» на задачу на «минимум». Для этого нужно изменить коэффициенты при аргументах целевой функции на противоположные.

- изменить тип неравенства « $\geq$ » на противоположный. Для этого необходимо умножить обе части неравенства на «-1».



- в первом ограничении перенести вектор неизвестных величин  $Y$  в левую часть неравенства.

После указанных изменения математическая модель задачи примет вид:

$$Z(X, Y) = -C \cdot X + S \cdot Y \rightarrow \min \text{ при ограничениях: } \begin{cases} A \cdot X - Y \leq B \\ -f \cdot X \leq -F \\ \sum_{i=1}^5 S_i \cdot Y_i \leq Q \\ X \geq 0 \\ Y \geq 0 \end{cases}$$

2. Целевая функция задачи должна зависеть только от одного вектора неизвестных величин. Поэтому объединим вектора  $X$  (размерность  $3 \times 1$ ) и вектор  $Y$  (размерность  $5 \times 1$ ) в один вектор  $x$  (размерность  $8 \times 1$ ) и запишем математическую модель с исходными коэффициентами. Тогда целевая функция примет вид:

$$Z(x) = -68x_1 - 90x_2 - 70x_3 + 4x_4 + 5x_5 + 8x_6 + 6x_7 + 9x_8$$

Первое ограничение системы ограничений можно представить как:

$$\begin{cases} 2x_1 + 1x_2 + 4x_3 - 1x_4 - 0x_5 - 0x_6 - 0x_7 - 0x_8 \leq 12 \\ 4x_1 + 1x_2 + 5x_3 - 0x_4 - 1x_5 - 0x_6 - 0x_7 - 0x_8 \leq 25 \\ 2x_1 + 3x_2 + 2x_3 - 0x_4 - 0x_5 - 1x_6 - 0x_7 - 0x_8 \leq 13 \\ 3x_1 + 6x_2 + 3x_3 - 0x_4 - 0x_5 - 0x_6 - 1x_7 - 0x_8 \leq 12 \\ 9x_1 + 5x_2 + 1x_3 - 0x_4 - 0x_5 - 0x_6 - 0x_7 - 1x_8 \leq 12 \end{cases}$$

Второе ограничение запишется в виде:

$$\begin{cases} -3x_1 - 2x_2 - 6x_3 - 0x_4 - 0x_5 - 0x_6 - 0x_7 - 0x_8 \leq -13 \\ -4x_1 - 5x_2 - 1x_3 - 0x_4 - 0x_5 - 0x_6 - 0x_7 - 0x_8 \leq -14 \end{cases}$$

Третье ограничение:

$$0x_1 + 0x_2 + 0x_3 + 4x_4 + 5x_5 + 8x_6 + 6x_7 + 9x_8 \leq 200$$

3. «Совместим» полученные неравенства в единую систему ограничений и запишем математическую модель в следующей форме:

$$-68x_1 - 90x_2 - 70x_3 + 4x_4 + 5x_5 + 8x_6 + 6x_7 + 9x_8 \rightarrow \min$$

$$\left\{ \begin{array}{l} 2x_1 + 1x_2 + 4x_3 - 1x_4 - 0x_5 - 0x_6 - 0x_7 - 0x_8 \leq 12 \\ 4x_1 + 1x_2 + 5x_3 - 0x_4 - 1x_5 - 0x_6 - 0x_7 - 0x_8 \leq 25 \\ 2x_1 + 3x_2 + 2x_3 - 0x_4 - 0x_5 - 1x_6 - 0x_7 - 0x_8 \leq 13 \\ 3x_1 + 6x_2 + 3x_3 - 0x_4 - 0x_5 - 0x_6 - 1x_7 - 0x_8 \leq 12 \\ 9x_1 + 5x_2 + 1x_3 - 0x_4 - 0x_5 - 0x_6 - 0x_7 - 1x_8 \leq 12 \\ -3x_1 - 2x_2 - 6x_3 - 0x_4 - 0x_5 - 0x_6 - 0x_7 - 0x_8 \leq -13 \\ -4x_1 - 5x_2 - 1x_3 - 0x_4 - 0x_5 - 0x_6 - 0x_7 - 0x_8 \leq -14 \\ 0x_1 + 0x_2 + 0x_3 + 4x_4 + 5x_5 + 8x_6 + 6x_7 + 9x_8 \leq 200 \end{array} \right.$$

Тогда исходные данные для решения задачи в MATLAB можно представить в виде:

$$\gg Z = [-C; S]$$

$$Z =$$

$$-68$$

$$-90$$

$$-70$$

$$4$$

$$5$$

$$8$$

$$6$$

$$9$$

```

» A=[A -eye(5);-f zeros(2,5);zeros(1,3) S']
A =
    2     1     4    -1     0     0     0     0
    4     1     5     0    -1     0     0     0
    2     3     2     0     0    -1     0     0
    3     6     3     0     0     0    -1     0
    9     5     1     0     0     0     0    -1
   -3    -2    -6     0     0     0     0     0
   -4    -5    -1     0     0     0     0     0
    0     0     0     4     5     8     6     9

» B = [B;-F;Q]
B =
    12
    25
    13
    12
    12
   -13
   -14
   200

» [x,p]= linprog(Z,A,B,[],[],zeros(8,1),[])
Optimization terminated.
x =
    0.0000
    1.8750
    4.6250
    8.3750
    0.0000
    1.8750
   13.1250
    2.0000

p =
  -347.2500

```

При вызове функции *linprog* были использованы пустые множества в качестве входных аргументов, обозначенные «[]».

Наличие первых двух пустых множеств говорит о том, что в системе ограничений не было ограничений в виде равенств. Последнее пустое множество в списке аргументов показывает, что отсутствует верхний предел, ограничивающий искомые значения оптимального решения.

В результате решения задачи был получен вектор, состоящий из 8 элементов. Первые три элемента – это оптимальный план производства продукции, последние 5 элементов – это оптимальный план закупки ресурсов.

## **2.6. Интегрирование MATLAB и MS Excel. Экспорт и импорт**

Для загрузки данных из Excel в среде MATLAB существует функция *xlsread*.

Синтаксис функции:

- »  $A = \text{xlsread}('FileName')$ . Если в качестве входного аргумента указано только имя файла, то происходит считывание данных с первого листа книги. Если первый лист не содержит данных, то функция вернет пустой массив. При считывании данных с листа книги Excel игнорируются ячейки, содержащие текст. Если в диапазон значений входят ячейки, содержащие текст и пустые ячейки, то вместо них в возвращаемом функцией массиве будет находиться NaN (не числа - Not a Number).
- »  $A = \text{xlsread}('FileName', n)$ . Для считывания данных с произвольного листа книги необходимо указать его номер или имя в качестве второго входного аргумента.
- »  $A = \text{xlsread}('FileName', n, 'Range')$ . Функция позволяет считать данные из конкретного диапазона ячеек. Если диапазон данных указан в качестве второго входного аргумента функции, то происходит считывание данных из этого диапазона на первом листе книги Excel, а если во втором входном аргументе функции указать имя листа, а диапазон данных - в третьем, то будет производиться считывание данных из заданного диапазона этого листа.

Примеры: Пусть имеется файл book.xlsx,

|   | A                       | B         | C         | D         | E         | F |
|---|-------------------------|-----------|-----------|-----------|-----------|---|
| 1 | Технологическая матрица |           |           |           |           |   |
| 2 |                         | Продукт 1 | Продукт 2 | Продукт 3 | Продукт 4 |   |
| 3 | Ресурс 1                | 2         | 3,4       | 4,2       | 6         |   |
| 4 | Ресурс 2                | 2,5       | 4         | 2         | 1,7       |   |
| 5 | Ресурс 3                | 4,8       | 5,9       | 6,3       | 3,2       |   |
| 6 | Ресурс 4                | 7,4       | 7,5       | 4,8       | 3,1       |   |
| 7 | Ресурс 5                | 9,4       | 8,3       | 2,6       | 3,6       |   |
| 8 |                         |           |           |           |           |   |
| 9 |                         |           |           |           |           |   |

Рисунок 2.1. – Лист Excel с данными

Тогда результаты работы функции xlsread будут следующие:

```

» A = xlsread('book.xlsx')
A =
    2.0000    3.4000    4.2000    6.0000
    2.5000    4.0000    2.0000    1.7000
    4.8000    5.9000    6.3000    3.2000
    7.4000    7.5000    4.8000    3.1000
    9.4000    8.3000    2.6000    3.6000

» A = xlsread('book.xlsx',1)
A =
    2.0000    3.4000    4.2000    6.0000
    2.5000    4.0000    2.0000    1.7000
    4.8000    5.9000    6.3000    3.2000
    7.4000    7.5000    4.8000    3.1000
    9.4000    8.3000    2.6000    3.6000

» A = xlsread('book.xlsx',1,'B3:C7')
A =
    2.0000    3.4000
    2.5000    4.0000
    4.8000    5.9000
    7.4000    7.5000
    9.4000    8.3000

```

Для экспорта данных в Excel используется функция `xlswrite`. Ее синтаксис:

- » `A = xlswrite('FileName',M)`. Функция записывает в первый лист книги Excel массив `M`. Массив записывается на листе, начиная с ячейки `A1`. Если на листе Excel существуют данные, то функция их перезаписывает.
- » `A = xlswrite('FileName',M,n)`. Функция записывает в лист под номером `n` книги Excel массив `M`.
- » `A = xlswrite('FileName',n,'Range')`. Функция записывает в лист под номером `n` книги Excel массив `M`. Массив записывается в ячейки, указанные в диапазоне `Range`. В качестве диапазона `Range` можно указать одну ячейку. В этом случае массив запишется таким образом, что элемент массива `M(1,1)` будет помещен в ячейку `Range`.

## 2.7. Интегрирование MATLAB и баз данных MS SQL Server. Экспорт и импорт

Для интеграции MATLAB и SQL Server необходимо выполнить настройку источников данных ODBC. Эта программа интегрирована в систему Windows и запускается из панели управления.

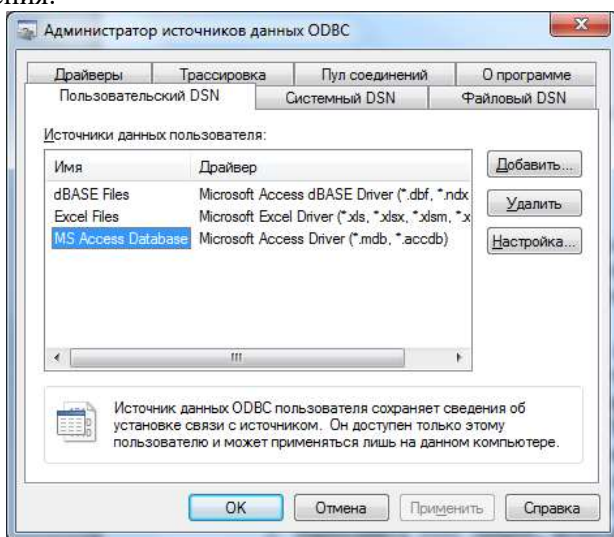


Рисунок 2.2. – Администратор источников данных ODBC

Для создания нового источника данных нужно нажать кнопку «Добавить» и в окне «Создание нового источника данных» выбрать драйвер SQL Server Native Client 10.0 или SQL Server в зависимости от версии установленного SQL сервера.

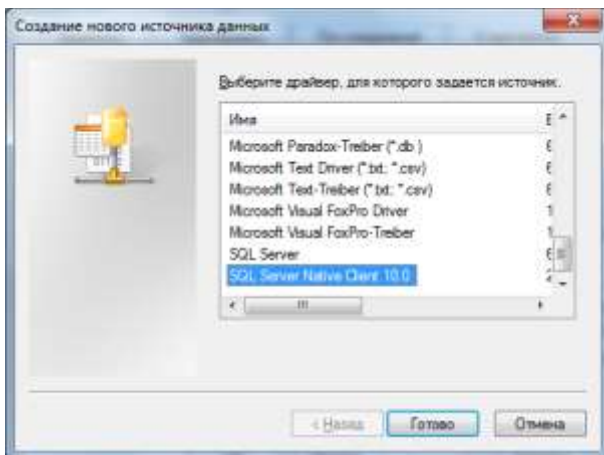


Рисунок 2.3. – Создание нового источника данных

В появившемся окне необходимо указать имя источника данных и имя установленного SQL сервера.

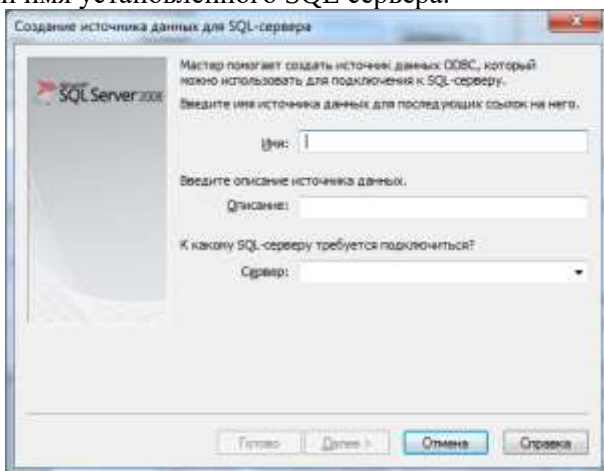


Рисунок 2.4. – Создание нового источника данных SQL Server

Следующие окна предназначены для настройки дополнительных параметров источника данных.

После создания источника данных в ODBC можно использовать базу данных SQL Server для работы в среде MATLAB.

Для импорта данных из базы данных SQL Server необходимо последовательно выполнить следующие операции (эти же операции необходимо выполнить и для экспорта данных в базу данных):

1. Установить соединение с базой данных SQL Server. Для подключения к базе данных используется функция:

`conn = database('DataSourceName','UserName','Password')`, где DataSourceName – имя базы данных SQL Server, UserName и Password – соответственно имя пользователя и пароль, которые необходимо указать для подключения к базе данных.

2. Подготовить SQL-запрос к базе данных.

3. Выполнить этот запрос, используя функции `fetch` или `exes`. Функция `fetch` позволяет загрузить данные из базы данных. Возвращаемые функцией данные представлены в виде массивов ячеек. Для преобразования массива ячеек в массив чисел применяется функция `cell2mat`.

## **2.8. Визуализация данных**

В состав системы MATLAB входит мощная графическая подсистема, которая поддерживает средства визуализации двумерной и трехмерной графики. Следует выделить несколько уровней работы с графическими объектами. В первую очередь это команды и функции, ориентированные на конечного пользователя и предназначенные для построения графиков в прямоугольных и полярных координатах, гистограмм и столбцовых диаграмм, трехмерных поверхностей и линий уровня, анимации.

Графические команды высокого уровня автоматически контролируют масштаб, выбор цветов, не требуя манипуляций со свойствами графических объектов. Соответствующий низкоуровневый интерфейс обеспечивается дескрипторной графикой, когда каждому графическому объекту ставится в соответствие графическая поддержка (дескриптор), на который



можно ссылаться при обращении к этому объекту. Используя дескрипторную графику, можно создавать меню, кнопки вызова, текстовые панели и другие объекты графического интерфейса.

### 2.8.1. Двумерные графики и диаграммы

- PLOT - график в линейном масштабе

Команда `plot(y)` строит график элементов одномерного массива `y` в зависимости от номера элемента

Команда `plot(x, y)` соответствует построению обычной функции, когда одномерный массив `x` соответствует значениям аргумента, а одномерный массив `y` - значениям функции. Когда один из массивов `X` или `Y` либо оба двумерные, реализуются следующие построения:

- если массив `Y` двумерный, а массив `X` одномерный, то строятся графики для столбцов массива `Y` в зависимости от элементов вектора `X`;
- если двумерным является массив `X`, а массив `y` одномерный, то строятся графики столбцов массива `X` в зависимости от элементов вектора `Y`;
- если оба массива `X` и `Y` двумерные, то строятся зависимости столбцов массива `Y` от столбцов массива `X`.

Команда `plot(x, y, s)` позволяет выделить график функции, указав способ отображения линии, способ отображения точек, цвет линий и точек с помощью строковой переменной `s`, которая может включать до трех символов из следующей таблицы:

Таблица 2.4. – Параметры настройки линий графика

| Тип линии        |    | Тип точки |   | Цвет       |   |
|------------------|----|-----------|---|------------|---|
| Непрерывная      | -  | Точка     | . | Желтый     | y |
| Штриховая        | -- | Плюс      | + | Фиолетовый | m |
| Двойной пунктир  | :  | Звездочка | * | Голубой    | c |
| Штрих-пунктирная | -. | Кружок    | o | Красный    | r |
|                  |    | Крестик   | x | Зеленый    | g |
|                  |    |           |   | Синий      | b |
|                  |    |           |   | Белый      | w |
|                  |    |           |   | Черный     | k |

```

» x = -5 : 0.5 : 5;
» y = x.*x;
» plot(x,y,'-- x r')

```

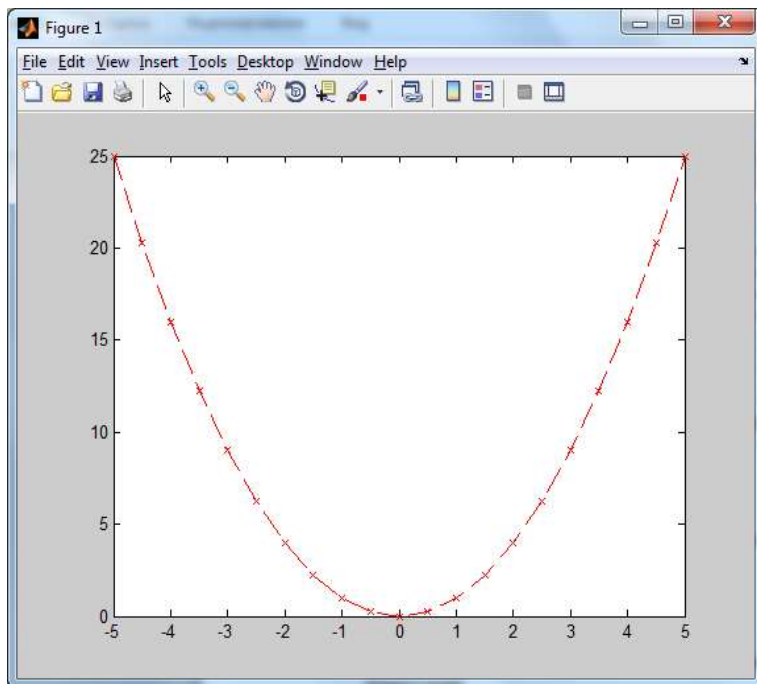


Рисунок 2.5. – График PLOT

Если цвет линии не указан, он выбирается по умолчанию из шести первых цветов, с желтого до синего, повторяясь циклически.

Команда `plot(x1, y1, s1, x2, y2, s2, ...)` позволяет объединить на одном графике несколько функций  $y_1(x_1)$ ,  $y_2(x_2)$ , ..., определив для каждой из них свой способ отображения.

Обращение к командам `plot` вида `plot(x, y, s1, x, y, s2)` позволяет для графика  $y(x)$  определить дополнительные свойства, для указания которых применения одной строковой переменной `s1` недостаточно, например, при задании разных цветов для линии и для точек на ней.

```

» x = -5:0.5:5;
» y1=x.*x;
» y2=x.^3;
» plot(x,y1,'-- x r',x,y2,'- o b')

```

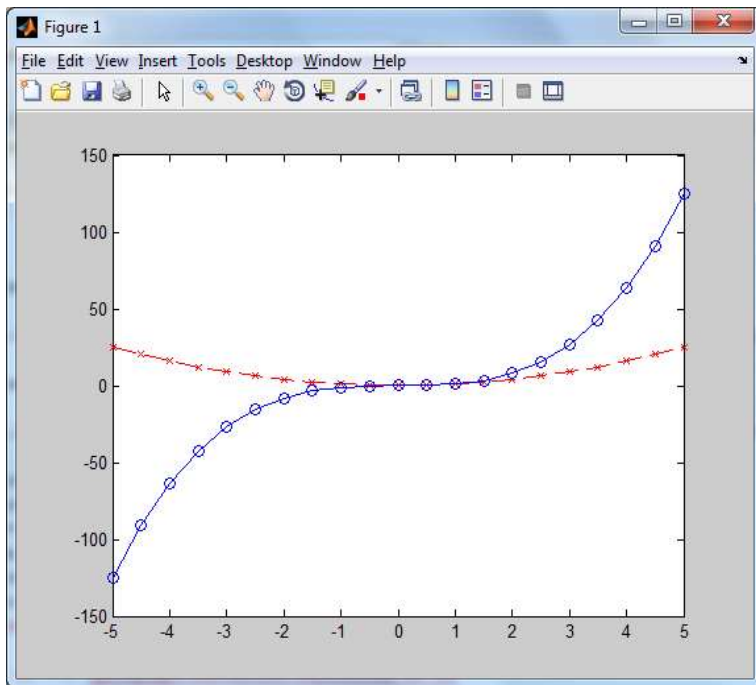


Рисунок 2.6. – График PLOT

- BAR - столбцовые диаграммы

Команда `bar(y)` выводит график элементов одномерного массива `y` в виде столбцовой диаграммы.

Команда `bar(x, y)` выводит график элементов массива `y` в виде столбцов в позициях, определяемых массивом `x`, элементы которого должны быть упорядочены в порядке возрастания.

Если `X` и `Y` - двумерные массивы одинаковых размеров, то каждая диаграмма определяется

соответствующей парой столбцов и они надстраиваются одна над другой.

```
» x = -5:0.5:5;  
» y2 = x.^3;  
» bar(x,y2)
```

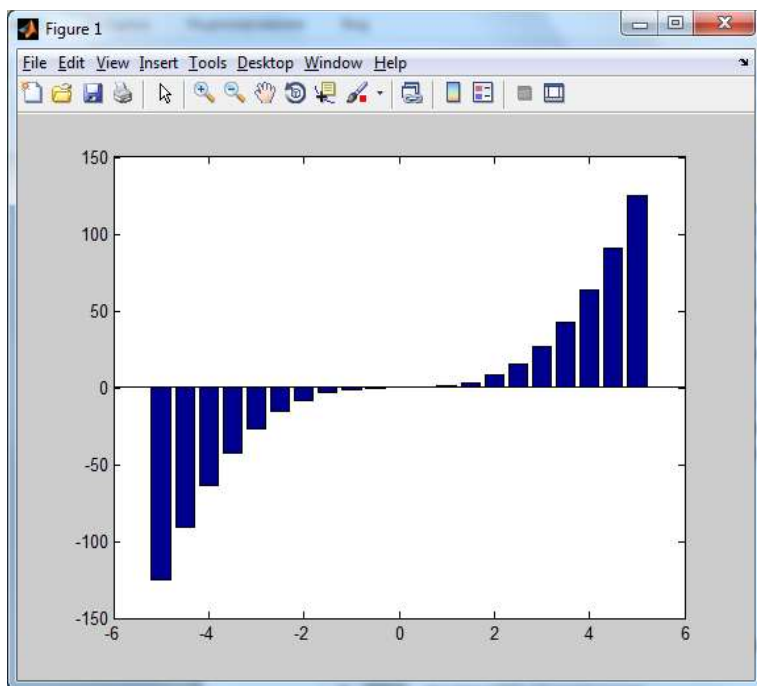


Рисунок 2.7. – Диаграмма BAR

- **ERRORBAR** - график с указанием интервала погрешности

Команда `errorbar(x, y, l, u, '<тип линии>')` строит график функции  $y$  в зависимости от  $x$  с указанием интервала погрешности, который определяется массивами  $l$  и  $u$ . Массивы  $x$ ,  $y$ ,  $l$ ,  $u$  должны быть одного размера. Погрешности в каждой точке  $\{x(i), y(i)\}$  определяются отклонениями вниз  $l(i)$  и вверх  $u(i)$  относительно точки графика, так что суммарная

погрешность равна  $l(i) + u(i)$ . Строковая переменная '<тип линии>' аналогична используемой в команде plot.

```
» x = 1 : 10;  
» y = sin(x);  
» e = std(y)*ones(size(x))/5;  
» errorbar(x, y, e)
```

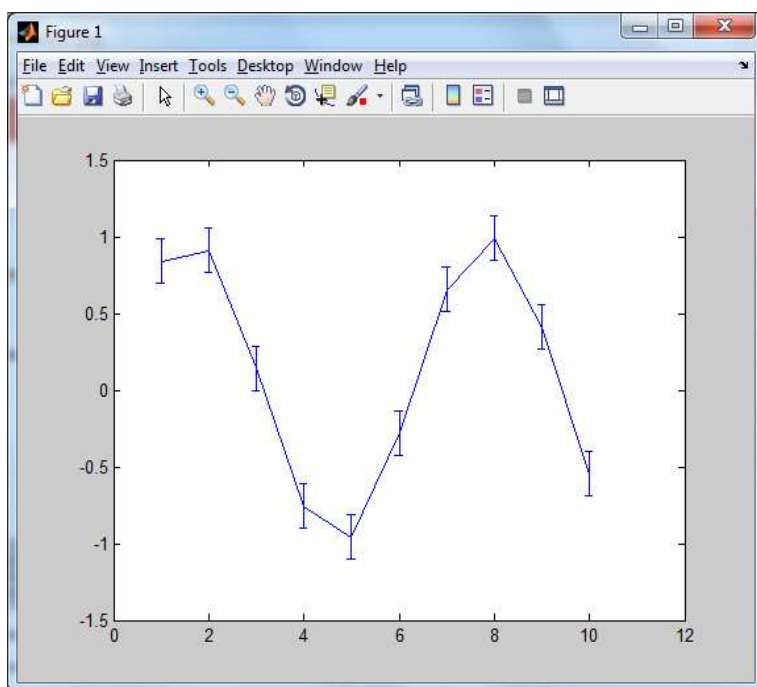


Рисунок 2.8. – График ERRORBAR

- HIST - построение гистограммы

Команды hist(...) подсчитывают и отображают на графике количество элементов массива y, значения которых попадают в заданный интервал; для этого весь диапазон значений y делится

на  $n$  интервалов (по умолчанию 10) и подсчитывается количество элементов в каждом интервале.

Команда `hist(y)` выводит гистограмму для 10 интервалов.

Команда `hist(y, n)` выводит гистограмму для  $n$  интервалов.

Команда `hist(y, x)` выводит гистограмму с учетом диапазона изменения переменной  $x$ .

```
» x = -2.9 : 0.1 : 2.9;  
» y = randn(10000,1);  
» hist(y, x)
```

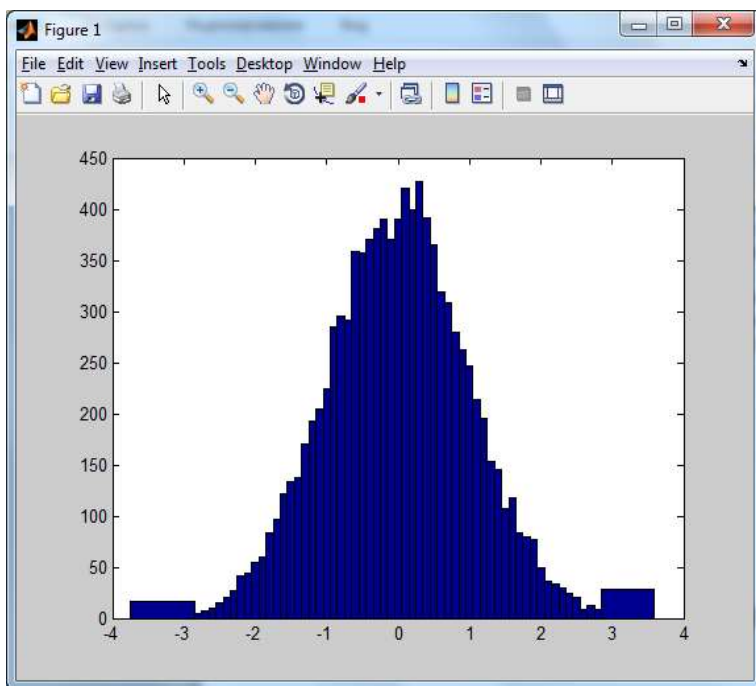


Рисунок 2.9. – Гистограмма HIST

- STEM - дискретные графики

Команда `stem(y)` выводит график элементов одномерного массива  $y$  в виде вертикальных линий, которые заканчиваются в точках графика, помечаемых кружочком.

Команда `stem(x, y)` выводит график элементов массива `y` в виде вертикальных линий в позициях, определяемых массивом `x`, элементы которого должны быть упорядочены в порядке возрастания.

Команды `stem(y, <тип линии>)`, `stem(x, y, <тип линии>)` позволяют задать тип линий, используемых для построения дискретного графика, по аналогии с командой `plot`.

```
» x = -5:0.5:5;  
» y = x.*x;  
» stem(x, y)
```

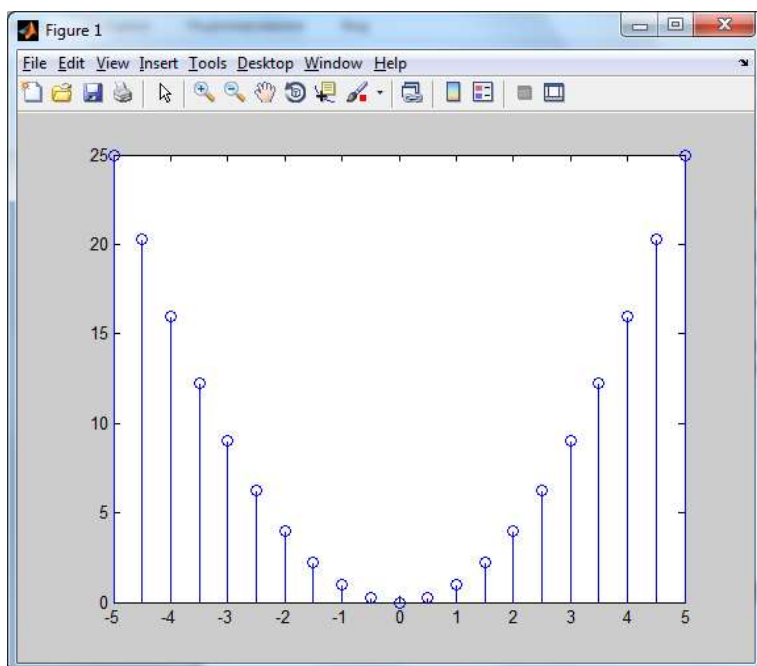


Рисунок 2.10. – График STEM

- STAIRS - ступенчатый график

Команда `stairs(y)` выводит график элементов одномерного массива `y` в виде ступенчатой функции, аналогичной столбцовой

диаграмме, но без вертикальных линий. Такие графики целесообразно использовать при выводе процессов в дискретно-непрерывных системах.

Команда `stairs(x, y)` выводит график элементов массива `y` в виде ступенчатой функции в позициях, определяемых массивом `x`, элементы которого должны быть упорядочены в порядке возрастания.

```
» x = -5:0.5:5;  
» y = x.*x;  
» stairs(x,y)
```

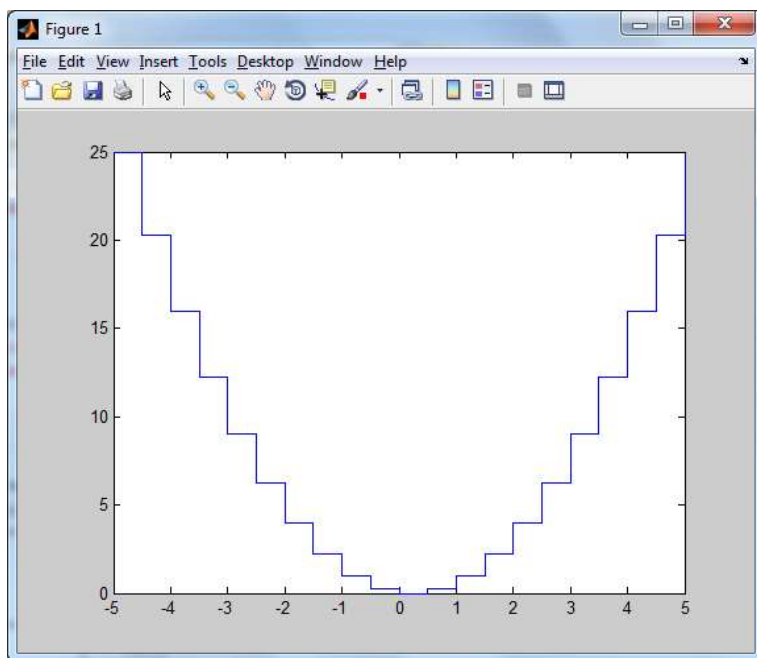


Рисунок 2.11. – График STAIRS

- COMET - движение точки по траектории

Команда `comet(y)` рисует движение точки по траектории, заданной одномерным массивом `y`, в виде головы и хвоста кометы.



Команда `comet(x, y)` рисует движение точки по траектории, заданной массивами  $x$  и  $y$ .

Команда `comet(x, y, p)` управляет длиной хвоста кометы  $p \cdot \text{length}(y)$  с помощью параметра  $p$ ; по умолчанию  $p = 0.10$ .

```
» t = -pi : pi/200 : pi;  
» comet(t, tan(sin(t)) - sin(tan(t)))
```

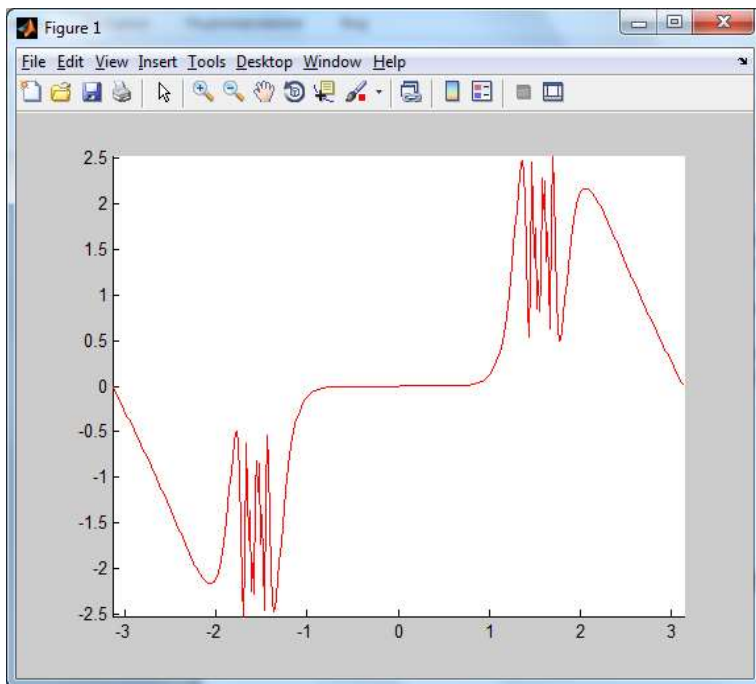


Рисунок 2.12. – График КОМЕТ

### 2.8.2. Трехмерные графики и диаграммы

В системе MATLAB предусмотрено несколько команд и функций для построения трехмерных графиков. Значения элементов числового массива рассматриваются как  $z$ -координаты точек над плоскостью, определяемой координатами  $x$  и  $y$ . Возможно несколько способов соединения этих точек.

Первый из них - это соединение точек в сечении (функция `plot3`), второй - построение сетчатых поверхностей (функции `mesh` и `surf`). Поверхность, построенная с помощью функции `mesh`, - это сетчатая поверхность, ячейки которой имеют цвет фона, а их границы могут иметь цвет, который определяется свойством `EdgeColor` графического объекта `surface`. Поверхность, построенная с помощью функции `surf`, - это сетчатая поверхность, у которой может быть задан цвет не только границы, но и ячейки; последнее управляется свойством `FaceColor` графического объекта `surface`.

- `PLOT3` - построение линий и точек в трехмерном пространстве

Команды `plot3(...)` являются трехмерными аналогами функции `plot(...)`.

Команда `plot3(x, y, z)`, где `x`, `y`, `z` - одномерные массивы одинакового размера, строит точки с координатами `x(i)`, `y(i)`, `z(i)` и соединяет их прямыми линиями.

Команда `plot3(X, Y, Z)`, где `X`, `Y`, `Z` - двумерные массивы одинакового размера, строит точки с координатами `x(i, :)`, `y(i, :)`, `z(i, :)` для каждого столбца и соединяет их прямыми линиями.

Команда `plot3(x1, y1, z1, s1, x2, y2, z2, s2, ...)` позволяет объединить на одном графике несколько функций `z1(x1, y1)`, `z2(x2, y2)`, ..., определив для каждой из них свой способ отображения.

Обращение к команде `plot3` вида `plot3(x, y, z, s1, x, y, z, s2)` позволяет для графика `z(x, y)` определить дополнительные свойства, для указания которых применения одной строковой переменной `s1` недостаточно, например при задании разных цветов для линии и для точек на ней.

```
» [ X, Y ] = meshgrid([ -2 : 0.1 : 2 ]);
» Z = X .* exp(- X .^ 2 - Y .^ 2);
» plot3(X, Y, Z)
```

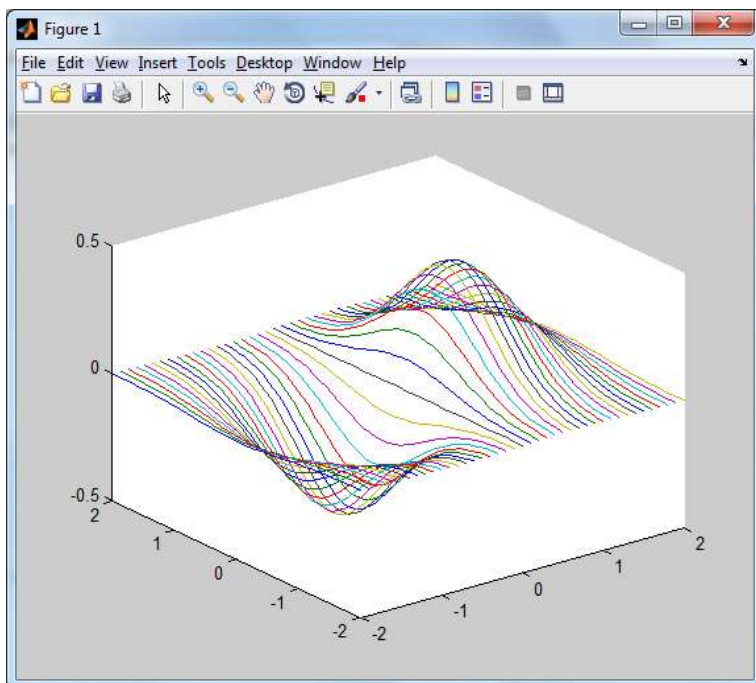


Рисунок 2.12. – Поверхность PLOT3

- MESHGRID - формирование двумерных массивов X и Y

Функция  $[X, Y] = \text{meshgrid}(x, y)$  задает сетку на плоскости  $x$ - $y$  в виде двумерных массивов  $X$ ,  $Y$ , которые определяются одномерными массивами  $x$  и  $y$ . Строки массива  $X$  являются копиями вектора  $x$ , а столбцы - копиями вектора  $y$ . Формирование таких массивов упрощает вычисление функций двух переменных, позволяя применять операции над массивами.

Функция  $[X, Y] = \text{meshgrid}(x)$  представляет собой упрощенную форму записи для функции  $[X, Y] = \text{meshgrid}(x, x)$ .

- MESH, MESHG, MESHZ - трехмерная сетчатая поверхность

Команда `mesh(X, Y, Z, C)` выводит на экран сетчатую поверхность для значений массива `Z`, определенных на множестве значений массивов `X` и `Y`. Цвета узлов поверхности задаются массивом `C`. Цвета ребер определяются свойством `EdgeColor` объекта `surface`. Можно задать одинаковый цвет для всех ребер, определив его в виде вектора `[r g b]` интенсивности трех цветов - красного, зеленого, синего. Если определить спецификацию `none`, то ребра не будут прорисовываться. Если определить спецификацию `flat`, то цвет ребер ячейки определяется цветом того узла, который был первым при обходе этой ячейки. Поскольку одни и те же ребра обходятся несколько раз, то цвета будут замещаться. Если определить спецификацию `interp`, то будет реализована линейная интерполяция цвета между вершинами ребра.

Применение функции `shading` после обращения к функции `mesh` изменяет спецификации свойств `EdgeColor` и `FaceColor` согласно следующей таблице.

Таблица 2.5. – Изменение свойств поверхности

| Свойство               | Применяемая функция |                           |                             |
|------------------------|---------------------|---------------------------|-----------------------------|
|                        | <code>mesh</code>   | <code>shading flat</code> | <code>shading interp</code> |
| <code>EdgeColor</code> | <code>flat</code>   | <code>flat</code>         | <code>interp</code>         |
| <code>FaceColor</code> | Цвет фона           | Цвет фона                 | Цвет фона                   |

Команда `mesh(x, y, Z, C)` выполняет ту же функцию, но вместо двумерных массивов `X`, `Y` использует их одномерные проекции, так что если `length(x) = n`, а `length(y) = m`, то `[m, n] = size(Z)`. В этом случае узлы сетчатой поверхности определяются тройками `{x(j), y(i), Z(i, j)}`, где вектор `x` определяет столбцы массива `Z`, а `y` - строки.

Команда `mesh(Z, C)` использует сетку, которая определяется одномерными массивами `x = 1 : n` и `y = 1 : m`.

Команды `mesh(X, Y, Z)`, `mesh(x, y, Z)`, `mesh(Z)` используют в качестве массива цвета `C = Z`, то есть цвет в этом случае пропорционален высоте поверхности.

Группа команд `meshc(...)` в дополнение к трехмерным поверхностям строит проекцию линий постоянного уровня.

Группа команд `meshz(...)` в дополнение к трехмерным поверхностям строит плоскость отсчета на нулевом уровне, закрывая поверхность, лежащую ниже этого уровня.

Функция `h = mesh(...)` возвращает дескриптор `h` для графического объекта `surface`.

```
» [ X, Y ] = meshgrid([ -2 : 0.1 : 2 ]);  
» Z = X.*exp(- X .^ 2 - Y .^ 2);  
» mesh(X,Y,Z)  
» meshc(X, Y, Z)  
» meshz(X,Y,Z)
```

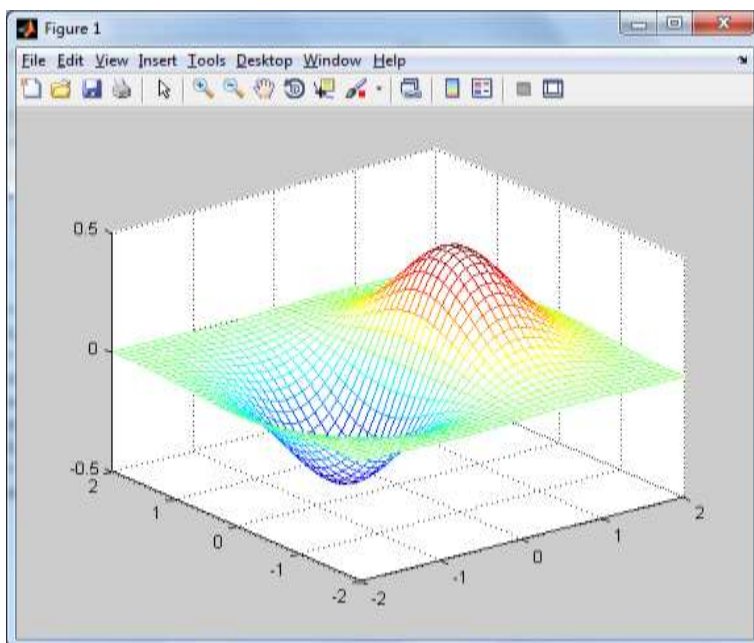


Рисунок 2.13. – Поверхность MESH

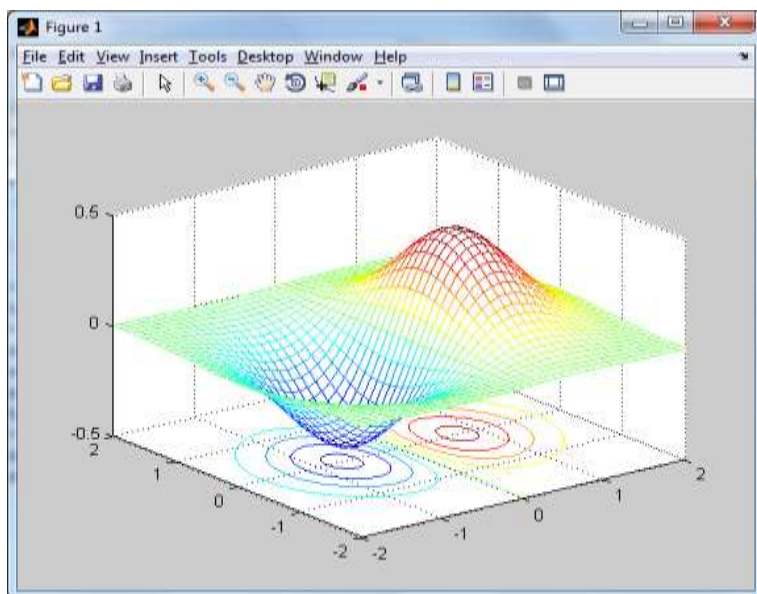


Рисунок 2.14. – Поверхность MESHС

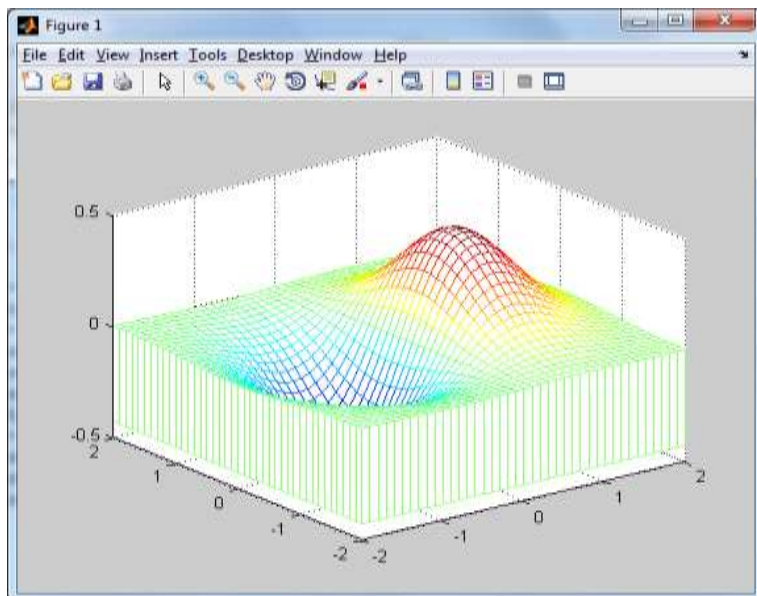


Рисунок 2.15. – Поверхность MESHZ

- SURF, SURFC - затененная сетчатая поверхность

Команда `surf(X, Y, Z, C)` выводит на экран сетчатую поверхность для значений массива `Z`, определенных на множестве значений массивов `X` и `Y`. Цвет ячейки определяется массивом `C`. Цвет ребер - черный, определяется свойством `EdgeColor`, специфицированным как `[0 0 0]`. Можно задать одинаковый цвет для всех ребер, определив его в виде вектора `[r g b]` интенсивности трех цветов - красного, зеленого, синего. Если определить спецификацию `none`, то ребра не будут прорисовываться.

Применение функции `shading` после обращения к функции `surf` изменяет спецификации свойств `EdgeColor` и `FaceColor` графического объекта `surface` согласно следующей таблице.

Таблица 2.6. – Изменение свойств поверхности

| Свойство               | Применяемая функция  |                           |                             |
|------------------------|----------------------|---------------------------|-----------------------------|
|                        | <code>surf</code>    | <code>shading flat</code> | <code>shading interp</code> |
| <code>EdgeColor</code> | <code>[0 0 0]</code> | <code>none</code>         | <code>none</code>           |
| <code>FaceColor</code> | <code>flat</code>    | <code>flat</code>         | <code>interp</code>         |

Команда `surf(x, y, Z, C)` выполняет ту же функцию, но вместо двумерных массивов `X`, `Y` использует их одномерные проекции, так что если `length(x) == n`, а `length(y) = m`, то `[m, n] = size(Z)`. В этом случае узлы сетчатой поверхности определяются тройками `{x(j), y(i), Z(i, j)}`, где вектор `x` определяет столбцы массива `Z`, а `y` - строки.

Команда `surf(Z, C)` использует сетку, которая определяется одномерными массивами `x = 1 : n` и `y = 1 : m`.

Команды `surf(X, Y, Z)`, `surf(x, y, Z)`, `surf(Z)` используют в качестве массива цвета `C = Z`, то есть цвет в этом случае пропорционален высоте поверхности.

Группа команд `surfc(...)` в дополнение к трехмерным затененным поверхностям строит проекцию линий постоянного уровня.

Функция `h = surf(...)` возвращает дескриптор `h` для графического объекта `surface`.

```
» [ X, Y ] = meshgrid([ -2 : 0.1 : 2 ]);  
» Z = X.*exp(- X .^ 2 - Y .^ 2);  
» surf(X, Y, Z)  
» surfc(X, Y, Z)
```

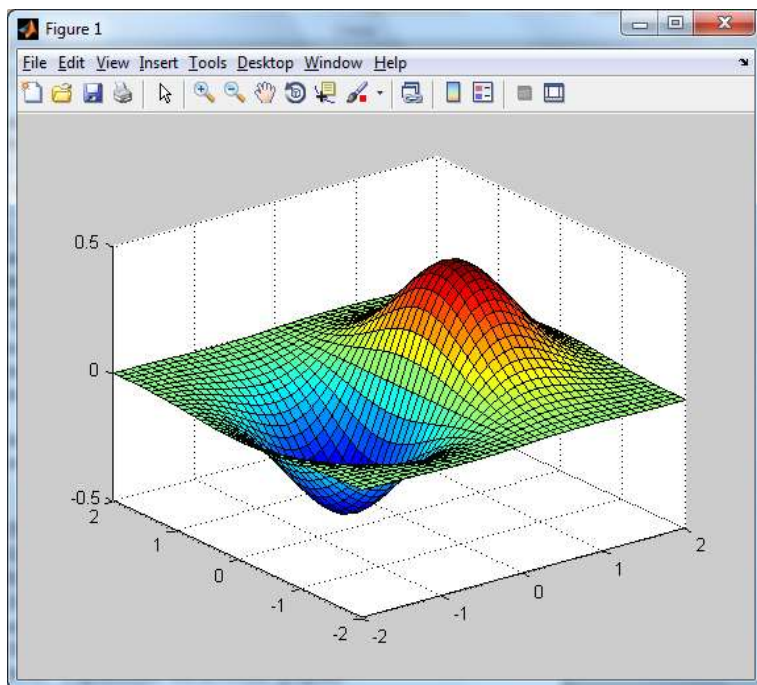


Рисунок 2.16. – Поверхность SURF



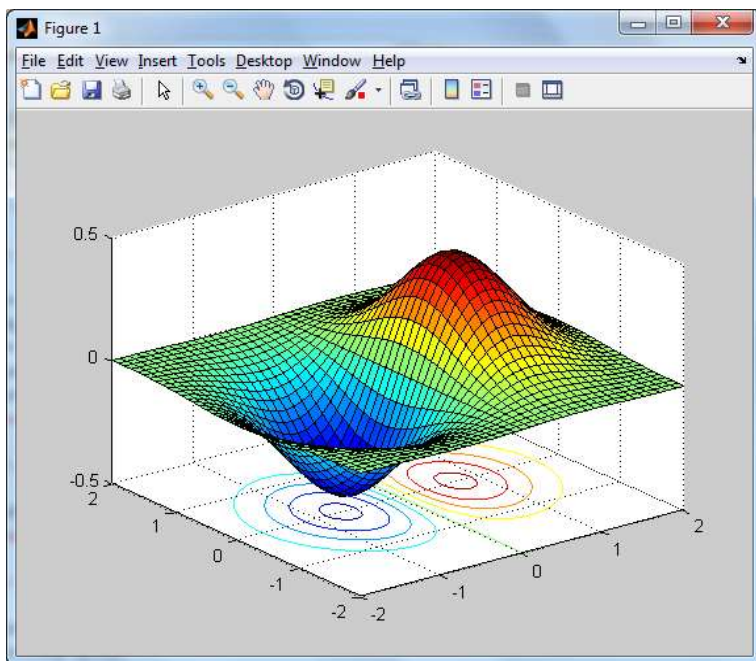


Рисунок 2.17. – Поверхность SURFC

- SURFL - затененная поверхность с подсветкой

Команда `surfl(X, Y, Z, s)` выводит на экран затененную поверхность с подсветкой для значений массива  $Z$ , определенных на множестве значений массивов  $X$  и  $Y$ . Направление на источник света может быть задано с помощью вектора  $s = [S_x, S_y, S_z]$  в декартовых координатах или вектора  $s = [az, elev]$  в сферических координатах. По умолчанию азимут  $az = -37.5^\circ$ , возвышение  $elev = 30^\circ$ . Подсветка учитывает модели рассеяния, отражения и зеркального эффекта освещения поверхности.

Команда `surfl(X, Y, Z, s, k)` позволяет управлять параметрами рассеяния, отражения и зеркального эффекта, используя вектор  $k = [k_a, k_d, k_s, spread]$ , который учитывает эффекты отраженного света  $k_a$ , диффузного отражения  $k_d$ ,

зеркального отражения  $k_s$  и зеркального распространения  $\text{spread}$ . По умолчанию вектор  $k$  имеет значения  $[0.55 \ 0.6 \ 0.4 \ 10]$ .

Команда `surf(X, Y, Z)` использует значения параметров по умолчанию.

Команды `surf(Z, ...)` строят графики, не учитывая истинных значений массивов  $X$  и  $Y$ .

```
» [X, Y] = meshgrid(-3 : 1/8 : 3);  
» Z = X.*exp(- X .^ 2 - Y .^ 2);  
» surf(X, Y, Z)
```

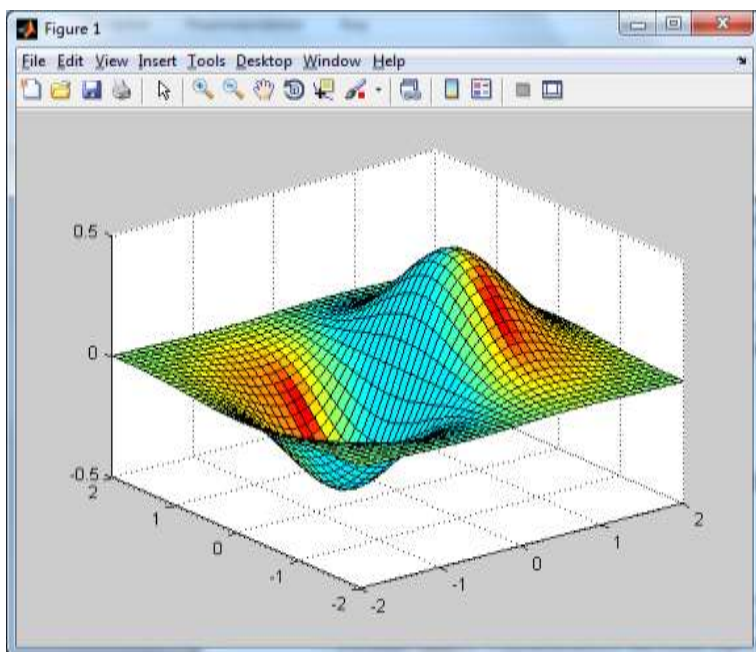


Рисунок 2.18. – Поверхность SURFL

- `CONTOURC` - формирование массива описания линий уровня

Функция  $C = \text{contourc}(Z)$  формирует массив  $C$  описания линий уровней для их вывода на экран с помощью функции `contour`.

Функция  $C = \text{contourc}(x, y, Z)$  формирует массив линий уровней  $C$  с учетом диапазона изменения координат  $x$  и  $y$ .

Функции  $C = \text{contourc}(Z, n)$  и  $C = \text{contourc}(x, y, Z, n)$  формируют массив  $C$  для  $n$  линий уровней.

Функции  $C = \text{contourc}(Z, v)$  и  $C = \text{contourc}(x, y, Z, v)$  формируют массив  $C$  только для линий уровня, которые заданы в векторе  $v$ .

- **CONTOUR** - изображение линий уровня для трехмерной поверхности

Команда `contour(Z)` рисует двумерные линии уровня для массива данных  $Z$ , определяющих поверхность в трехмерном пространстве без учета диапазона изменения координат  $x$  и  $y$ .

Команда `contour(x, y, Z)`, где  $x$  и  $y$  - векторы, рисует линии уровня для массива данных  $Z$  с учетом диапазона изменения координат  $x$  и  $y$ .

Команды `contour(Z, n)`, `contour(x, y, Z, n)` рисует  $n$  линий уровня для массива данных  $Z$ ; по умолчанию,  $n$  равно 10.

Команды `contour(Z, v)`, `contour(x, y, Z, v)` рисуют линии уровня для заданных значений, которые указаны в векторе  $v$ .

Команда `contour(..., '<тип линии>')` рисует линии уровня, тип и цвет которых определяются параметром `<тип линии>` команды `plot`.

Функция  $C = \text{contour}(...)$  возвращает массив  $C$  описания линий уровней по аналогии с функцией `contourc` для последующего использования командой `clabel`.

Функция  $[C, h] = \text{contour}(...)$  возвращает массив  $C$  и вектор-столбец дескрипторов  $h$  графических объектов `line` для каждой линии уровня.

```
» x = -2 : .2 : 2; y = x;  
» [X, Y] = meshgrid(x);  
» Z = X.* exp(- X.^2 - Y.^2);
```

» `contour(X, Y, Z)`

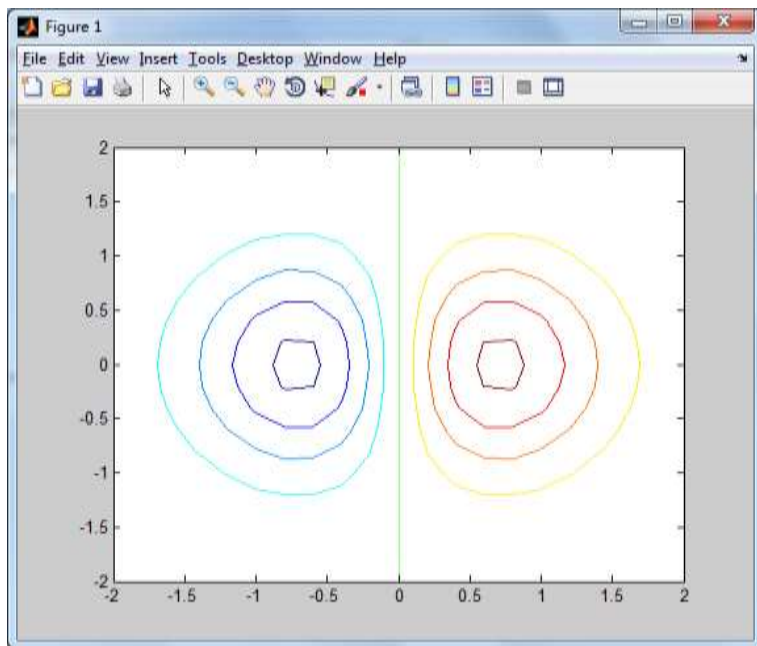


Рисунок 2.19. – Линии уровня CONTOUR

- **CONTOUR3** - изображение трехмерных линий уровня

Команда `contour3(Z)` рисует трехмерные линии уровня для массива данных `Z`, определяющих поверхность в трехмерном пространстве без учета диапазона изменения координат `x` и `y`.

Команда `contour3(X, Y, Z)`, где `X` и `Y` - двумерные массивы, вычисленные с помощью функции `meshgrid`, рисует линии уровня для массива данных `Z` с учетом диапазона изменения координат `x` и `y`.

Команды `contour3(Z, n)`, `contour(X, Y, Z, n)` рисуют `n` линий уровня для массива данных `Z`; по умолчанию `n` равно 10.

Функция `C = contour3(...)` возвращает массив `C` описания линий уровней по аналогии с функцией `contourc` для последующего использования командой `clabel`.

Функция  $[C, h] = \text{contour3}(\dots)$  возвращает массив  $C$  и вектор-столбец дескрипторов  $h$  графических объектов line для каждой линии уровня.

```
» x = -2 : .2 : 2;  
» y = -2 : .2 : 2;  
» [X, Y] = meshgrid(x, y);  
» Z = X.* exp(- X.^2 - Y.^2);  
» contour3(X, Y, Z)
```

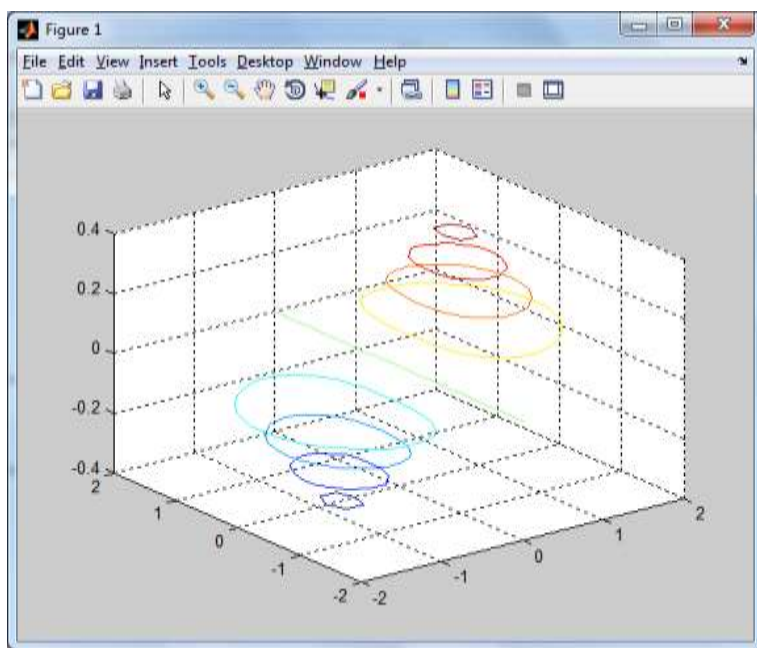


Рисунок 2.20. – Линии уровня CONTOUR3

- COMET3 - движение точки по пространственной траектории

Команда  $\text{comet}(z)$  рисует движение точки по траектории, заданной одномерным массивом  $z$ , в виде головы и хвоста кометы.

Команда `comet3(x, y, z)` рисует движение точки по траектории, заданной массивами  $x$  и  $y$ .

Команда `comet3(x, y, z, p)` управляет длиной хвоста кометы  $p \cdot \text{length}(z)$  с помощью параметра  $p$ ; по умолчанию  $p = 0.10$ .

```
» t = -pi : pi/500 : pi;  
» comet3(sin(5*t), cos(3*t), t)
```

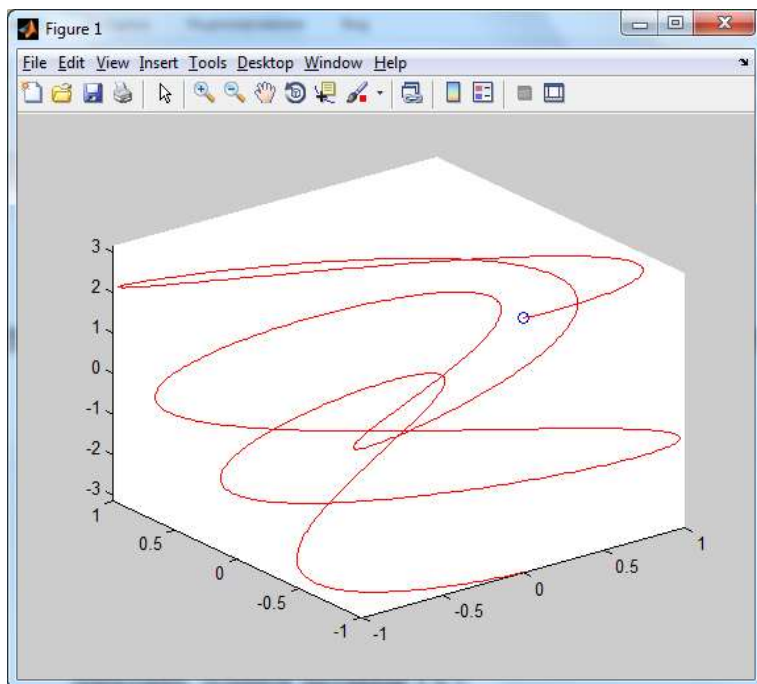


Рисунок 2.21. – Траектория COMET3

- SLICE - сечения функции от трех переменных

Команда `slice(x, y, z, V, xi, yi, zi, n)` строит плоские сечения функции от трех переменных  $v(x, y, z)$  вдоль осей  $x, y, z$ ; позиции сечений определяются векторами  $xi, yi, zi$ . Размер двумерного массива  $V$  равен  $m \times n \times p$ , где  $m = \text{length}(y)$ ,  $n = \text{length}(x)$ ,  $p = \text{length}(z)$ .

Команда `slice(X, Y, Z, V, xi, yi, zi, n)` вместо одномерных массивов использует двумерные массивы `X`, `Y`, `Z`, которые вычисляются с помощью функции `meshgrid`.

Команда `slice(V, xi, yi, zi, n)` использует для задания области построения массивы `x = 1 : n`, `y = 1 : m`, `z = 1 : p`.

Функция `h = slice(...)` возвращает вектор-столбец дескрипторов для графических объектов `surface`, которыми являются сечения трехмерной функции.

```
» x = -2 : .2 : 2;  
» y = -2 : .25 : 2;  
» z = -2 : .16 : 2;  
» [X, Y, Z] = meshgrid(x, y, z);  
» V = X .* exp(-X.^2 - Y.^2 - Z.^2);  
» slice(x, y, z, V, [2], [2], [ -0.75 0.5],  
    length(x)), grid
```

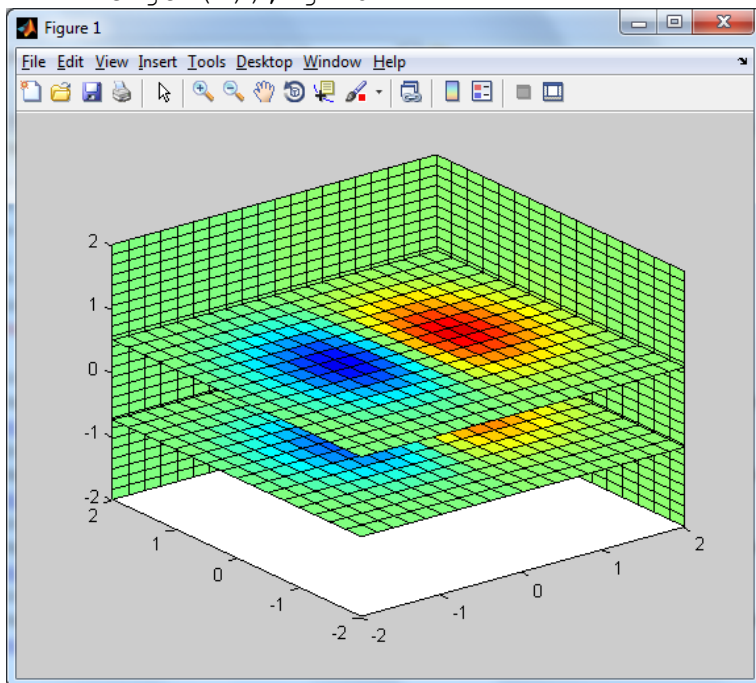


Рисунок 2.22. – Сечения функции SLICE

- WATERFALL - трехмерная поверхность

Команда `waterfall(X, Y, Z, C)` строит поверхность для значений массива `Z`, определенных на множестве значений массивов `X` и `Y`. Она аналогична команде `mesh`, но не прорисовывает ребер сетки.

Команда `waterfall(x, y, Z, C)` выполняет ту же функцию, но вместо двумерных массивов `X`, `Y` использует их одномерные проекции, так что если `length(x) = n`, а `length(y) = m`, то `[m, n] = size(Z)`.

Команда `waterfall(Z, C)` использует сетку, которая определяется одномерными массивами `x = 1 : n` и `y = 1 : m`.

Команды `waterfall(X, Y, Z)`, `mesh(x, y, Z)`, `mesh(Z)` используют в качестве массива цвета `C = Z`, то есть цвет в этом случае пропорционален высоте поверхности.

Функция `h = waterfall(...)` возвращает вектор дескрипторов `h` для графических объектов `patch`.

```
» [X, Y] = meshgrid([ -2 : 0.1 : 2 ]);  
» Z = X .*exp(- X.^2 - Y.^2);  
» waterfall(X, Y, Z)
```



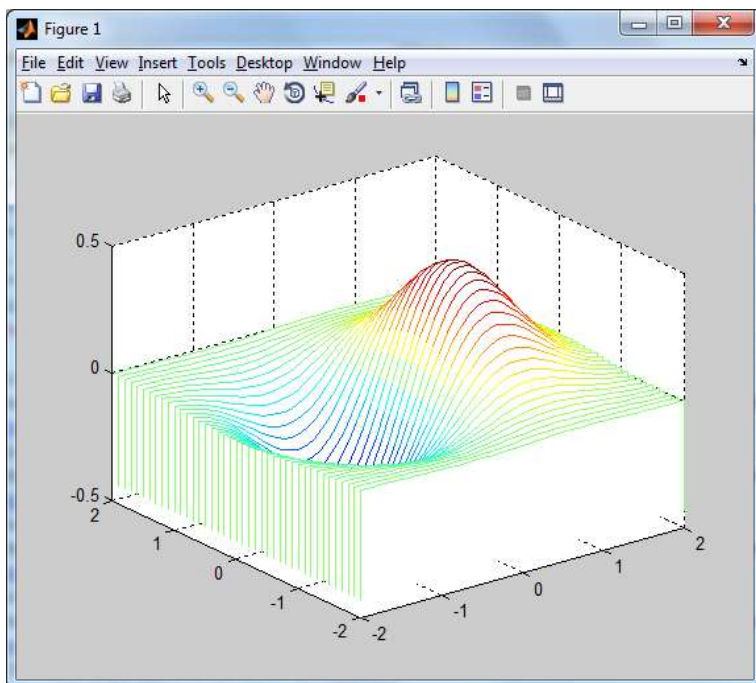


Рисунок 2.23. – Поверхность WATERFALL

### 2.8.3. Надписи, оформление и пояснения к графикам

- TITLE - заголовки для двух- и трехмерных графиков

Команда `title('<текст>')` размещает текст над графиком.

- XLABEL, YLABEL, ZLABEL - обозначение осей

Команда `xlabel('<текст>')` помещает текст для двумерного графика вдоль оси x, для трехмерного графика - вдоль оси x либо под графиком.

Команда `ylabel('<текст>')` помещает текст для двумерного графика вдоль оси y, для трехмерного графика - вдоль оси y либо под графиком.

Команда `zlabel('<текст>')` помещает текст вдоль оси трехмерного графика.

Повторное использование команды приводит к замене старого текста новым.

- CLABEL - маркировка линий уровня

Команда `clabel(C)` добавляет метки к линиям уровня в случайно выбранных позициях.

Команда `clabel(C, v)` маркирует линии уровня контура, которые заданы в векторе `v`.

Команда `clabel(C, 'manual')` маркирует линии уровней в позициях, указываемых с помощью мыши. Нажатие клавиши Return или правой кнопки мыши завершает маркировку. Если мышь недоступна, то можно использовать клавишу пробела для перехода между линиями уровней и клавиши-“стрелки” для перемещения перекрестия.

```
» x = -2 : .2 : 2; y = x;  
» [X, Y] = meshgrid(x);  
» Z = X.* exp(- X.^2 - Y.^2);  
» C = contour(X, Y, Z);  
» clabel(C)
```

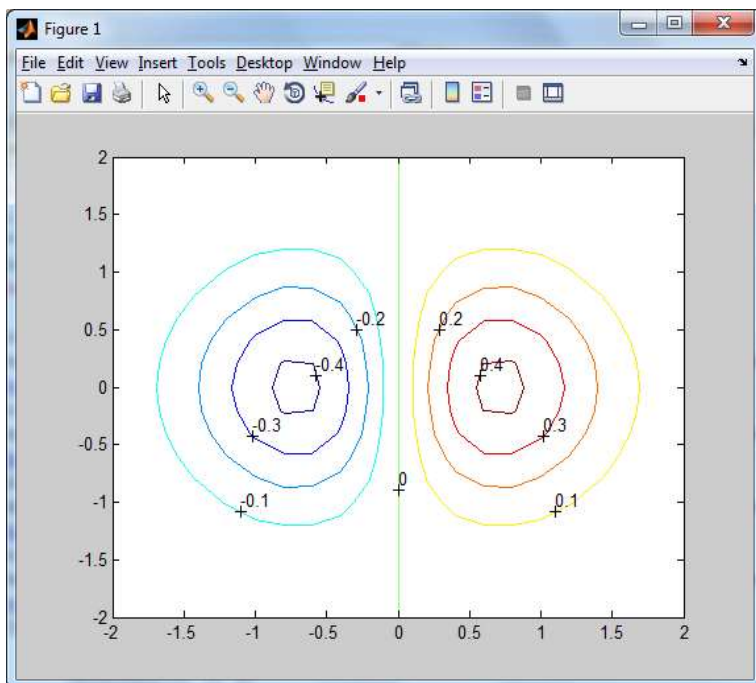


Рисунок 2.24. – Маркировка линий CLABEL

- TEXT - добавление к текущему графику текста

Команда `text(x, y, '<текст>')` помещает в заданной точке (x, y) двумерного графика начало текста, указанного в качестве третьего аргумента. Если x и y одномерные массивы, заданный текст помещается во все позиции, определяемые координатами [x(i) y(i)].

Команда `text(x, y, z, '<текст>')` выводит текст на трехмерный график.

- » `plot([1 5 10], [1 10 20], 'x')` – записывает символ 'x' в трех точках с координатами (1, 1), (5, 10), (10, 20):
- » `text(5, 10, 'Action point')` – записывает, начиная с позиции (5, 10), «Action

```

point»
» h = text(5, 10, 'Активная точка',
'FontName', 'TimesET', 'FontSize', 11)
- записывает, начиная с позиции (5,
10), «Активная точка»:

```

- GTEXT - размещает заданный текст на графике с использованием мыши

Команда `gtext('<текст>')` высвечивает в активном графическом окне перекрестие, перемещение которого позволяет указать место ввода заданного текста; по завершении позиционирования нажатие кнопки мыши или любой клавиши вводит заданный текст.

Для того чтобы настроить процедуру `gtext` на ввод русского текста, следует в тело процедуры вслед за строкой `set(ht, 'units', 'units')` добавить строку `set(ht, 'FontName', '<имя шрифта>')`

```

» x = 1e-6 : 0.05 : 10;
» plot(x, sin(x)./x)
» grid
» text('Скорость')

```

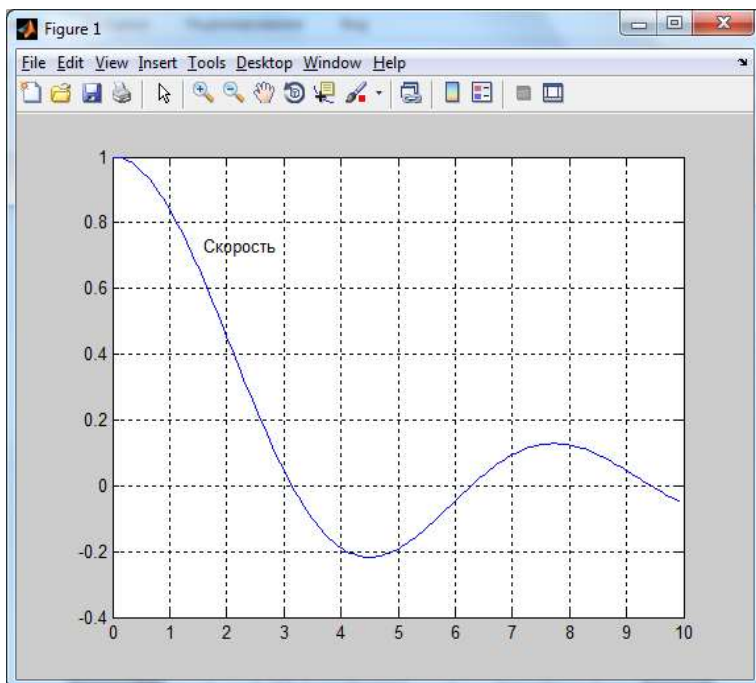


Рисунок 2.25. – Размещение текста на графике GTEXT

- **LEGEND** - пояснение к графику

Команда `legend(' <текст1>', ' <текст2>', ' <текст3>', ...)` добавляет к текущему графику пояснение в виде указанных текстовых строк.

Команда `legend(' <тип линии1>', ' <текст1>', ' <тип линии2>', ' <текст2>', ...)` позволяет специфицировать тип линии, которая выносится в пояснение, так, как это делается в команде `plot`.

Команда `legend(h, ...)` добавляет пояснение к графику с дескриптором `h`.

Команды `legend(M)` и `legend(h, M)`, где `M` - массив строк, также допустимы для формирования пояснения. Следует помнить, что строки массива `M` должны иметь одинаковую длину.

Команда `legend off` удаляет пояснение с текущего графика.

Команда `legend(..., n)` устанавливает предельное количество позиций для размещения пояснения. Если оказывается, что в области графика места недостаточно, график перестраивается и пояснение размещается вне пределов графика. Если  $n = -1$ , то пояснение размещается вне области графика. Если  $n = 0$ , то пояснение размещается в области графика, если места для этого достаточно.

Для перемещения пояснения следует нажать левую кнопку мыши, находясь в этой области, а затем переместить пояснение в нужную позицию.

- **COLORBAR** - шкала палитры

Команда `colorbar('vert')` добавляет к текущему графику вертикальную шкалу палитры.

Команда `colorbar('horiz')` добавляет к текущему графику горизонтальную шкалу палитры.

Команда `colorbar(h)` добавляет к графику с дескриптором `h` шкалу палитры. Размещение шкалы реализуется автоматически в зависимости от соотношения ширины и высоты графика.

Команда `colorbar` без аргументов размещает на текущем графике новую вертикальную шкалу палитры или обновляет прежнюю.

- **AXIS** - масштабирование осей и вывод на экран

Команда `axis` обеспечивает преемственность предшествующих версий системы MATLAB, ориентированных на символьную обработку, с версиями 4.x, ориентированными на графический интерфейс.

Команда `axis([xmin xmax ymin ymax])` устанавливает масштаб по осям  $x$ ,  $y$  для активного графического окна.

Команда `axis([xmin xmax ymin ymax zmin zmax])` устанавливает масштаб по осям  $x$ ,  $y$ ,  $z$  для активного графического окна.

Команда `axis('auto')` возвращает масштаб по осям к штатным значениям (принятым по умолчанию).

Команда `axis(axis)` фиксирует текущие значения масштабов для последующих графиков, как если бы был включен режим `hold`.

Функция `v = axis` возвращает вектор-строку масштабов по осям для активного графика. Если график двумерный, то `v` имеет 4 компонента; если трехмерный - 6 компонентов.

Команда `axis('ij')` перемещает начало отсчета в левый верхний угол, сохраняет положение осей и реализует отсчет по вертикальной оси из верхнего левого угла (матричная система координат).

Команда `axis('xy')` возвращает декартову систему координат; начало отсчета находится в нижнем левом углу; ось `x` горизонтальна и размечается слева направо, ось `y` вертикальна и размечается снизу вверх.

Команда `axis('square')` устанавливает одинаковый диапазон изменения переменных по осям.

Команда `axis('equal')` устанавливает масштаб, который обеспечивает одинаковые расстояния между метками по осям `x` и `y`.

Команда `axis('image')` устанавливает масштаб, который обеспечивает квадратные размеры пикселей.

Команда `axis('normal')` восстанавливает полноразмерный масштаб, отменяя масштабы, установленные командами `axis('square')` и `axis('equal')`.

Команда `axis('off')` снимает с осей их обозначения и маркеры.

Команда `axis('on')` восстанавливает на осях их обозначения и маркеры.

Функция `[s1, s2, s3] = axis('state')` возвращает строку, определяющую вектор состояния объекта `axes`:

- `s1 = 'auto' | 'manual'.`
- `s2 = 'on' | 'off'.`
- `s3 = 'xy' | 'ij'.`

Команда `axis(s1, s2, s3)` устанавливает параметры объекта `axes` в соответствии с вектором состояния `[s1, s2, s3]`. По умолчанию этот вектор принимает значения `['auto', 'on', 'xy']`.

- GRID - нанесение сетки

Команда `grid on` наносит координатную сетку на текущие оси.

Команда `grid off` удаляет координатную сетку.

Команда `grid` выполняет роль переключателя с одной функции на другую.

Команды группы `grid` выполняют установку свойств `'XGrid', 'YGrid', 'ZGrid'` объекта `axes`.

- HOLD - управление режимом сохранения текущего графического окна

Команда `hold on` включает режим сохранения текущего графика и свойств объекта `axes`, так что последующие команды приведут к добавлению новых графиков в графическом окне.

Команда `hold off` выключает режим сохранения графика.

Команда `hold` реализует переключение от одного режима к другому.

Команды `hold` воздействуют на значения свойства `NextPlot` объектов `figure` и `axes`:

- `hold on` присваивает свойству `NextPlot` для текущих объектов `figure` и `axes` значение `add`;
- `hold off` присваивает свойству `NextPlot` для текущих объектов `figure` и `axes` значение `replace`.

- SUBPLOT - разбиение графического окна

Данная команда выполняется перед обращением к функциям построения графиков для одновременной выдачи нескольких графиков в различных частях графического окна.

Команды `subplot(mnp)` или `subplot(m, n, p)`, где `mnp` - 3 цифры, производит разбивку графического окна на несколько



подокон, создавая при этом новые объекты `axes`; значение `m` указывает, на сколько частей разбивается окно по горизонтали, `n` - по вертикали, а `p` - номер подокна, куда будет выводиться очередной график. Эти же команды могут использоваться для перехода от одного подокна к другому.

Команда `subplot(h)`, где `h` - дескриптор для объекта `axes` соответствующего подокна, - другой способ выбора подокна для размещения графика.

Команды `clf`, `subplot(111)`, `subplot(1, 1, 1)` выполняют одну и ту же функцию - удаляют все подокна и возвращают графическое окно в штатное состояние.

```
» x = -1:.1:1;
» y1 = sin(x);
» subplot(2, 1, 1), plot(x, y1)
» y2 = log(abs(y1));
» subplot(2, 1, 2), plot(x, y2)
```

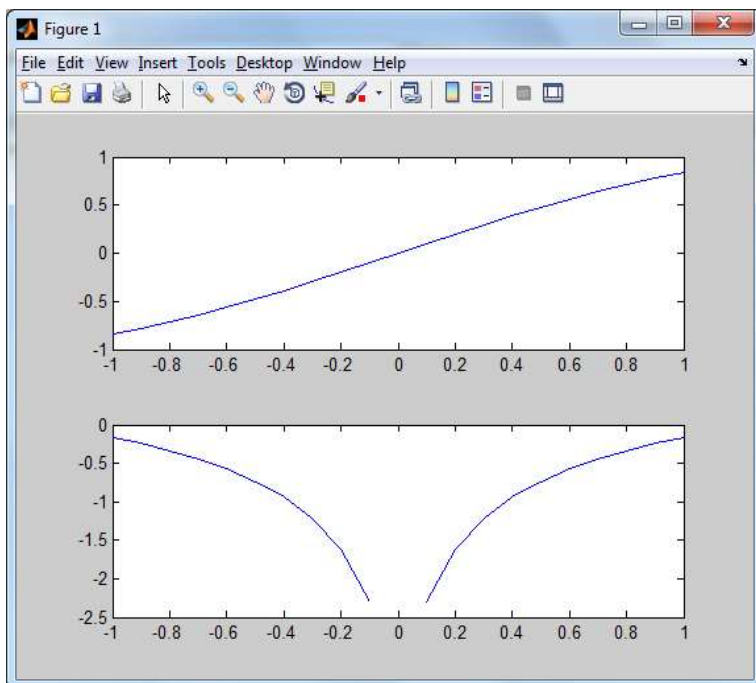


Рисунок 2.26. – Разбиение графического окна

- ZOOM - управление масштабом графика

Команда `zoom on` включает режим масштабирования активного графика. Теперь при нажатии левой клавиши мыши вблизи интересующей вас точки графика масштаб окна увеличивается в 2 раза; при нажатии правой клавиши масштаб в 2 раза уменьшается. Удерживая левую клавишу, можно выделить интересующую вас область окна.

Команда `zoom off` выключает режим масштабирования.

Команда `zoom` реализует переключение от одного режима к другому.

Команда `zoom out` возвращает график в исходное состояние.

- COLORMAP - палитра цветов

Палитра цветов  $C$  - это матрица размера  $m \times 3$  действительных чисел из диапазона  $[0.0 \ 1.0]$ . Строка  $k$  палитры сформирована из трех чисел, которые указывают интенсивность красного, зеленого и синего цветов, то есть  $C(k, :) = [r(k) \ g(k) \ b(k)]$ .

Команда `colormap(C)` устанавливает палитру согласно матрице  $C$ . Если значение элемента матрицы выходит за пределы интервала  $[0 \ 1]$ , выдается сообщение об ошибке

Команды `colormap('default')` или `colormap(hsv)` устанавливают штатную палитру, которая соответствует модели hue-saturation-value (оттенок-насыщенность-значение). Последовательность цветов этой палитры соответствует цветам радуги: red, yellow, green, cyan, blue, magenta.

В MATLAB реализованы следующие палитры:

Таблица 2.7. – Цветовые палитры

| Обозначение | Палитра                                                                              |
|-------------|--------------------------------------------------------------------------------------|
| bone        | Серая палитра с оттенком синего                                                      |
| cool        | Палитра с оттенками голубого и фиолетового                                           |
| copper      | Линейная палитра в оттенках меди                                                     |
| flag        | Палитра с чередованием красного, синего и черного                                    |
| gray        | Линейная палитра в оттенках серого                                                   |
| hot         | Палитра с чередованием черного, красного, желтого и белого                           |
| hsv         | Палитра радуги                                                                       |
| jet         | Разновидность hsv-палитры                                                            |
| pink        | Розовая палитра с оттенками пастели                                                  |
| prism       | Палитра с чередованием красного, оранжевого, желтого, зеленого, синего и фиолетового |
| white       | Белая палитра                                                                        |

- **CAXIS** - установление соответствия между палитрой цветов и масштабированием осей

Команда `saxis([cmin cmax])` устанавливает диапазон из палитры цветов для масштабирования данных, которые

используются объектами `surface` и `patch`, созданными с помощью команд `mesh`, `pcolor`, `surf`. Параметры `cmin` и `cmax` предназначены для выбора граничных цветов из текущей палитры. Значения цветов вне интервала `[cmin cmax]` отсекаются, то есть соответствующие фрагменты и пиксели не выводятся на экран.

Команда `caxis('auto')` устанавливает штатное масштабирование данных палитрой цветов; в этом случае отсекаются цвета, которые соответствуют значениям `Inf` и `NaN`.

- SHADING - затенение поверхностей

Команды группы `shading` устанавливают способ затенения графических объектов `surface` и `patch`, которые создаются при использовании функций `mesh`, `surf`, `pcolor`, `fill` и `fill3`.

Команда `shading faceted` устанавливает равномерную раскраску ячеек с нанесением черных граней. Такое затенение поверхности часто оказывается наиболее эффективным и принято по умолчанию.

Команда `shading flat` устанавливает раскраску каждой ячейки или грани определенными цветами, которые зависят от цвета узлов сетки.

Команда `shading interp` устанавливает раскраску каждой ячейки или грани цветами, которые определяются билинейной интерполяцией цветов в узлах сетки.

### 3. ИСПОЛЬЗОВАНИЕ ПАКЕТА SIMULINK ДЛЯ СОЗДАНИЯ ИМИТАЦИОННОЙ ИНФОРМАЦИОННО-МАТЕМАТИЧЕСКОЙ МОДЕЛИ

#### 3.1. Основные приемы работы в среде Simulink

Создать новый файл модели можно с помощью команды File/New/Model, или используя соответствующую кнопку на панели инструментов.

Чтобы расположить блоки в окне модели необходимо открыть соответствующий раздел библиотеки. Далее, указав курсором на требуемый блок и нажав на левую клавишу «мыши» - «перетащить» блок в созданное окно.

Для удаления блока необходимо выбрать блок а затем нажать клавишу **Delete** на клавиатуре.

Для изменения параметров блока необходимо дважды щелкнуть левой клавишей «мыши», указав курсором на изображение блока. Откроется окно редактирования параметров данного блока. При задании численных параметров следует иметь в виду, что в качестве десятичного разделителя должна использоваться точка, а не запятая.

После установки на схеме всех блоков из требуемых библиотек нужно выполнить соединение элементов схемы. Для соединения блоков необходимо указать курсором на «выход» блока, а затем, нажав и, не отпуская левую клавишу «мыши», провести линию к входу другого блока. После чего отпустить клавишу. В случае правильного соединения изображение стрелки на входе блока изменяет цвет. Для создания точки разветвления в соединительной линии нужно подвести курсор к предполагаемому узлу и, нажав правую клавишу «мыши», протянуть линию. Для удаления линии требуется выбрать линию (так же, как это выполняется для блока), а затем нажать клавишу **Delete** на клавиатуре.

Для повышения наглядности модели удобно использовать текстовые надписи. Для создания надписи нужно указать мышью место надписи и дважды щелкнуть левой клавишей мыши. После этого появится прямоугольная рамка с курсором ввода. Аналогичным образом можно изменить и подписи к блоками моделей.

## 3.2. Библиотека Simulink

Все блоки Simulink структурированы по своему назначению и объединены в следующие разделы библиотеки:

### 1. Раздел Sources (Источники)

Блоки, входящие в раздел «Источники», предназначены для описания рабочей нагрузки моделируемой системы, а также для формирования сигналов, обеспечивающих управление работой модели в целом или отдельных ее частей.

Все блоки-источники имеют по одному выходу и не имеют входов.

Основные блоки раздела Sources:

1. **Clock**. Отображение и обеспечение времени моделирования. Обеспечивает представление текущего значения модельного времени на очередном шаге моделирования и используется при моделировании непрерывных систем.

2. **Constant**. Генерация постоянного значения. Данный блок генерирует определенное значение, независимое от времени. Блок генерирует один выход, который может быть скалярным или векторным. Пиктограмма блока отображает определенное значение или значения.

3. **Digital Clock**. Источник дискретного временного сигнала. Применяется в случаях, когда необходимо знать текущее время моделирования внутри дискретной системы. Его особенность состоит в том, что он не только формирует величину шага, но и вычисляет новое значение модельного времени, которое используется для проверки условия окончания моделирования. Очередное значение модельного времени вычисляется как сумма предыдущего значения и величины шага моделирования.

4. **Random Number**. Генерирование нормально распределенных случайных чисел. Блок обеспечивает формирование сигналов, амплитуда которых является

случайной величиной, распределенной по нормальному закону с заданными параметрами.

5. **Uniform Random Number**. Генерирование однородно распределенных случайных чисел. Блок однородного случайного числа генерирует однородно распределенные случайные числа на определенном интервале с определенным начальным значением. Значение сбрасывается каждый раз при начале моделирования. Сгенерированная последовательность является повторяющейся и может быть произведена любым блоком однородного случайного числа (**Uniform Random Number block**) с тем же начальным значением и параметрами.

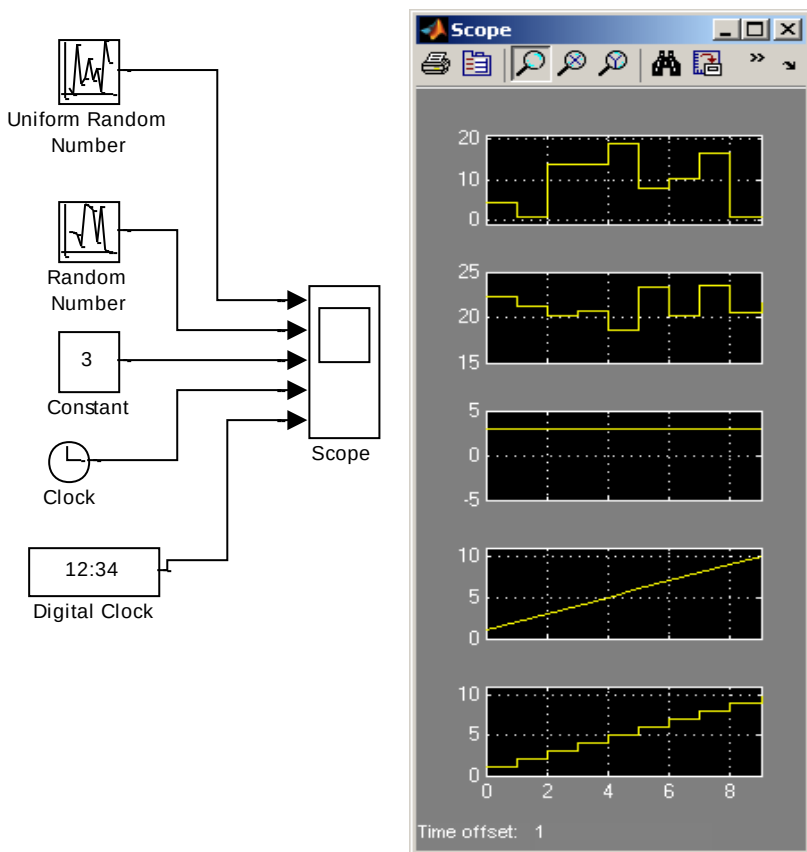


Рисунок 3.1. Демонстрация возможностей блоков раздела Sources

## 2. Раздел Sinks (Получатели)

Условно блоки данного раздела можно разделить на 3 вида:

- Блоки, используемые при моделировании в качестве «смотровых окон»: **Scope**, **XYGraph**, **Display**.
- Блоки, обеспечивающие сохранение промежуточных или входных результатов моделирования: **To File**, **To Workspace**.
- Блок управления моделированием: **Stop Simulation**.



Блоки раздела Sinks:

1. **Scope**. Отображение сигналов, генерируемых во время моделирования. Экран отображает свой вход как функцию от времени моделирования. Блок принимает один вход и может отобразить до 30 сигналов. Экран (Scope block) позволяет контролировать количество времени и область отображаемых входных значений. Если сигнал непрерывный, экран (Scope block) выводит график, нарисованный по точкам. Если сигнал дискретный, блок выводит ступенчатый график. Блок позволяет в процессе моделирования наблюдать динамику изменения интересующих исследователя характеристик системы. По оси ординат шкалы измерений откладываются значения наблюдаемой величины, по оси абсцисс – значения модельного времени.

2. **Display**. Отображение значения входа. Предназначен для вывода на экран численных значений величин, фигурирующих в модели.

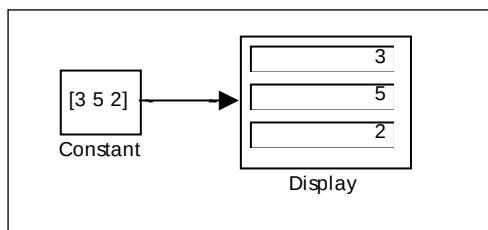


Рисунок 3.2. - Вывод блоком Display векторных величин

3. **Stop Simulation**. Прекращение моделирования при ненулевом входе. Блок останова моделирования (Stop Simulation block) прекращает моделирование при ненулевом входе. Моделирование заканчивает текущий временной интервал прежде, чем остановиться. Если вход блока векторный, любой ненулевой элемент вектора вызывает останов моделирования. Вы можете использовать этот блок в соединении с оператором отношения (Relational Operator block) для контроля, когда останавливается моделирование.

4. **To File**. Блок обеспечивает запись в MAT-файл данных, полученных в ходе моделирования.

5. **XY Graph**. Отображение графика сигналов с помощью окна рисунков MATLAB. Первый вход предназначен для ввода аргумента, второй – для ввода значений функций. Блок обеспечивает построение двумерных графиков зависимостей произвольных величин, фигурирующих в модели. Блок имеет два входа, первый из которых предназначен для ввода аргумента, второй – для ввода значений функции этого аргумента.

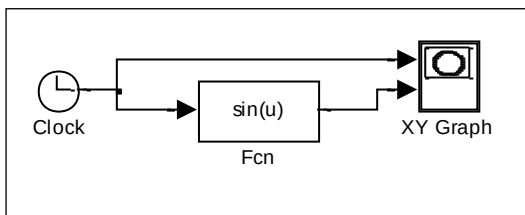


Рис 3.3. - Построение графика функции  $\sin(x)$

6. **To Workspace**. Запись данных в матрицу рабочей области. Блок **To Workspace** записывает свой вход в определенную матрицу рабочей области. Блок записывает свой вход построчно, при этом каждая строка содержит все элементы входного вектора на временном шаге. Если матрица уже существует, ее содержимое перезаписывается.

### 3. Раздел Math Operations

Данный раздел содержит блоки, которые реализуют элементарные алгебраические и тригонометрические функции.

1. **Abs**. Вывод абсолютного значения входа. Данный блок генерирует в качестве выхода абсолютное значение входа. Блок принимает один вход и генерирует один выход.

2. **Algebraic Constraint**. Ограничение входного сигнала до нуля. Данный блок ограничивает входной сигнал **f(z)** до нуля и выводит алгебраическое состояние **z**. Блок выводит значение, необходимое для того, чтобы произвести нуль на входе. Выход должен влиять на вход через какую-нибудь обратную связь.

3. **Dot Product**. Генерирование произведения. Блок генерирует произведение двух векторов.

4. **Gain**. Усиление входа блока. Усилитель (**Gain block**) генерирует свой выход путем умножения входа на определенную константу, переменную или выражение.

5. **Math Function**. Выполнение математической функции. Блок математической функции (**Math Function block**) выполняет многочисленные математические функции. Название функции отображается на пиктограмме блока. SIMULINK автоматически рисует нужное число портов.

6. **MinMax**. Вывод минимального или максимального входного значения. Блок минимакса (**MinMax block**) выводит или минимальный, или максимальный элемент или элементы входа (входов). Можно выбрать, какую функцию применять, выбрав ее из списка параметра **Function**. Если блок содержит один входной порт, то он выводит скаляр, т.е., минимальный ли максимальный элемент входного вектора. Если блок содержит несколько входных портов, то он выполняет поэлементное сравнение входных векторов. Каждый элемент выходного вектора блока является результатом сравнения входных векторов.

7. **Product**. Генерирование произведения или частного входов блока. Блок произведения (**Product block**) умножает или делит входы блока в зависимости от значения параметра **Number of inputs**: Если значение представляет собой комбинацию символов « \* » и « / », число входов блока равно числу символов. Пиктограмма блока показывает соответствующий символ рядом с каждым входным портом. Выход блока равен произведению всех входов, отмеченных « \* », деленному на все входы, отмеченные « / ».

**8. Rounding Function.** Выполнение функции округления. Блок функции округления (**Rounding Function block**) выполняет общие функции округления.

**9. Sign.** Отображение знака входа. Блок знака (**Sign block**) показывает знак входа: 1, если вход больше нуля, 0 – если вход равен нулю, -1 – если вход меньше нуля.

**10. Sum.** Генерирование суммы входов. Сумматор (**Sum block**) складывает скалярные и/или векторные входы, или элементы единичного векторного входа, в зависимости от числа входов блока. Если у блока более одного входа, выход блока равен поэлементной сумме входов. Если все входы скалярные, выход тоже скаляр. Если у блока один векторный вход, выход блока скалярная сумма элементов входа.

**11. Trigonometric Function.** Блок обеспечивает выполнение тригонометрической функции. Блок тригонометрической функции (**Trigonometric Function block**) вычисляет общие тригонометрические функции. Можно выбрать одну из этих функций из списка **Function**: sin, cos, tan, asin, acos, atan, atan2, sinh, cosh, tanh. Выход блока – результат вычисления функции над входом или входами. На пиктограмме блока появляется имя функции. SIMULINK автоматически рисует необходимое число входных портов.

#### 4. Раздел Discrete (Дискретные системы)

В данный раздел входят блоки, с помощью которых в модели может быть описано поведение дискретных систем.

Блоки раздела Discrete:

**1. Memory.** Вывод входа блока на предыдущем шаге интегрирования. Блок памяти (**Memory block**) выводит свой вход на предыдущем интервале времени.

## 5. Раздел Discontinuities

Данный раздел содержит достаточно универсальные блоки, которые могут быть использованы при построении моделей систем различных типов.

Блоки раздела:

1. **Dead Zone**. Обеспечение области нулевого выхода. Блок мертвой зоны (Dead Zone block) генерирует нулевой выход внутри определенной области, называемой его мертвой зоной. Нижний и верхний пределы мертвой зоны определяются как параметры **Start of dead zone** и **End of dead zone**. Выход блока зависит от входа мертвой зоны. Если вход находится внутри мертвой зоны (больше чем нижний предел и меньше чем верхний предел), выход равен нулю. Если вход больше чем или равен верхнему пределу, выход равен входу минус верхний предел. Если вход меньше чем или равен нижнему пределу, выход равен входу минус нижний предел. Если нижний и верхний пределы равны, входной сигнал проходит через блок и выводится как выход

2. **Relay**. Переключение выхода между двумя константами. Реле (Relay block) позволяет переключать выход между двумя определенными значениями. Реле остается включенным до тех пор, пока значение входа не опустится ниже значения параметра **Switch off point**. Реле остается выключенным до тех пор, пока вход не достигнет значения параметра **Switch on point**. Блок принимает один вход и генерирует один выход.

3. **Saturation**. Ограничение диапазона сигнала. Блок насыщения (Saturation block) накладывает верхнюю и нижнюю границу на сигнал. Когда входной сигнал находится внутри диапазона, определенного параметрами **Lower limit** и **Upper limit**, то он проходит через блок без изменений. Когда входной сигнал выходит за эти границы, то он отсекается верхней или нижней границей

## 6. Раздел Logic and Bit Operations

**1. Relational Operator.** Выполнение определенной операции отношения. Оператор отношения (Relational Operator block) выполняет операцию отношения с двумя входами и производит соответствующий выход.

## 7. Раздел Signal Routing

Данный раздел блоков является, пожалуй, самым полезным для имитационного моделирования.

**1. Demux.** Разделение векторного сигнала на выходные сигналы. Демультимплексор (Demux block) разделяет векторный входной сигнал на выходные линии, каждая из которых может содержать скалярный или векторный сигнал. Если параметр **Number of outputs** скалярный, блок разделяет входной сигнал на данное число выходных сигналов. Ширина выходных сигналов зависит от ширины входного вектора и количества выходов. Если ширина входного сигнала равна количеству выходов, блок разделяет вектор входного сигнала на скалярные сигналы. Если ширина входного сигнала делится нацело на число выходов, блок разделяет входной сигнал на векторные сигналы равной ширины. Если ширина входного сигнала не делится нацело на число выходов (и они не одинаковы), блок разделяет входной сигнал на векторные сигналы неравной ширины, и SIMULINK выдает предупреждение.

**2. From.** Прием входа от блока перехода (Goto block). Блок Из (From block) принимает сигнал от соответствующего ему блока перехода (Goto block), затем выводит его в качестве выхода. Блоки Из и перехода позволяют передавать сигнал от одного блока к другому без реального соединения этих блоков. Каждый блок Из (From block) связан с блоком перехода (Goto block). Вход блока перехода (Goto block) передается блоку Из (From block), который затем передает его блоку, соединенному с ним. Чтобы связать блок перехода (Goto block) с блоком Из (From block), введите признак блока перехода (Goto block) как параметр **Goto tag**. Блок Из (From block) может получать сигнал

только от одного блока перехода (Goto block), а блок перехода (Goto block) может передавать свой сигнал нескольким блокам Из (From block).

**3. Goto.** Передача входа блока блокам Из (From block). Блок перехода (Goto block) передает свой вход соответствующим блокам Из (From block). Блоки Из (From block) и перехода (Goto block) позволяют передавать сигнал от одного блока к другому без реального соединения между ними. Блок перехода (Goto block) может передавать свой входной сигнал нескольким блокам Из (From block), однако блок Из (From block) может получать сигнал только от одного блока перехода (Goto block). Вход блока перехода (Goto block), с которым связаны блоки Из (From block), передается так, как будто блоки соединены физически. Об ограничениях на использование блоков Из (From block) и перехода (Goto block) см. описание блока Из (From block) на странице 9–59. Блоки перехода (Goto block) и Из (From block) сочетаются по использованию признаков перехода, определенных как параметр **Tag**.

**4. Manual Switch.** Переключение между двумя входами. Ручной переключатель (Manual Switch block) – это ключ, который выбирает один из двух входов, через который будет происходить передача на выход. Чтобы переключиться между входами, дважды щелкните на пиктограмме блока (диалогового окна не существует). Выбранный вход передается на выход в то время, как невыбранный вход игнорируется. Вы можете установить переключатель до начала моделирования или переключить его во время моделирования для интерактивного управления ходом сигнала. Ручной переключатель (Manual Switch block) принимает все типы сигналов и сохраняет свое текущее состояние при сохранении модели.

**5. Multiport Switch.** Выбор между входами блока. Блок многопортового переключения (Multiport Switch) выбирает один вход среди нескольких. Первый (верхний) вход – это управляющий вход, а другие входы – входы переключения. Значение управляющего входа определяет, какой вход

переключения пройдет через выходной порт. SIMULINK округляет значение управляющего входа до ближайшего (положительного) целого и пропускает соответствующий этому значению вход переключения.

6. **Mux**. Объединение нескольких входных линий в векторную линию. Мультиплексор (Mux block) объединяет нескольких входных линий в одну векторную линию. Каждая входная линия может содержать скалярный или векторный сигнал. Выход мультиплексора (Mux block) – вектор. Если вы определите параметр **Number of inputs** как скаляр, SIMULINK определит ширину входов путем проверки выходных портов блоков, подающих входные сигналы на мультиплексор (Mux block). Если какой-нибудь вход – вектор, все его элементы объединяются блоком.

7. **Selector**. Выбор входных элементов. Селектор (Selector block) генерирует в качестве выхода выбранные элементы входного вектора. Параметр **Elements** определяет порядок элементов входного вектора в выходном векторе. Параметр должен быть определен как вектор, иначе будет выбран только один элемент. Пиктограмма блока показывает порядок элементов входного вектора графически. Если блок достаточно небольшой, на пиктограмме отображается имя блока.

8. **Switch**. Переключение между двумя входами. Ключ (Switch block) пропускает один из двух входов на выход в зависимости от значения третьего входа, называемого управляющим входом. Если сигнал на управляющем (втором) входе больше чем или равен параметру **Threshold**, блок пропускает первый вход; иначе, он пропускает третий вход.

## 8. Раздел User-Defined Functions

1. **Fcn**. Применение определенного выражения к входу. Блок функции (Fcn) применяет определенное выражение языка C к входу. Выражение может быть составлено из одного или нескольких следующих компонентов:



- $u$  – вход блока. Если  $u$  – вектор,  $u(i)$  представляет собой  $i$ -ый элемент вектора;  $u(1)$  или просто  $u$  представляет собой первый элемент

- Числовые константы
- Арифметические операторы ( $+$   $-$   $*$   $/$ )
- Операторы отношений ( $=$   $\neq$   $>$   $<$   $\geq$   $\leq$  ). Выражение возвращает 1, если отношение истинно; иначе возвращает 0.

- Логические операторы ( $\&\&$   $\parallel$   $!$ ). Выражение возвращает 1, если отношение истинно; иначе возвращает 0.

- Скобки
- Математические функции –  $\text{abs}$ ,  $\text{acos}$ ,  $\text{asin}$ ,  $\text{atan}$ ,  $\text{atan2}$ ,  $\text{ceil}$ ,  $\text{cos}$ ,  $\text{cosh}$ ,  $\text{exp}$ ,  $\text{fabs}$ ,  $\text{floor}$ ,  $\text{hypot}$ ,  $\text{ln}$ ,  $\text{log}$ ,  $\text{log10}$ ,  $\text{pow}$ ,  $\text{power}$ ,  $\text{rem}$ ,  $\text{sign}$ ,  $\text{sin}$ ,  $\text{sinh}$ ,  $\text{sqrt}$ ,  $\text{tan}$ ,  $\text{tanh}$ .

- Переменные рабочей области. Имена переменных, которые не распознаются в списке переменных, передаются для оценки в MATLAB. Элементы векторов и матриц должны быть обозначены специально (например,  $A(1,1)$ , вместо  $A$  для первого элемента матрицы).

2. **MATLAB Fcn.** Применение функции или выражения MATLAB к входу. Блок функции MATLAB (MATLAB Fcn block) применяет определенную функцию или выражение MATLAB к входу. Блок принимает один вход и генерирует один выход. К входу применяется определенная функция или выражение. Выход функции должен соответствовать ширине выхода блока, иначе возникнет ошибка.

3. **Level-2 M-file S-Function.** Данный блок позволяет использовать в модели собственные M-функции.

## 9. Раздел Commonly Used Blocks

1. **Ground.** Заземление неподсоединенного входного порта. Земля (Ground block) используется для соединения блоков, чьи входные порты не подсоединены к другим блокам. Если вы запустите моделирование с блоками, имеющими неподсоединенные входные порты, SIMULINK выведет предупредительное сообщение. Использование земли (Ground block) для «заземления» таких блоков позволяет избежать

предупредительных сообщений. Земля (Ground block) выводит сигнал с нулевым значением.

**2. Terminator.** Завершение не присоединенного выходного порта. Наконечник (Terminator block) может быть использован для покрытия блоков, чьи выходные порты не соединены с другими блоками. Если вы запустите моделирование с блоками, имеющими не присоединенные выходные порты, SIMULINK выдаст предупредительное сообщение. Использование наконечников (Terminator block) для покрытия таких блоков помогает избежать предупредительных сообщений.

### **3.3. Использование S-функций для реализации математических моделей**

S-функции позволяют, используя язык MATLAB, создавать полноценные пользовательские блоки SIMULINK, имеющие несколько входных и выходных портов. Типы входных и выходных сигналов функции могут любыми, в том числе матричными.

Каждая S-функция включает в себя следующие разделы:

- Раздел определения количества входных и выходных портов блока.
- Раздел настройки атрибутов блока, таких как размеры входных и выходных портов, а также типы сигналов, проходящих через эти порты.
- Раздел настройки количества параметров блока и проверки обоснованности этих параметров.
- Раздел регистрации основных методов блока.

В разделе описания входных и выходных портов и параметров блока необходимо указать количество этих портов и параметров:

- block.NumDialogPrms – количество параметров блока,
- block.NumInputPorts – количество входных портов блока,
- block.NumOutputPorts – количество выходных портов блока.

В разделе описания атрибутов входных и выходных портов блока указывают:

- `block.OutputPort(n).Dimensions` - Размерность сигнала, проходящего через порт `n`.
- `block.OutPort(n).DatatypeID = ID` – тип сигнала, проходящего через порт `n`. Переменная `ID` может принимать следующие значения:
  - 0 – для типа данных `Double`,
  - 1 – для типа данных `Single`,
  - 2 – для типа данных `Int8`,
  - 3 – для типа данных `UInt8`,
  - 4 – для типа данных `Int16`,
  - 5 – для типа данных `UInt16`,
  - 6 – для типа данных `Int32`,
  - 7 – для типа данных `UInt32`,
  - 8 – для типа данных `Boolean`
- `block.SetPreCompInPortInfoToDynamic` – метод, показывающий, что входные порты блока наследуют состав своих свойств из модели.
- `block.SetPreCompOutPortInfoToDynamic` – метод, показывающий, что выходные порты блока наследуют состав своих свойств из модели.

В разделе регистрации основных методов блока указываются функции, используемые внутри S-функции.

- `block.RegBlockMethod('Outputs', @Output)` – Метод получения выходных сигналов
- `block.RegBlockMethod('SetInputPortDimensions', @SetInPortDims)` – метод определения размерностей входных портов.

## 4. ПРИМЕР РАЗРАБОТКИ ИМИТАЦИОННОЙ ИММ

### 4.1. Постановка задачи

Задание. Разработать имитационную информационно-математическую модель деятельности производственного предприятия согласно следующим требованиям:

1. В состав ИИММ включить две аналитические задачи:
  - Задача планирования производства
  - Задача размещения производства
2. Хранение исходных и результативных данных осуществить в базе данных SQL Server.
3. Временной цикл планирования деятельности предприятия равен 5 периодам.
4. Доля прибыли от реализации продукции в каждом периоде (начиная со второго) направляется на реинвестирование в последующем периоде.

### 4.2. Проектирование и разработка базы данных SQL Server

Для организации хранения данных, используемых в ИИММ, необходимо создать базу данных, состоящую из следующих таблиц:

1. FINANCE. Содержит информацию о величине инвестиций в производство, прибыли, затратах, связанных с размещением производства, и доле прибыли, идущей на реинвестирование в каждом временном периоде.

Таблица 4.1. – Поля таблицы FINANCE.

| Поле      | Тип            | Ключ  | Описание                                                  |
|-----------|----------------|-------|-----------------------------------------------------------|
| ID_PERIOD | Tinyint        | PK,FK | Идентификатор периода                                     |
| INVEST    | Money          |       | Инвестиции                                                |
| PRIB      | Money          |       | Прибыль                                                   |
| REINVEST  | decimal(18, 2) |       | Доля прибыли прошлого периода, идущая на реинвестирование |

|        |       |  |                                    |
|--------|-------|--|------------------------------------|
| ZATRAT | Money |  | Затраты на размещение производства |
|--------|-------|--|------------------------------------|

2. OBJEM. Содержит сведения о минимально и максимально возможном объеме производства каждого вида продукции в каждом временном интервале.

Таблица 4.2. – Поля таблицы OBJEM.

| Поле       | Тип     | Ключ |     | Описание                        |
|------------|---------|------|-----|---------------------------------|
| ID_PRODUCT | Int     | PK   | FK1 | Идентификатор вида продукции    |
| ID_PERIOD  | Tinyint |      | FK2 | Идентификатор периода           |
| MIN_OBJEM  | Float   |      |     | Минимальный объем производства  |
| MAX_OBJEM  | Float   |      |     | Максимальный объем производства |

3. OBOUD. Содержит данные об используемых производственных мощностях (оборудовании), необходимые для реализации задачи размещения производства по производственным мощностям.

Таблица 4.3. – Поля таблицы OBOUD.

| Поле     | Тип           | Ключ | Описание                                 |
|----------|---------------|------|------------------------------------------|
| ID_OBOUD | Int           | PK   | Идентификатор производственных мощностей |
| OBOUD    | nvarchar(150) |      | Название производственной мощности       |

4. PERIOD. Содержит названия периодов, для которых будут рассчитываться показатели работы ИИММ.

Таблица 4.4. – Поля таблицы PERIOD.

| Поле      | Тип          | Ключ | Описание                            |
|-----------|--------------|------|-------------------------------------|
| ID_PERIOD | Tinyint      | PK   | Идентификатор<br>временного периода |
| PERIOD    | nvarchar(50) |      | Название временного<br>периода      |

*Примечание. В предложенной модели значение поля ID\_PERIOD представляет собой массив чисел {1,2,3,4,5}. В запросах к базе данных используется именно эти значения. Если значения в данном поле отличаются от указанных, то необходимо перестроить запросы.*

5. PLAN\_PRODUCT. В таблице хранится рассчитанный план производства продукции каждого вида в каждом временном периоде.

Таблица 4.5. – Поля таблицы PLAN\_PRODUCT.

| Поле         | Тип     | Ключ |     | Описание                        |
|--------------|---------|------|-----|---------------------------------|
| ID_PRODUCT   | Int     | PK   | FK1 | Идентификатор<br>вида продукции |
| ID_PERIOD    | Tinyint |      | FK2 | Идентификатор<br>периода        |
| PLAN_PRODUCT | Float   |      |     | План<br>производства            |

6. PLAN\_RESURS. В таблице хранится рассчитанный план закупки материальных ресурсов каждого вида в каждом временном периоде.

Таблица 4.6. – Поля таблицы PLAN\_RESURS.

| Поле      | Тип | Ключ |     | Описание                                          |
|-----------|-----|------|-----|---------------------------------------------------|
| ID_RESURS | Int | PK   | FK1 | Идентификатор<br>вида<br>материального<br>ресурса |

|             |         |  |     |                                    |
|-------------|---------|--|-----|------------------------------------|
| ID_PERIOD   | Tinyint |  | FK2 | Идентификатор периода              |
| PLAN_RESURS | Float   |  |     | План закупки материального ресурса |

7. PRODUCT. Содержит данные о видах продукции.

Таблица 4.7. – Поля таблицы PRODUCT.

| Поле       | Тип           | Ключ | Описание                     |
|------------|---------------|------|------------------------------|
| ID_PRODUCT | int           | PK   | Идентификатор вида продукции |
| PRODUCT    | nvarchar(150) |      | Название продукции           |

8. PRODUCT\_PRICE. Таблица содержит данные о ценах на продукцию каждого вида в каждом временном интервале.

Таблица 4.8. – Поля таблицы PRODUCT\_PRICE.

| Поле          | Тип     | Ключ |     | Описание                     |
|---------------|---------|------|-----|------------------------------|
| ID_PRODUCT    | Int     | PK   | FK1 | Идентификатор вида продукции |
| ID_PERIOD     | Tinyint |      | FK2 | Идентификатор периода        |
| PRODUCT_PRICE | Money   |      |     | Цена за единицу продукции    |

9. RASHOD. Содержит технологические матрицы, т.е. величину расхода материальных ресурсов каждого вида на производство единицы продукции каждого вида в каждом временном периоде. Наличие в таблице поля «ID\_PERIOD» предполагает, что технологическая матрица в каждом периоде может измениться.

Таблица 4.9. – Поля таблицы RASHOD.

| Поле       | Тип | Ключ |     | Описание                     |
|------------|-----|------|-----|------------------------------|
| ID_PRODUCT | Int | PK   | FK1 | Идентификатор вида продукции |

|           |         |  |     |                                                                            |
|-----------|---------|--|-----|----------------------------------------------------------------------------|
| ID_RESURS | Int     |  | FK2 | Идентификатор<br>вида материального<br>ресурса                             |
| ID_PERIOD | Tinyint |  | FK3 | Идентификатор<br>периода                                                   |
| RASHOD    | Float   |  |     | Расход<br>материального<br>ресурса на<br>производство<br>единицы продукции |

10.RESURS. Содержит данные о видах материальных ресурсов.

Таблица 4.10. – Поля таблицы RESURS.

| Поле      | Тип           | Ключ | Описание                                  |
|-----------|---------------|------|-------------------------------------------|
| ID_RESURS | Int           | PK   | Идентификатор<br>материального<br>ресурса |
| RESURS    | nvarchar(150) |      | Наименование<br>материального<br>ресурса  |

11.RESURS\_PRICE. Таблица содержит данные о ценах на материальные ресурсы каждого вида в каждом временном интервале.

Таблица 4.11. – Поля таблицы RESURS\_PRICE.

| Поле         | Тип     | Ключ |     | Описание                                  |
|--------------|---------|------|-----|-------------------------------------------|
| ID_RESURS    | Int     | PK   | FK1 | Идентификатор<br>материального<br>ресурса |
| ID_PERIOD    | Tinyint |      | FK2 | Идентификатор<br>периода                  |
| RESURS_PRICE | Money   |      |     | Цена<br>материального<br>ресурса          |



12.TEP. Содержит сведения о технико-экономических показателях производства продукции.

Таблица 4.12. – Поля таблицы TEP.

| Поле     | Тип           | Ключ | Описание                                                                                                     |
|----------|---------------|------|--------------------------------------------------------------------------------------------------------------|
| ID_TEP   | Int           | PK   | Идентификатор технико-экономического показателя                                                              |
| TEP      | nvarchar(150) |      | Наименование ТЭП                                                                                             |
| TEP_TYPE | Int           |      | Тип ТЭП. Тип определяет знак в системе ограничений целевой функции.<br>-1 – знак «<=»<br>0 – «=»<br>1 – «>=» |

13.TEP\_EFFECT. В таблице хранятся единичные эффективности производства каждого вида продукции в каждом периоде по каждому технико-экономическому показателю.

Таблица 4.13. – Поля таблицы TEP\_EFFECT.

| Поле       | Тип     | Ключ |     | Описание                                        |
|------------|---------|------|-----|-------------------------------------------------|
| ID_TEP     | Int     | PK   | FK1 | Идентификатор технико-экономического показателя |
| ID_PRODUCT | Int     |      | FK2 | Идентификатор вида продукции                    |
| ID_PERIOD  | Tinyint |      | FK3 | Идентификатор периода                           |
| TEP_EFFECT | Float   |      |     | Единичная эффективность производства продукции. |

14.TEP\_NORMA. Содержит данные о заранее определенном уровне каждого ТЭП в каждом периоде.

Таблица 4.14. – Поля таблицы TEP\_NORMA.

| Поле      | Тип     | Ключ |     | Описание                                        |
|-----------|---------|------|-----|-------------------------------------------------|
| ID_TEP    | Int     | PK   | FK1 | Идентификатор технико-экономического показателя |
| ID_PERIOD | Tinyint |      | FK2 | Идентификатор периода                           |
| TEP_NORMA | Float   |      |     | Уровень ТЭП                                     |

15.ZAGRUZ. В таблице хранится информация о технико-экономических показателях загрузки каждой единицы оборудования (производственной мощности) при производстве единицы продукции каждого вида в каждом временном периоде, а также план загрузки производственных мощностей.

Таблица 4.15. – Поля таблицы ZAGRUZ.

| Поле         | Тип     | Ключ |     | Описание                                      |
|--------------|---------|------|-----|-----------------------------------------------|
| ID_OBORUD    | Int     | PK   | FK1 | Идентификатор производственной мощности       |
| ID_PRODUCT   | Int     |      | FK2 | Идентификатор вида продукции                  |
| ID_PERIOD    | Tinyint |      | FK3 | Идентификатор периода                         |
| NORMA_ZAGRUZ | Float   |      |     | Показатель загрузки производственной мощности |
| PLAN_ZAGRUZ  | Float   |      |     | План загрузки производственной мощности       |

16.ZAGRUZ\_NORMA. Содержатся данные о нормах загрузки каждой единицы производственных мощностей в каждом временном интервале.

Таблица 4.16. – Поля таблицы ZAGRUZ\_NORMA.

| Поле         | Тип     | Ключ |     | Описание                                 |
|--------------|---------|------|-----|------------------------------------------|
| ID_OBORUD    | Int     | PK   | FK1 | Идентификатор производственной мощности  |
| ID_PERIOD    | Tinyint |      | FK2 | Идентификатор периода                    |
| ZAGRUZ_NORMA | Float   |      |     | Норма загрузки производственной мощности |

17.ZAPAS. Содержит данные о величине запаса материальных ресурсов каждого вида в каждом временном интервале.

Таблица 4.17. – Поля таблицы ZAPAS.

| Поле      | Тип     | Ключ |     | Описание                              |
|-----------|---------|------|-----|---------------------------------------|
| ID_RESURS | Int     | PK   | FK1 | Идентификатор материального ресурса   |
| ID_PERIOD | Tinyint |      | FK2 | Идентификатор периода                 |
| ZAPAS     | Float   |      |     | Величина запаса материального ресурса |

Таким образом, диаграмма базы данных примет вид, представленный на рисунке 4.1.

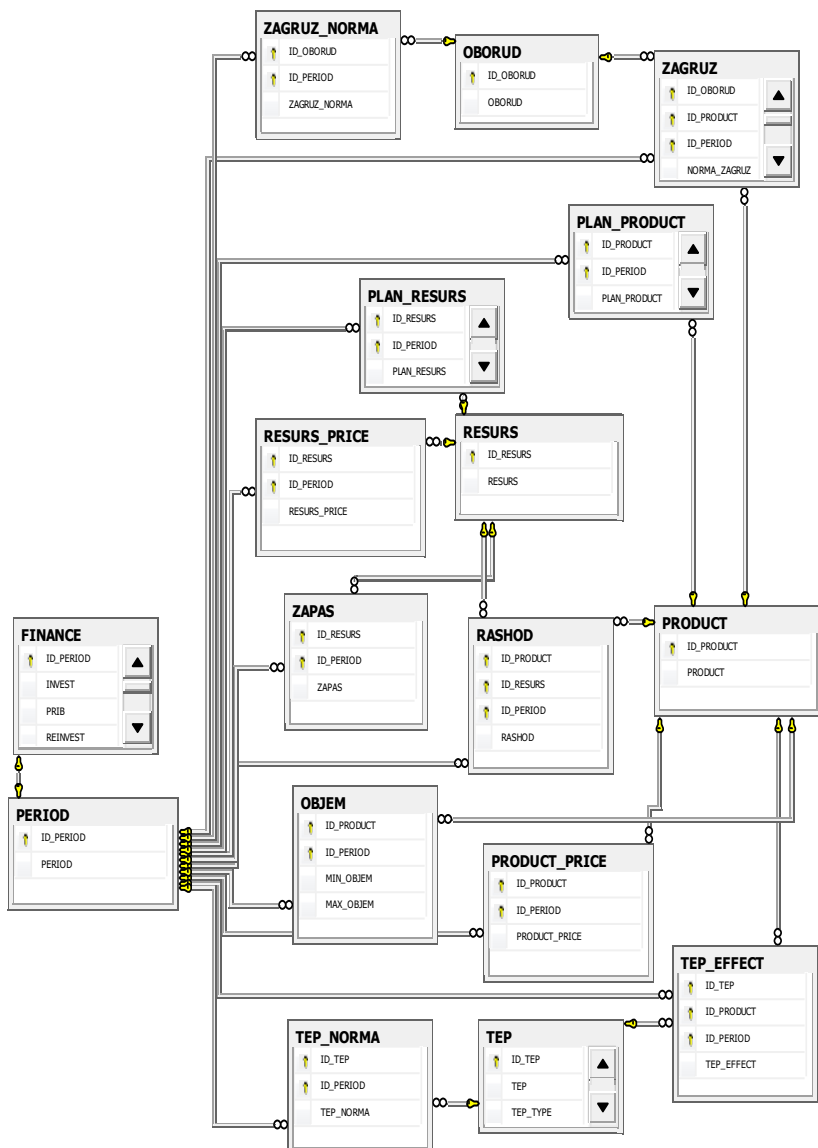


Рисунок 4.1. – Диаграмма базы данных

### 4.3. Программирование m-функций решения задач, входящих в состав ИИММ

Решение каждой из аналитических задач, указанных в требованиях к разработке ИИММ (см. п. 4.1.), предполагает их «разбиение» на подзадачи. Эти подзадачи оформлены в виде следующих m-функций:

1. Функция ProductCount. Возвращает количество видов продукции.

```
function C = ProductCount()
    if nargin > 0
        error('Функция не имеет входных параметров');
    end
    if nargout ~= 1
        error('Должен быть только 1 выходной параметр');
    end

    conn = database('imm','','');
    q = strcat('SELECT COUNT(*) FROM PRODUCT');
    C = cell2mat(fetch(conn,q));
end
```

Примечание.

Алгоритм работы функции следующий:

- Выполняется проверка на количество входных и выходных параметров функции.
- Осуществляется подключение к базе данных, подготовка запроса и выполнение его, преобразование полученных результатов запроса к значениям необходимого типа данных.

*По приведенному алгоритму работы данной функции работают и функции 2-4, представленные ниже.*

2. Функция ResursCount. Возвращает количество видов материальных ресурсов, расходуемых при производстве продукции.

```
function C = ResursCount()
    if nargin > 0
        error('Функция не имеет входных параметров');
    end
    if nargin ~= 1
        error('Должен быть только 1 выходной параметр');
    end

    conn = database('imm','','');
    q = strcat('SELECT COUNT(*) FROM RESURS');
    C = cell2mat(fetch(conn,q));
end
```

3. Функция TepCount. Возвращает количество технико-экономических показателей.

```
function C = TepCount()
    if nargin > 0
        error('Функция не имеет входных параметров');
    end
    if nargin ~= 1
        error('Должен быть только 1 выходной параметр');
    end

    conn = database('imm','','');
    q = strcat('SELECT COUNT(*) FROM TEP');
    C = cell2mat(fetch(conn,q));
end
```

4. Функция `OborudCount`. Возвращает количество единиц производственных мощностей.

```
function C = OborudCount()
    if nargin > 0
        error('Функция не имеет входных параметров');
    end
    if nargout ~= 1
        error('Должен быть только 1 выходной параметр');
    end

    conn = database('imm','','');
    q = strcat('SELECT COUNT(*) FROM OBORUD');
    C = cell2mat(fetch(conn,q));
end
```

5. Функция `Invest`. Функция возвращает объем инвестиций в производство в каждом периоде.

```
function Q = Invest(T)
    if nargin ~= 1
        error('Должен быть только 1 входной параметр');
    end
    if nargout ~= 1
        error('Должен быть только 1 выходной параметр');
    end

    conn = database('imm','','');
    q = strcat('SELECT INVEST FROM FINANCE WHERE ID_PERIOD = ', int2str(T));
    Res = cell2mat(fetch(conn,q));

    if length(Res) == 1
        Q = Res;
    end
```

```

else
    error('Не задан объем инвестиций. ');
end
end
end

```

Примечание.

Алгоритм работы функции следующий:

- Выполняется проверка на количество входных и выходных параметров функции.
- Осуществляется подключение к базе данных, подготовка запроса и выполнение его, преобразование полученных результатов запроса к значениям необходимого типа данных.
- Выполняется проверка полученных результатов на их размерности.

В приведенной функции SQL-запрос к базе данных записывается в переменную «q». Сам текст запроса представляет собой объединение двух строк посредством функции *strcat*.

- 'SELECT INVEST FROM FINANCE WHERE ID\_PERIOD = '
- Int2str(T). В этой конструкции происходит преобразование целочисленного формального параметра «Т» в переменную строкового типа.

В результате объединения строк в одну, SQL-запрос при вызове функции, например:

» Q = Invest(3)

примет вид:

```

q = 'SELECT INVEST FROM FINANCE WHERE
ID_PERIOD = 3'

```

Данный запрос следует читать так: «Получить значение из поля «INVEST» таблицы «FINANCE» для записи, в которой значение поля «ID\_PERIOD» равно 3».



После выполнения данного запроса `fetch(conn,q)` происходит преобразование полученного результата при помощи функции `cell2mat` (см. п. 2.7.).

На последнем шаге осуществляется проверка полученного из базы данных значения на его размерность. Так ожидалось получение из базы данных скалярной величины, то проверяется равенство его размерности единицы. В случае, если размерность полученного из базы данных значения равна единице, то это значение передается в выходную переменную функции.

*По приведенному алгоритму работы функции Invest работают и остальные функции, представленные ниже.*

6. Функция `Objem`. Функция возвращает два вектора - минимальный и максимальный план производства.

```
function [d,D] = Objem(T)
    if nargin ~= 1
        error('Должен быть только 1 входной параметр');
    end
    if narginout ~= 2
        error('Должно быть 2 выходных параметра');
    end

    pCount = ProductCount();

    conn = database('imm','','');
    q = strcat('SELECT MIN_OBJEM, MAX_OBJEM FROM',
               'OBJEM WHERE ID_PERIOD = ',
               int2str(T),'ORDER BY',
               'ID_PRODUCT');
    Res = cell2mat(fetch(conn,q));

    if length(Res) == pCount
        d = Res(:,1);
        D = Res(:,2);
```

```

else
    error('Не все значения заданы в выбранном
          периоде. ');
end
end

```

7. Функция ProductPrice. Функция возвращает вектор цен на продукцию.

```

function C = ProductPrice(T)
    if nargin ~= 1
        error('Должен быть только 1 входной
              параметр');
    end
    if nargout ~= 1
        error('Должен быть только 1 выходной
              параметр');
    end

    pCount = ProductCount();

    conn = database('imm','','');
    q = strcat('SELECT PRODUCT_PRICE FROM
              PRODUCT_PRICE WHERE ID_PERIOD=',
              int2str(T), 'ORDER BY
              ID_PRODUCT');
    Res = cell2mat(fetch(conn,q));

    if length(Res) == pCount
        C = Res;
    else
        error('Не все цены на продукты заданы в
              выбранном периоде. ');
    end
end

```

8. Функция Rashod. Функция возвращает технологическую матрицу

```

function A = Rashod(T)
    if nargin ~= 1
        error('Должен быть только 1 входной параметр');
    end
    if nargsout ~= 1
        error('Должен быть только 1 выходной параметр');
    end

    pCount = ProductCount();
    rCount = ResursCount();

    conn = database('imm','','');
    q = strcat('SELECT RASHOD FROM RASHOD WHERE
                ID_PERIOD = ', int2str(T), 'ORDER
                BY ID_PRODUCT, ID_RESURS');
    Res = cell2mat(fetch(conn,q));

    if length(Res) == pCount * rCount
        A = reshape(Res,rCount,pCount);
    else
        error('Не все коэффициенты технологической
              матрицы заданы в выбранном
              периоде. ');
    end
end
end

```

9. Функция ResursPrice. Возвращает вектор цен на материальные ресурсы.

```

function S = ResursPrice(T)
    if nargin ~= 1
        error('Должен быть только 1 входной параметр');
    end
    if nargsout ~= 1

```

```

        error('Должен быть только 1 выходной
                параметр');
    end
    rCount = ResursCount();
    conn = database('imm','','');
    q = strcat('SELECT RESURS_PRICE FROM
                RESURS_PRICE WHERE ID_PERIOD = ',
                int2str(T),'ORDER BY
                ID_RESURS');
    Res = cell2mat(fetch(conn,q));

    if length(Res) == rCount
        S = Res;
    else
        error('Не все цены на ресурсы заданы в
                выбранном периоде. ');
    end
end
end

```

**10. Функция TepEffect.** Функция возвращает матрицу единичных эффективностей производства.

```

function f = TepEffect(T)
    if nargin ~= 1
        error('Должен быть только 1 входной
                параметр');
    end
    if narginout ~= 1
        error('Должен быть только 1 выходной
                параметр');
    end
    pCount = ProductCount();
    tCount = TepCount();
    conn = database('imm','','');
    q = strcat('SELECT TEP_EFFECT FROM
                TEP_EFFECT WHERE ID_PERIOD = ',
                int2str(T),'ORDER BY ID_PRODUCT,
                ID_TEP');

```

```

Res = cell2mat(fetch(conn,q));

if length(Res) == pCount * tCount
    f = reshape(Res,tCount,pCount);
else
    error('Не все коэффициенты матрицы ТЭП
          заданы в выбранном периоде.');
```

end

end

**11. Функция TepNorma.** Функция возвращает вектор  
заранее определенного уровня ТЭП

```

function [F,TT] = TepNorma(T)
    if nargin ~= 1
        error('Должен быть только 1 входной
              параметр');
```

end

```

    if nargsout ~= 2
        error('Должно быть 2 выходных
              параметра');
```

end

```

    tCount = TepCount();

    conn = database('imm','','');
    q = strcat('SELECT TEP_NORMA, TEP_TYPE FROM
               TEP_NORMA TN INNER JOIN TEP T ON
               T.ID_TEP = TN.ID_TEP WHERE
               ID_PERIOD = ', int2str(T),
               'ORDER BY TN.ID_TEP');
```

Res = cell2mat(fetch(conn,q));

```

    if length(Res) == tCount
        F = Res(:,1);
        TT = Res(:,2);
    else
        error('Не все нормы ТЭП заданы в выбранном
              периоде.');
```

```

end
end

```

**12. Функция Zagruz.** Функция возвращает матрицу коэффициентов загрузки оборудования

```

function U = Zagruz(T)
    if nargin ~= 1
        error('Должен быть только 1 входной параметр');
    end
    if narginout ~= 1
        error('Должен быть только 1 выходной параметр');
    end

    pCount = ProductCount();
    oCount = OborudCount();

    conn = database('imm','','');
    q = strcat('SELECT NORMA_ZAGRUZ FROM ZAGRUZ
                WHERE ID_PERIOD = ', int2str(T),
                'ORDER BY ID_PRODUCT,
                ID_OBORUD');
    Res = cell2mat(fetch(conn,q));

    if length(Res) == pCount * oCount
        U = reshape(Res,oCount,pCount);
    else
        error('Не все коэффициенты матрицы
                загрузки оборудования заданы в
                выбранном периоде. ');
    end
end

```

**13. Функция ZagruzNorma.** Функция возвращает вектор нормы загрузки оборудования

```

function K = ZagruzNorma(T)
    if nargin ~= 1
        error('Должен быть только 1 входной параметр');
    end
    if nargout ~= 1
        error('Должен быть только 1 выходной параметр');
    end

    oCount = OborudCount();

    conn = database('imm','','');
    q = strcat('SELECT ZAGRUZ_NORMA FROM
                ZAGRUZ_NORMA WHERE ID_PERIOD = ',
                int2str(T),'ORDER BY ID_OBORUD');
    Res = cell2mat(fetch(conn,q));

    if length(Res) == oCount
        K = Res;
    else
        error('Не все нормы загрузки оборудования заданы в выбранном периоде.');
```

**14. Функция Zapas.** Функция возвращает вектор запаса материальных ресурсов

```

function B = Zapas(T)
    if nargin ~= 1
        error('Должен быть только 1 входной параметр');
    end
    if nargout ~= 1
        error('Должен быть только 1 выходной параметр');
    end
end
```

```

rCount = ResursCount;

conn = database('imm','','');
q = strcat('SELECT ZAPAS FROM ZAPAS WHERE
           ID_PERIOD = ', int2str(T), 'ORDER
           BY ID_RESURS');
Res = cell2mat(fetch(conn,q));

if length(Res) == rCount
    B = Res;
else
    error('Не все запасы ресурсов заданы в
          выбранном периоде. ');
end
end

```

Для решения аналитических задач, входящих в ИИММ, используются две m-функции:

1. Функция PlanProizv. Функция возвращает оптимальный план производства, план закупки материальных ресурсов и прибыль от реализации продукции. Также функция осуществляет запись полученных результатов в базу данных.

```

function [X,Y,P] = PlanProizv(T)
    if nargin ~= 1
        error('Должен быть только 1 входной
              параметр');
    end
    if narginout ~= 3
        error('Должно быть 3 выходных
              параметра');
    end
end

```

*Примечание. В последующих строках происходит загрузка из базы данных необходимых для решения задачи массивов*



```

C = ProductPrice(T);
S = ResursPrice(T);
A = Rashod(T);
B = Zapas(T);
[d,D] = Objem(T);
[F,TT] = TepNorma(T);
f = TepEffect(T);
Q = Invest(T);

```

*Примечание. Для второго, третьего и т.д. периода необходимо увеличить сумму инвестиций в производство на величину реинвестированной прибыли предыдущего периода. Поэтому запрос, указанный ниже, позволяет получить из базы данных долю прибыли прошлого периода, направленную на реинвестирование, и на это значение увеличить запланированные инвестиции.*

```

if T > 1
    conn = database('imm','','');
    q = strcat('SELECT ISNULL(FLOOR(PRIB *
        REINVEST),0) AS REINVEST FROM
        FINANCE WHERE ID_PERIOD =
        ',int2str(T - 1));
    Res = cell2mat(fetch(conn,q));
    Q = Q + Res;
end

ProductCount = length(C);
ResursCount = length(S);
TepCount = length(F);

func = [-C; S];

```

*Примечание. В нижеследующем цикле происходит «разделение» матриц единичных эффе*

*производства и заранее определенного уровня ТЭП по типам ТЭП.*

```
f1 = []; f2 = []; f3 = [];  
F1 = []; F2 = []; F3 = [];  
  
for k = 1:1:TepCount  
    if TT(k) == -1  
        f1 = [f1;f(k,:)];  
        F1 = [F1;F(k)];  
    end  
    if TT(k) == 0  
        f2 = [f2;f(k,:)];  
        F2 = [F2;F(k)];  
    end  
    if TT(k) == 1  
        f3 = [f3;f(k,:)];  
        F3 = [F3;F(k)];  
    end  
end  
end
```

*Примечание. В нижеследующем фрагменте кода осуществляется подготовка параметров для функции linprog и вызов этой функции.*

```
a = [A -eye(ResursCount);  
     f1 zeros(length(F1),ResursCount);  
     -f3 zeros(length(F3),ResursCount);  
     zeros(1,ProductCount) S'];  
b = [B;F1;-F3;Q];  
aeq = [f2 zeros(length(F2),ResursCount)];  
beq = F2;  
lb = [d;zeros(ResursCount,1)];  
ub = [D;inf(ResursCount,1)];  
[Plan, Prib, ExitFlag]=  
    linprog(func,a,b,aeq,beq,lb,ub);
```

*Примечание. В зависимости от того, удалось ли функции linprog оптимальное решение, осуществляется запись в базу данных этого решения.*

```
if ExitFlag < 0
    error('Решение не найдено');
else
    X = Plan(1 : ProductCount, 1);
    Y = Plan(ProductCount+1 : length(Plan), 1);
    P = - Prib;

    try
```

*Примечание. В нижеследующем фрагменте кода происходит удаление «старых» планов производства продукции и закупки материальных ресурсов.*

```
    conn = database('imm','','');
    exec(conn, strcat('DELETE FROM
        PLAN_PRODUCT WHERE ID_PERIOD =
        ', int2str(T)));
    exec(conn, strcat('DELETE FROM
        PLAN_RESURS WHERE ID_PERIOD =
        ', int2str(T)));
```

*Примечание. Для корректной записи планов производства продукции и закупки материальных ресурсов, необходимо получить вектор идентификаторов видов продукции и ресурсов.*

```
    q = strcat('SELECT ID_PRODUCT FROM
        PRODUCT');
    Res = cell2mat(fetch(conn, q));
    Product = Res;
    q = strcat('SELECT ID_RESURS FROM
        RESURS');
    Res = cell2mat(fetch(conn, q));
    Resurs = Res;
```

*Примечание. В нижеследующих циклах осуществляется запись плана производства продукции и закупки материальных ресурсов в базу данных.*

```
for k = 1:1:ProductCount
    q = strcat('INSERT INTO PLAN_PRODUCT
    (ID_PRODUCT, ID_PERIOD, PLAN_PRODUCT)
    VALUES(',int2str(Product(k)),',',',
    int2str(T),',',',num2str(X(k)),',')');
    exec(conn,q);
end
for k = 1:1:ResursCount
    q = strcat('INSERT INTO PLAN_RESURS
    (ID_RESURS, ID_PERIOD, PLAN_RESURS)
    VALUES(',int2str(Resurs(k)),',',',
    int2str(T),',',',num2str(Y(k)),',')');
    exec(conn,q);
end
```

*Примечание. Происходит запись в базу данных значение прибыли, полученной в текущем временном интервале.*

```
q = strcat('UPDATE FINANCE SET PRIB =
',num2str(P),'WHERE ID_PERIOD = ',
int2str(T));
exec(conn,q);
```

*Примечание. Выдается сообщение об ошибке в случае, если какая-либо операция записи в базу данных завершилась некорректно.*

```
catch me
    error('Ошибка при подключении к базе
    данных');
end
end
end
```

2. Функция PlanRazm. Функция возвращает оптимальный план размещения производства по производственным мощностям.

```
function [X,P] = PlanRazm(T,Plan)
    if nargin ~= 2
        error('Должен быть 2 входных параметра');
    end
    if nargout ~= 2
        error('Должно быть 2 выходных параметра');
    end
```

*Примечание. В последующих строках происходит загрузка из базы данных необходимых для решения задачи массивов*

```
K = ZagruzNorma(T);
U = Zagruz(T);
```

```
ProductCount = length(Plan);
OborudCount = length(K);
```

*Примечание. В последующей строке кода происходит преобразование матрицы единичных эффективностей производства в вектор.*

```
func=reshape(U',1,ProductCount*OborudCount);
```

*Примечание. В последующем цикле осуществляется формирование матриц для функции linprog.*

```
a = [];
aeq = [];

for i = 1:1:OborudCount
    b = [];
    for j = 1:1:OborudCount
        if i == j
            b = [b U(i,:)];
        end
    end
end
```

```

        else
            b = [b zeros(1,ProductCount)];
        end
    end
    a = [a;b];
    aeq = [aeq eye(ProductCount)];
end

b = ZagruzNorma(T);
beq = Plan;
lb = zeros(1,ProductCount * OborudCount);

[PlanRaz,Zatrat,ExitFlag] =
    linprog(func,a,b,aeq,beq,lb,[]);

```

*Примечание. В зависимости от того, удалось ли функции `linprog` оптимальное решение, осуществляется запись в базу данных этого решения.*

```

if ExitFlag < 0
    error('Решение не найдено');
else
    try

```

*Примечание. Для корректной записи планов размещения производства продукции, необходимо получить вектор идентификаторов видов продукции и производственных мощностей.*

```

conn = database('imm','','');
q = strcat('SELECT ID_PRODUCT FROM
            PRODUCT');
Res = cell2mat(fetch(conn,q));
Product = Res;
q = strcat('SELECT ID_OBORUD FROM
            OBORUD');
Res = cell2mat(fetch(conn,q));
Oborud = Res;

```

*Примечание. В нижеследующих циклах осуществляется запись плана размещения производства продукции в базу данных.*

```

k = 1;
for i = 1:1:OborudCount
    for j = 1:1:ProductCount
        q = strcat('UPDATE ZAGRUI SET
            PLAN_ZAGRUI=', num2str(PlanRaz(k)),
            'WHERE ID_PERIOD = ', int2str(T), '
            AND ID_OBORUD = ',
            int2str(Oborud(i)),
            'AND ID_PRODUCT = ',
            int2str(Product(j)));
        exec(conn, q);
        k = k + 1;
    end
end

```

*Примечание. Происходит запись в базу данных значения затрат на размещение производства, полученных в текущем временном интервале.*

```

q = strcat('UPDATE FINANCE SET ZATRAT =
    ', num2str(Zatrat), 'WHERE ID_PERIOD =
    ', int2str(T));
exec(conn, q);
catch me
    error('Ошибка при подключении к базе
        данных');
end

X=reshape(PlanRaz, ProductCount, OborudCount) '
P = Zatrat;
end
end

```

В имитационную информационно-математическую модель следует также включить функции для отображения результатов моделирования. Такими функциями являются:

1. gDynFinance. Функция строит график динамики изменения инвестиций и прибыли.

```
function gDynFinance()
    if nargin ~= 0
        error('Функция не имеет входных параметров');
    end

    conn = database('imm','','');
    q = strcat('select * from FINANCE order by ID_PERIOD');
    Res = fetch(conn,q);
    x = cell2mat(Res(:,1));
    Inv = cell2mat(Res(:,2));
    Prib = cell2mat(Res(:,3));

    plot(x,Inv,'-o r',x,Prib,'-* b');
    title('Динамика инвестиций и прибыли');
    colormap jet;
end
```

2. gDynOborud. Функция строит диаграмму загрузки оборудования.

```
function gDynOborud(Oborud)
    if nargin ~= 1
        error('Должен быть только 1 входной параметр');
    end

    conn = database('imm','','');
    q = strcat('select Z.ID_OBORUD, ID_PERIOD, sum(PLAN_ZAGRUZ) as PLAN_ZAGRUZ from
```



```

        ZAGRUZ Z INNER JOIN OBORUD O ON
        O.ID_OBORUD = Z.ID_OBORUD where O.OBORUD
        = '',Oborud,'' GROUP BY
        Z.ID_OBORUD,ID_PERIOD ORDER BY
        Z.ID_PERIOD');
Res = fetch(conn,q);
dan = cell2mat(Res(:,3));

bar(dan);
title(strcat('Динамика загрузки
              оборудования: ',Oborud));
colormap jet;
end

```

3. gDynProizv. Функция строит диаграмму динамики производства выбранного вида продукции.

```

function gDynProizv(Product)
    if nargin ~= 1
        error('Должен быть только 1 входной
              параметр');
    end

    conn = database('imm','','');
    q = strcat('SELECT P.PRODUCT,
        PP.PLAN_PRODUCT,O.MIN_OBJEM,O.MAX_OBJEM
        FROM PLAN_PRODUCT PP INNER JOIN PRODUCT
        P ON P.ID_PRODUCT = PP.ID_PRODUCT INNER
        JOIN OBJEM O ON O.ID_PRODUCT =
        PP.ID_PRODUCT AND O.ID_PERIOD =
        PP.ID_PERIOD WHERE P.PRODUCT =
        '',Product,'' ORDER BY PP.ID_PERIOD');
    Res = fetch(conn,q);
    dan = cell2mat(Res(:,2));
    min = cell2mat(Res(:,3));
    max = cell2mat(Res(:,4));

    bar([min,dan,max]);

```

```

legend('Минимальный объем', 'План
        производства', 'Максимальный план', 0);
title(strcat('Динамика плана производства:
        ', Product));
colormap jet;
end

```

4. gDynResurs. Функция строит диаграмму динамики закупки выбранного вида материального ресурса.

```

function gDynResurs(Resurs)
    if nargin ~= 1
        error('Должен быть только 1 входной
                параметр');
    end

    conn = database('imm', '', '');
    q = strcat('SELECT R.RESURS, PR.PLAN_RESURS
        FROM PLAN_RESURS PR INNER JOIN RESURS R
        ON R.ID_RESURS = PR.ID_RESURS WHERE
        R.RESURS = ''', Resurs, '''' ORDER BY
        PR.ID_PERIOD');
    Res = fetch(conn, q);
    dan = cell2mat(Res(:, 2));

    bar(dan);
    title(strcat('Динамика плана закупок:
        ', Resurs));
    colormap jet;
end

```

5. gStructOborud. Функция строит диаграмму загрузки оборудования в выбранном периоде.

```

function gStructOborud(T)
    if nargin ~= 1
        error('Должен быть только 1 входной
                параметр');
    end

```

```

end

conn = database('imm','','');
q = strcat('SELECT O.OBORUD,
          SUM(Z.PLAN_ZAGRUZ) AS PLAN_ZAGRUZ FROM
          ZAGRUZ Z INNER JOIN OBORUD O ON
          O.ID_OBORUD = Z.ID_OBORUD WHERE
          ID_PERIOD = ',int2str(T),' GROUP BY
          O.OBORUD, Z.ID_OBORUD, ID_PERIOD');
Res = fetch(conn,q);
leg = cellstr(Res(:,1));
dan = cell2mat(Res(:,2));

pie(dan);
legend(leg,0);
title(strcat('Структура загрузки
оборудования в периоде ',int2str(T)));
colormap jet;
end

```

6. gStructProizv. Функция строит диаграмму структуры производства в выбранном периоде.

```

function gStructProizv(T)
    if nargin ~= 1
        error('Должен быть только 1 входной
              параметр');
    end

    conn = database('imm','','');
    q = strcat('SELECT PRODUCT + '' - '' +
              CAST(PLAN_PRODUCT AS VARCHAR(20)) AS
              PRODUCT, ROUND(PLAN_PRODUCT,2) AS
              PLAN_PRODUCT FROM PLAN_PRODUCT PP INNER
              JOIN PRODUCT P ON P.ID_PRODUCT =
              PP.ID_PRODUCT WHERE ID_PERIOD = ',
              int2str(T));
    Res = fetch(conn,q);

```

```

leg = cellstr(Res(:,1));
dan = cell2mat(Res(:,2));

pie(dan);
legend(leg,0);
title(strcat('Структура производства в
              периоде ',int2str(T)));
colormap jet;
end

```

7. gStructResurs. Функция строит диаграмму структуры закупки материальных ресурсов в выбранном периоде.

```

function gStructResurs(T)
    if nargin ~= 1
        error('Должен быть только 1 входной
              параметр');
    end

    conn = database('imm','','');
    q = strcat('SELECT RESURS + '' - '' +
              CAST(PLAN_RESURS AS VARCHAR(20)) AS
              RESURS, ROUND(PLAN_RESURS,2) AS
              PLAN_RESURS FROM PLAN_RESURS PR INNER
              JOIN RESURS R ON R.ID_RESURS =
              PR.ID_RESURS WHERE ID_PERIOD = ',
              int2str(T));
    Res = fetch(conn,q);
    leg = cellstr(Res(:,1));
    dan = cell2mat(Res(:,2));

    pie(dan);
    legend(leg,0);
    title(strcat('Структура закупок ресурсов в
                  периоде ',int2str(T)));
    colormap jet;
end

```

#### 4.4. Разработка временной динамики работы ИИММ в среде SIMULINK

Среда SIMULINK позволяет построить имитационную информационно-математическую модель в наглядной форме при помощи графических блоков библиотеки.

Для реализации имитационной ИММ в SIMULINK необходимо разместить 3 блока:

1. Блок Clock. Данный блок будет последовательно генерировать натуральные числа, начиная с 1 до момента остановки модели. Данный сигнал можно интерпретировать как период моделирования.

2. Блок Level-2 M-file S-Function. Данный блок позволит включить в модель пользовательскую s-функцию, которая будет решать задачу планирования производства.

3. Блок Level-2 M-file S-Function. Данный блок позволит включить в модель пользовательскую s-функцию, которая будет решать задачу размещения производства.

Схематично информационные потоки между блоками модели можно изобразить следующим образом:

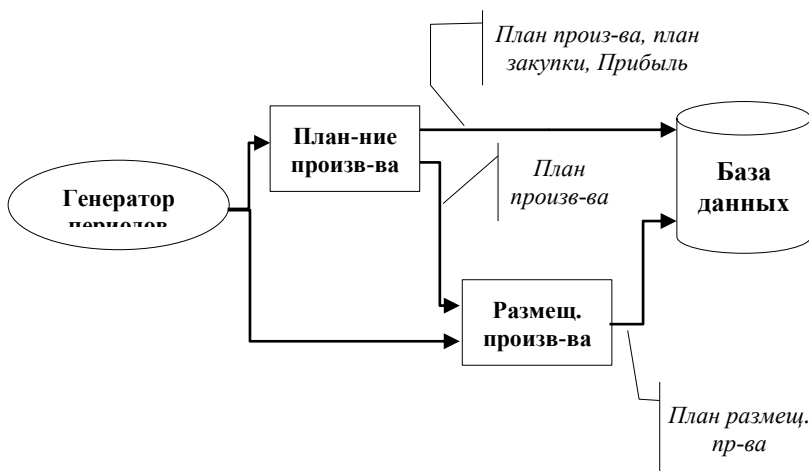


Рисунок 4.2. Схема информационных потоков ИИММ

Согласно приведенной схеме ИИММ в среде SIMULINK может быть представлена в виде:

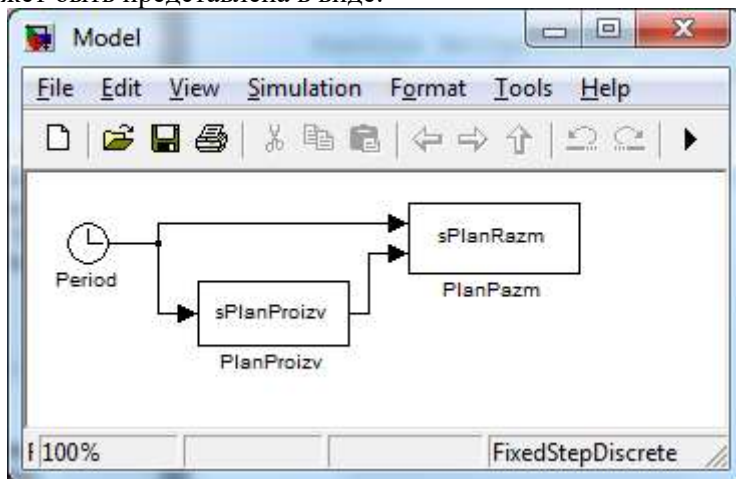


Рисунок 4.3. – ИИММ в среде SIMULINK

Для включения в модель m-функций, решающих задачи планирования производства и размещения производства, необходимо открыть окно настройки параметров блоков Level-2 M-file S-Function и в поле M-File Name ввести имя m-файла, содержащего необходимую s-функцию.

В модели используются 2 s-функции:

1. Функция sPlanProizv. Функция возвращает оптимальный план производства, план закупки материальных ресурсов и прибыль от реализации продукции для их использования в SIMULINK.

```
function sPlanProizv( block )
    Setup(block) ;
```

```
function Setup(block)
    block.NumDialogPrms = 0;
    block.NumInputPorts = 1;
    block.NumOutputPorts = 1;
```

```

block.SetPreCompInpPortInfoToDynamic;
block.SetPreCompOutPortInfoToDynamic;

pCount = ProductCount();
block.OutputPort(1).DatatypeID = 0;
block.OutputPort(1).Dimensions = pCount;

block.InputPort(1).DatatypeID = 0;

block.RegBlockMethod('Outputs', @Output);

block.RegBlockMethod('SetInputPortDimensions',
@SetInpPortDims);

function SetInpPortDims(block, idx, di)
    block.InputPort(idx).Dimensions = di;

```

*Примечание. В приведенной ниже функции в качестве входного аргумента функции PlanProizv указано «block.InputPort(1).Data». Это означает, что в функцию PlanProizv передается значение сигнала, поступающего на первый входной порт блока. Таким сигналом является значение периода моделирования.*

```

function Output(block)
    [X,Y,P]=PlanProizv(block.InputPort(1).Data);
    block.OutputPort(1).Data = X;

```

2. Функция sPlanRazm. Функция возвращает оптимальный план размещения производства по производственным мощностям для его использования в SIMULINK.

```

function sPlanRazm( block )
    Setup(block);

function Setup(block)
    block.NumDialogPrms = 0;
    block.NumInputPorts = 2;

```

```

block.NumOutputPorts = 0;

block.SetPreCompInpPortInfoToDynamic;
block.SetPreCompOutPortInfoToDynamic;

block.InputPort(1).DatatypeID = 0;
block.InputPort(2).DatatypeID = 0;

block.RegBlockMethod('Outputs', @Output);

block.RegBlockMethod('SetInputPortDimensions',
@SetInpPortDims);

function SetInpPortDims(block, idx, di)
    block.InputPort(idx).Dimensions = di;

function Output(block)
    [X,P]=PlanRazm(block.InputPort(1).Data,
        block.InputPort(2).Data);

```

*Примечание. Представленная s-функция не имеет фрагмента описания выходных параметров. Это сделано по причине того, что результаты решения задачи размещения производства нигде не используются в рамках ИИММ.*



## **5. СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ**

1. Гультяев А. Визуальное моделирование в среде MATLAB: учебный курс – СПб: Питер, 2000. – 432 с.: ил.
2. Ануфриев И.Е. Самоучитель MATLAB 5.3/6.x. – СПб.: БХВ-Петербург, 2002. – 736 с.: ил.
3. Смирнов Ю.Н. Процессно-задачный инжиниринг бизнес-процессов и стандарт управления предприятием //Научно-практический журнал «Интеграл» – 2007. - №5.- Стр.54-55.
4. Смирнов Ю.Н. Основные бизнес-задачи предприятия // Научно-практический журнал «Интеграл» – 2008. - №3.
5. Смирнов Ю.Н. О показателях эффективности бизнес-процессов предприятия //Научно-практический журнал «Интеграл» – 2008. - №5.
6. Смирнов Ю.Н. Общая и имитационная информационно-математические модели деятельности предприятия // Научно-практический журнал «Интеграл» – 2007. - № 6.
7. <http://www.matclub.ru>
8. <http://www.matlab.exponenta.ru>